



slington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CS6P05NI Final Year Project

Assessment Weightage & Type

40% FYP Final Report

Year and Semester

2024 Autumn

Network Vulnerability Assessment Tool

Student Name: Pawan Shah

London Met ID: 21040616

College ID: np01nt4a210010

Internal Supervisor: Akchayat Bikram Joshi

External Supervisor: Samir Gautam

Assignment Due Date: 24th April 2024

Assignment Submission Date: 24th April 2024

Word Count (Where Required): 9132

I confirm that I understand my coursework needs to be submitted online via my second teacher under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded

Abstract

This report outlines the development and implementation of NetGuardian, a network vulnerability assessment tool aimed at addressing the cybersecurity challenges faced by small-medium size businesses. Utilizing a structured approach based on the evolutionary methodology, the project begins with a comprehensive analysis of stakeholders' requirements and progresses through iterative stages of prototype development, client evaluation, and refinement.

NetGuardian offers essential features such as user authentication, network scanning, find vulnerability, and report generation, providing small-medium size businesses with the means to manage their network security proactively. Moreover, the report delves into the legal, ethical, and social issues that may arise from the use of NetGuardian, including considerations of privacy laws, intellectual property rights, and ethical implications of vulnerability testing.

Future work on NetGuardian will focus on enhancing capabilities such as user behavior analytics and automated vulnerability scanning to further strengthen its effectiveness in mitigating cyber threats. Through ongoing collaboration with stakeholders and adaptation to emerging technologies, NetGuardian remains committed to supporting small businesses in their journey towards enhanced cybersecurity resilience.

Table of Contents

1	Introduction	1
1.1	Project Description	2
1.2	Current Scenario	3
1.3	Problem Domain and Project as a Solution	6
1.4	Aim and Objectives	7
1.5	Structure of the Report	8
1.5.1	Background	8
1.5.2	Development	8
1.5.3	Testing and Analysis	9
1.5.4	Conclusion	9
2	Background	10
2.1	About the End Users	10
2.2	Understanding the Solution	11
2.3	Similar Projects	12
2.3.1	Nessus	12
2.3.2	Nmap	13
2.3.3	Sparta	14
2.3.4	OpenVAS (Open Vulnerability Assessment Scanner)	15
2.4	Comparisons	16
3	Development	18
3.1	Considered Methodologies	18
3.2	Selected Methodology	18
3.2.1	Evolutionary Prototyping	18
3.3	Phases Of the Methodology	20

3.4	Survey Results	21
3.4.1	Pre-Survey Results.....	21
3.4.2	Post-Survey Results	21
3.5	Requirement Analysis	22
3.6	Design.....	23
3.6.1	Mind Map.....	23
3.6.2	System Architecture	24
3.6.3	Use Case Diagram	25
3.6.4	Wireframe	25
3.6.5	Data Flow Diagrams	28
3.6.6	Flowcharts	33
3.7	Implementation.....	38
3.7.1	Database	38
3.7.2	Login GUI	38
3.7.3	Sign Up GUI	40
3.7.4	User Dashboard	41
3.7.5	Credential Testing GUI	42
3.7.6	Log Out Button	45
3.7.7	Back Button	45
3.7.8	Generate Report Button.....	46
3.7.9	Report Button	48
4	Testing and Analysis	50
4.1	Test Plan.....	50
4.1.1	Unit Testing, Test Plan	50
4.1.2	System Testing, Test Plan.....	50

4.2	Unit Testing	51
4.2.1	Evolution 1	51
4.2.1.1	Error Handling on Sign Up Button (1)	51
4.2.1.2	Error Handling on Sign Up Button (2)	52
4.2.1.3	Sign Up Button	53
4.2.1.4	Error Handling on Login Button (1)	54
4.2.1.5	Error Handling on Login Button (2)	55
4.2.1.6	Login button	56
4.2.1.7	Host Discovery Button	57
4.2.2	Evolution 2	59
4.2.2.1	Port Scanning Button	59
4.2.2.2	Find Vulnerability	60
4.2.2.3	Generate Report Button	61
4.2.2.4	Report Button	62
4.2.2.5	Error Handling on Generate Report Button	64
4.2.2.6	Log Out Button	65
4.2.3	Evolution 3	66
4.2.3.1	Credential Testing Button	66
4.2.3.2	Generate Report Button of Credential Testing Window	68
4.2.3.3	Back Button	70
4.2.3.4	Output Box	71
4.2.3.5	Error Handling for Network Address Entry	72
4.3	System Testing	73
4.3.1	Run Application	73
4.3.2	Sign Up	75
4.3.3	Login	76
4.3.4	Host Discovery	77
4.3.5	Port Scanning	79
4.3.6	Find Vulnerability	80
4.3.7	Credential Testing	81

4.3.8	Generate Report.....	82
4.3.9	Report.....	83
4.4	Critical Analysis.....	84
5	Conclusion.....	85
5.1	Legal, Social, and Ethical Issues.....	85
5.1.1	Legal Issues	85
5.1.2	Social Issues	85
5.1.3	Ethical Issues	86
5.2	Advantages	87
5.3	Limitations.....	88
5.4	Future Work	88
6	References.....	93
7	Bibliography	97
8	Appendix.....	98
8.1	Appendix A: Considered Methodology	98
8.1.1	Waterfall Methodology	98
8.1.2	Agile Methodology	99
8.2	Appendix B: Pre-Survey	100
8.2.1	Pre-Survey Form	100
8.2.2	Sample of Filled Pre-survey Forms	105
8.2.3	Pre-Survey Result	111
8.3	Appendix C: Post-Survey	119
8.3.1	Post-Survey Form.....	119
8.3.2	Sample of Filled Post-Survey Forms	123
8.3.3	Post-Survey Result.....	127

8.4	Appendix D: Sample Codes	131
8.5	Appendix E: Design.....	140
8.5.1	Sequence Diagram	140
8.5.2	Work Breakdown Structure (WBS)	144
8.5.3	Gantt Chart	145
8.6	Appendix F: Screenshots of the System	145
8.7	Appendix G: User Feedback	148
8.7.1	User Feedback Form	148
8.7.2	Sample of the Filled Feedback Forms	152
8.8	Appendix H: Future Work	157
8.8.1	Readings for the Future Work.....	157
8.9	Feature Changing Approval	159
8.10	Client Approval Letter	160

Table of Figures

Figure 1: Security and Vulnerability Management Market Statistics (Wadhwani, 2023)..	3
Figure 2: Market Size by Organization Size (Wadhwani, 2023).	4
Figure 3: Market Size By End User Industry (Wadhwani, 2023).....	5
Figure 4: US Security and Vulnerability Management Market (Wadhwani, 2023).	6
Figure 5: Nessus GUI (Tenable®, 2024).	13
Figure 6: Nmap Logo (Nmap, 2022).	14
Figure 7: Sparta GUI (Black Hat Ethical Hacking, 2022).	14
Figure 8: OpenVAS Logo (openvas, 2024).	15
Figure 9: Evolutionary Prototype (Lapada, 2019).	19
Figure 10: Project's Mind Map.	23
Figure 11: System Architecture	24
Figure 12: System Use Case Diagram.	25
Figure 13: Login Wireframe.	26
Figure 14: Signup Wireframe.....	26
Figure 15: User Dashboard Wireframe.....	27
Figure 16: Credential Testing Wireframe.....	27
Figure 17: Level 0 System DFD	28
Figure 18: Level 1 DFD of System.	28
Figure 19: Level 2 DFD of Login.....	29
Figure 20: Level 2 DFD of Signup.	29
Figure 21: Level 2 DFD of Host Discovery.	30
Figure 22: Level 2 DFD of Generate Report.....	30
Figure 23: Level 2 DFD of Port Scanning.....	31
Figure 24: Level 2 DFD of Find Vulnerability.	32
Figure 25: Level 2 DFD of Credential Testing.	32
Figure 26: System Flowchart.....	33
Figure 27: Host Discovery Flowchart.....	34
Figure 28: Port Scanning Flowchart.	34
Figure 29: Find Vulnerability Flowchart.	35
Figure 30: Credential Testing Flowchart.....	36

Figure 31: Report Flowchart.	37
Figure 32: Generate Report Flowchart	37
Figure 33: Code to Store User Credentials on Database.	38
Figure 34: User Credentials Database.	38
Figure 35: Login Code Part 1.	39
Figure 36: Login Code Part 2.	39
Figure 37: Login Code Part 3.	39
Figure 38: Login GUI	40
Figure 39: Code for Sign Up GUI.	40
Figure 40: Sign Up GUI	41
Figure 41: Code for User Dashboard.	41
Figure 42: User Dashboard.	42
Figure 43: Credential Testing Code Part 1.	43
Figure 44: Credential Testing Code Part 2.	43
Figure 45: Credential Testing Code Part 3.	44
Figure 46: Credential Testing Code Part 4.	44
Figure 47: Credential Testing GUI.....	45
Figure 48: Code of Logout Button.	45
Figure 49: Code of Back Button.	45
Figure 50: Generate Report code Part-1.	46
Figure 51: Generate Report code Part-2	47
Figure 52: Generate Report code Part-3.	47
Figure 53: Save Report As.	48
Figure 54: Code to List the Files.....	48
Figure 55: Select a PDF File	49
Figure 56: Error Handling on Sign Up Button (1).....	51
Figure 57: Error Handling on Sign Up Button (2).....	52
Figure 58: Register New User.	53
Figure 59: Error Handling on Login Button (1).....	54
Figure 60: Error Handling on Login Button (2).....	55
Figure 61: Login Successful.	56

Figure 62: Redirect to User Dashboard.	57
Figure 63: Host Discovery Button Output.	58
Figure 64: Port Scanning Button Output.	59
Figure 65: Find Vulnerability Button Output.	60
Figure 66: Naming File as "hostdiscovery".	61
Figure 67: Report Generated Successfully.	62
Figure 68: Selecting the "hostdiscovery" File.	63
Figure 69: Open "hostdiscovery" File.	63
Figure 70: Error Handling on Output Box.	64
Figure 71: Redirecting to Login Window.	65
Figure 72: Successfully Redirect to Login Window.	66
Figure 73: Selecting IP address.	67
Figure 74: Successfully Brute force the Selected IP Address.	68
Figure 75: Naming file as "brute".	69
Figure 76: Successfully Generated Report.	69
Figure 77: Redirecting to the Dashboard.	70
Figure 78: Successfully Redirect to the Dashboard.	71
Figure 79: Trying to Edit the Output Box.	72
Figure 80: Error Handling on Network Address Entry.	73
Figure 81: Running Python File.	74
Figure 82: Login System Open.	74
Figure 83: Registering User.	75
Figure 84: Store User Credential Testing on Database.	76
Figure 85: Login Successful.	77
Figure 86: Result of Host Discovery.	78
Figure 87: Results of Port Scanning.	79
Figure 88: Results of Find Vulnerability.	80
Figure 89: Output of Credential Testing.	81
Figure 90: Naming File.	82
Figure 91: File Saved.	83
Figure 92: List the file.	83

Figure 93: Report.	84
Figure 94: Wireframe of the System Dashboard	91
Figure 95: Waterfall Methodology Phases (Senarath, 2021).....	99
Figure 96: Agile Methodology Phases (Issuu, 2023).	100
Figure 97: Pre-Survey Form Part 1.	100
Figure 98: Pre-Survey Form Part 2.	101
Figure 99: Pre-Survey Form Part 3.	101
Figure 100: Pre-Survey Form Part 4.	102
Figure 101: Pre-Survey Form Part 5.	102
Figure 102: Pre-Survey Form Part 6.	103
Figure 103: Pre-Survey Form Part 7.	104
Figure 104: Pre-Survey Form Part 8.	104
Figure 105: Pre-Survey Form Part 9.	105
Figure 106: Filled Pre-Survey Forms Part 1.	105
Figure 107: Filled Pre-Survey Forms Part 2.	106
Figure 108: Filled Pre-Survey Forms Part 3.	107
Figure 109: Filled Pre-Survey Forms Part 4.	108
Figure 110: Filled Pre-Survey Forms Part 5.	109
Figure 111: Filled Pre-Survey Forms Part 6.	110
Figure 112: Filled Pre-Survey Forms Part 7.	111
Figure 113: Pre-Survey Question 1 Result.	113
Figure 114: Pre-Survey Question 2 Result.	113
Figure 115: Pre-Survey Question 3 Result.	114
Figure 116: Pre-Survey Question 4 Result.	114
Figure 117: Pre-Survey Question 5 Result.	115
Figure 118: Pre-Survey Question 6 Result.	115
Figure 119: Pre-Survey Question 7 Result.	116
Figure 120: Pre-Survey Question 8 Result.	116
Figure 121: Pre-Survey Question 9 Result.	117
Figure 122: Pre-Survey Question 10 Result.	117
Figure 123: Pre-Survey Question 11 Result.	118

Figure 124: Pre-Survey Question 12 Result.	118
Figure 125: Post-Survey Form Part 1.	119
Figure 126: Post-Survey Form Part 2.	120
Figure 127: Post-Survey Form Part 3.	120
Figure 128: Post-Survey Form Part 4.	121
Figure 129: Post-Survey Form Part 5.	122
Figure 130: Post-Survey Form Part 6.	122
Figure 131: Filled Post-Survey Forms Part 1.	123
Figure 132: Filled Post-Survey Forms Part 2.	124
Figure 133: Filled Post-Survey Forms Part 3.	125
Figure 134: Filled Post-Survey Forms Part 4.	126
Figure 135: Filled Post-Survey Forms Part 5.	126
Figure 136: Post-Survey Question 1 Result.	128
Figure 137: Post-Survey Question 2 Result.	128
Figure 138: Post-Survey Question 3 Result.	129
Figure 139: Post-Survey Question 4 Result.	129
Figure 140: Post-Survey Question 5 Result.	130
Figure 141: Post-Survey Question 6 Result.	130
Figure 142: Post-Survey Question 7 Result.	130
Figure 143: Sample Code Part 1.	131
Figure 144: Source Code Part 2.	132
Figure 145: Source Code Part 3.	132
Figure 146: Source Code Part 4.	133
Figure 147: Source Code Part 5.	133
Figure 148: Source Code Part 6.	134
Figure 149: Source Code Part 7.	135
Figure 150: Source Code Part 8.	135
Figure 151: Source Code Part 9.	136
Figure 152: Source Code Part 10.	136
Figure 153: Source Code Part 11.	137
Figure 154: Source Code Part 12.	137

Figure 155: Source Code Part 13.....	138
Figure 156: Source Code Part 14.....	139
Figure 157: Source Code Part 15.....	139
Figure 158: Source Code Part 16.....	140
Figure 159: Sequence Diagram of Login Button.....	140
Figure 160: Sequence Diagram of Sign Up Button.....	141
Figure 161: Sequence Diagram of Host Discovery.....	141
Figure 162: Sequence Diagram of Port Scanning Button.....	142
Figure 163: Sequence Diagram of Find Vulnerability Button.....	142
Figure 164: Sequence Diagram of Credential Testing Button.....	143
Figure 165: Sequence Diagram of Generate Report Button.....	144
Figure 166: Work Breakdown Structure of the Project.....	144
Figure 167: Gantt Chart for the Project.....	145
Figure 168: Login Window.....	145
Figure 169: Sign Up Window.....	146
Figure 170: User Dashboard.....	146
Figure 171: Credential Testing Window.....	147
Figure 172: User Feedback Form Part 1.....	148
Figure 173: User Feedback Form Part 2.....	149
Figure 174: User Feedback Form Part 3.....	149
Figure 175: User Feedback Form Part 4.....	150
Figure 176: User Feedback Form Part 5.....	151
Figure 177: User Feedback Form Part 6.....	151
Figure 178: Filled Feedback Forms Part 1.....	152
Figure 179: Filled Feedback Forms Part 2.....	153
Figure 180: Filled Feedback Forms Part 3.....	154
Figure 181: Filled Feedback Forms Part 4.....	155
Figure 182: Filled Feedback Forms Part 5.....	156
Figure 183: User and Entity Behavior Analytics (Rapid7, 2024).....	157
Figure 184: Threat Intelligence (Kaspersky, 2022).....	158
Figure 185: Docker (Rivera, 2020).....	159

Figure 186: Feature Changing Approval.....	159
Figure 187: Client Approval Letter.....	160

Table of Tables

Table 1: Comparison Table	16
Table 2: Error Handling on Sign Up Button (1).	51
Table 3: Error Handling on Sign Up Button (2).	52
Table 4: Register New User.	53
Table 5: Error Handling on Login Button (1).	54
Table 6: Error Handling on Login Button (2)	55
Table 7: Login Button Testing Table.....	56
Table 8: Host Discovery Button Testing Table.	57
Table 9: Port Scanning Button Testing Table.	59
Table 10: Find the Vulnerability Button Testing table.	60
Table 11: Generate Report Button Testing Table.	61
Table 12: Report Button Testing Table.....	62
Table 13: Error Handling on the Generate Report Button.....	64
Table 14: Logout Button Testing Table.	65
Table 15: Credential Testing Button Testing Table.	66
Table 16: Generate report Button Testing Table.	68
Table 17: Back Button Testing Table.	70
Table 18: Output Box Testing Table.....	71
Table 19: Network Address Entry Testing Table.	72
Table 20: Run Application.	73
Table 21: Sing Up feature Testing	75
Table 22: Login Features Testing.....	76
Table 23: Host Discovery Feature Testing.	77
Table 24: Port Scanning Feature Testing.....	79
Table 25: Find Vulnerability Feature Testing.	80
Table 26: Credential Testing Feature Testing.	81
Table 27: Generate Report Feature Testing.....	82
Table 28: Report Button Testing.....	83

1 Introduction

The modern world is changing into a global village, a single window that can connect one single person to the whole world. Around the world, societies are becoming increasingly dependent upon information and communications technology (ICT), driving rapid social, economic, and governmental development. As of January 2021, there were 4.66 billion active internet users worldwide - 59.5 percent of the global population. Of this total, 92.6 percent (4.32 billion) accessed the internet via mobile devices, with 4.2 billion active social media users (Malla & Timalsena, 2022). Along with this development, new threats to digital infrastructures and opportunities for misuse of cyberspace have emerged, affecting individuals, society, and governments alike.

Cybercrime is one of the fastest-growing areas of criminal activity. Advancement in the information technology (IT) sector resulting in the rapid growth in computer and internet users has not only led to its development but has also led to a rise in criminal activities and misuse of computer technology, posing new types of challenges for the justice delivery system (Curtis & Oxburgh, 2022). Criminals use modern information technology as it offers speed, convenience, and anonymity.

One of the most pressing issues is that vulnerabilities have emerged as a major source of cyber threats in the modern era of computing. The assessment of vulnerabilities in networking devices is crucial for maintaining a secure network (Sharma & Zamfiroiu, 2023). Unattended vulnerabilities in the network present immense possibilities for exploitation. Also, default or weak passwords for devices can be exploited by attackers, further emphasizing the need for robust security measures (Knieriem, et al., 2018). Software companies and government organizations are diligently working on various tools and methodologies to reduce security risks in networks.

In response to this growing concern, the network vulnerability assessment (NVA) tool comes into play. Network vulnerability assessment tools are crafted to conduct scans across an organization's network, seeking out vulnerabilities susceptible to exploitation. They conduct thorough examinations of the network infrastructure, scrutinizing servers, host devices, and other components for potential weaknesses. These tools possess the capability to detect various vulnerabilities, such as outdated firmware, weak passwords,

misconfigurations, and known exploits. Upon identifying these vulnerabilities, the tool furnishes detailed reports and recommendations for remediation (Chopskie, 2023).

1.1 Project Description

The “NetGuardian” project presents a comprehensive network vulnerability assessment tool designed to fortify network security through proactive identification and mitigation of potential vulnerabilities. Developed as a Graphical User Interface (GUI) application using Python's tkinter library, NetGuardian integrates a suite of functionalities aimed at enhancing network resilience and safeguarding against potential cyber threats.

For this project, we envision deploying NetGuardian within small to medium-sized businesses (SMBs) that face increasing challenges in protecting their digital infrastructure from cyber threats (Chidukwani, et al., 2022). Specifically, the tool is tailored to cater to the needs of IT departments or cybersecurity teams within these organizations. These departments are responsible for maintaining the integrity and security of the organization's computer networks, making them the primary users of NetGuardian.

Within the IT department or cybersecurity team, various roles may benefit from utilizing NetGuardian. Network administrators would utilize its host discovery and port scanning functionalities to gain insights into network topology and identify potential entry points for cyber threats. Cybersecurity analysts would leverage its vulnerability detection capabilities to assess security weaknesses and prioritize remediation efforts. Additionally, IT managers and business owners would utilize NetGuardian's report generation feature to track network security status and make informed decisions regarding cybersecurity investments.

Overall, NetGuardian serves as a valuable tool for organizations looking to enhance their cybersecurity posture by proactively identifying and addressing network vulnerabilities. Its user-friendly interface and powerful features make it accessible and effective for IT departments and cybersecurity teams within small to medium-sized businesses, ultimately helping to mitigate the risks associated with cyber threats.

1.2 Current Scenario

In 2022, the Security and Vulnerability Management Market reached a valuation of USD 14.5 billion, and it's projected to maintain a Compound Annual Growth Rate (CAGR) of over 10% from 2023 to 2032 (Wadhwani, 2023). This growth is primarily propelled by the escalating frequency of global cyber-attacks and threats. The increasingly sophisticated nature of these cyber threats has spurred a heightened demand for robust security solutions across various sectors. For instance, statistics from the Indian Ministry of Electronics and IT indicate that in the first half of 2023 alone, 36 government websites and over 4.29 lakh cyber security incidents related to financial institutions were recorded, underlining the critical need for effective security measures (Wadhwani, 2023).

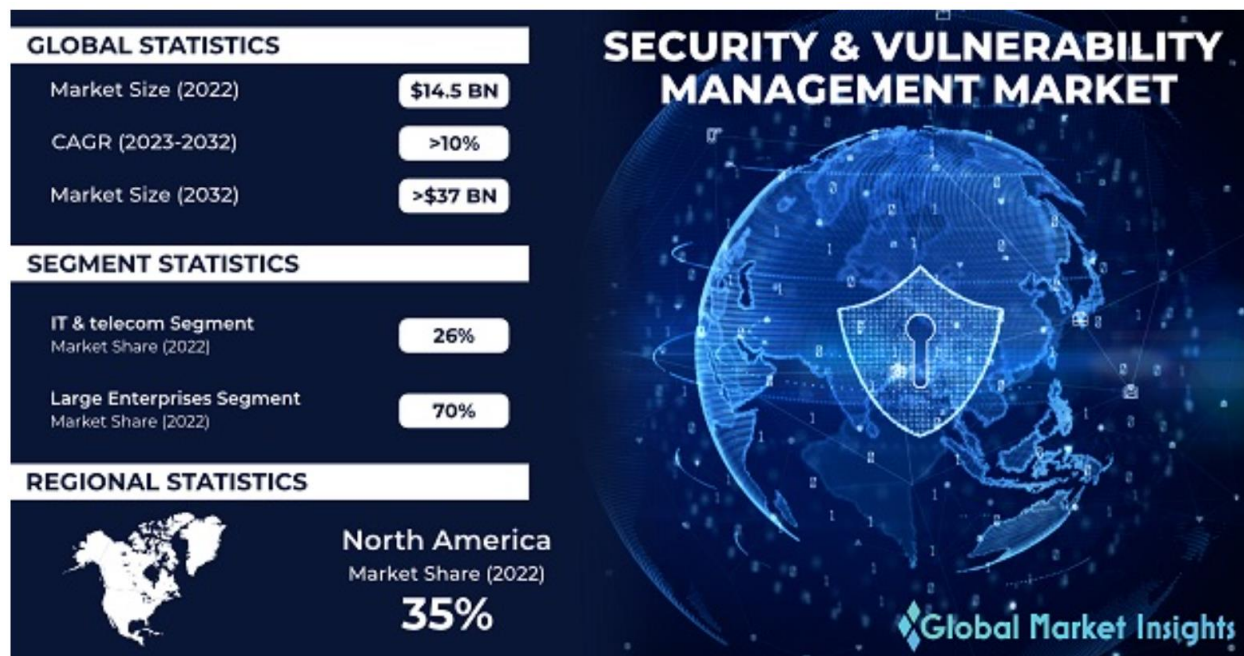


Figure 1: Security and Vulnerability Management Market Statistics (Wadhwani, 2023).

Furthermore, the proliferation of regulatory compliance mandates, such as GDPR and industry-specific regulations, has become a significant driver for the security and vulnerability management market. Organizations are under increasing pressure to meet stringent security standards, resulting in a surge in demand for comprehensive solutions that can effectively manage vulnerabilities while ensuring compliance and mitigating risks (Wadhwani, 2023).

In terms of organization size, large enterprises dominated the market in 2022, holding over 70% of the market share (Wadhvani, 2023). This dominance is attributed to their substantial IT infrastructure and heightened risk exposure, which necessitates comprehensive security measures. With the escalating cyber threats, large enterprises are increasingly investing in advanced vulnerability management solutions to address complex networks and diverse systems, thereby driving growth in this segment (Wadhvani, 2023).

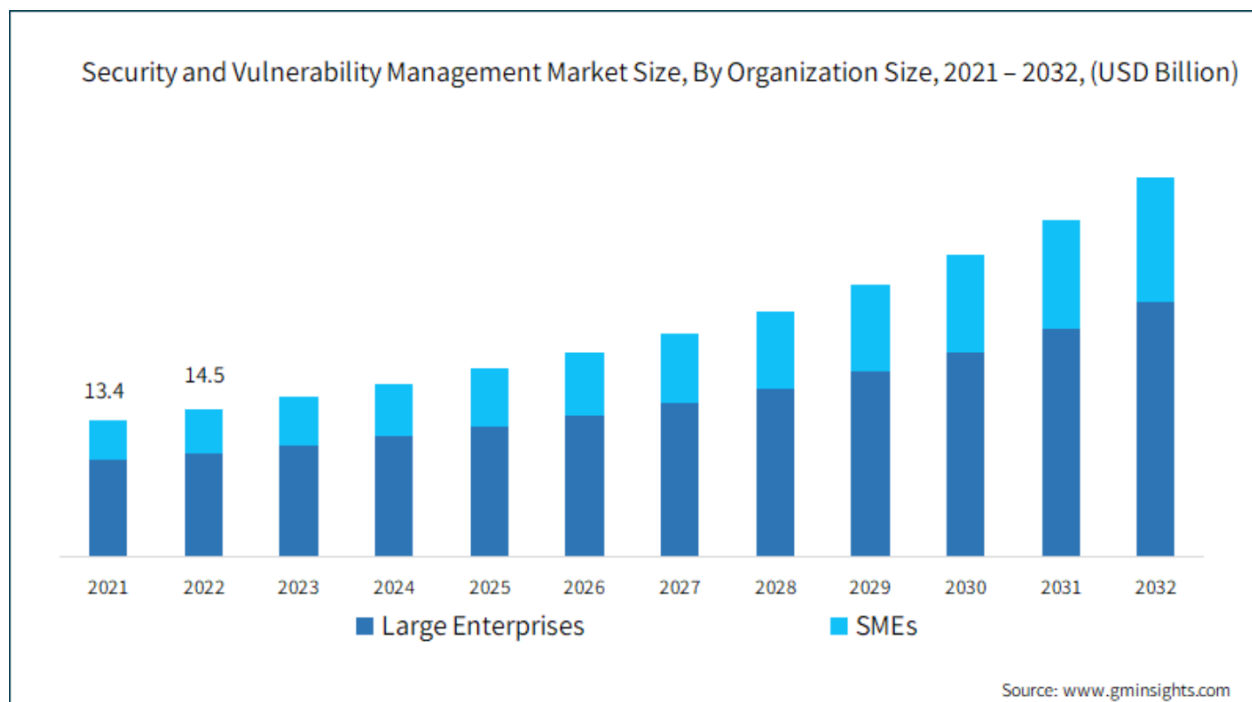


Figure 2: Market Size by Organization Size (Wadhvani, 2023).

Regarding end-use industries, the IT & telecom segment accounted for 26% of the market share in 2022 due to its increased digital dependency (Wadhvani, 2023). As the sector expands its networks and services, the vulnerability landscape widens, necessitating robust security measures. This has led to investments in sophisticated vulnerability management solutions to safeguard critical systems, protect sensitive data, and ensure uninterrupted services, thereby fueling growth within this segment.

Security and Vulnerability Management Market, By End-User Industry, 2022

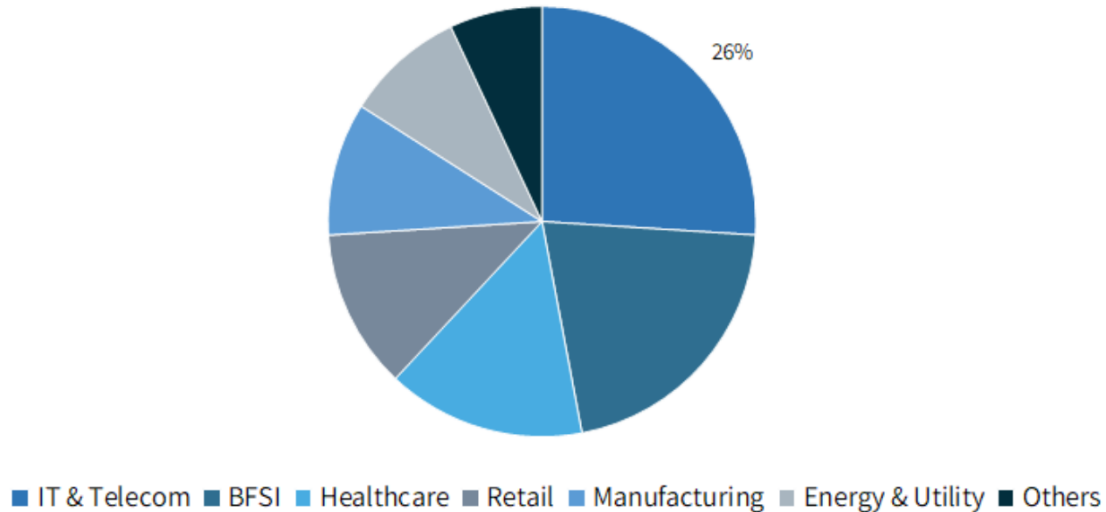
Source: www.gminsights.com

Figure 3: Market Size By End User Industry (Wadhwani, 2023).

Geographically, North America led the Security and Vulnerability Management Market in 2022, capturing more than 35% of the revenue share (Wadhwani, 2023). This dominance can be attributed to the region's heightened awareness of cyber threats, stringent regulatory compliance requirements, and substantial investments in cybersecurity. North American organizations prioritize vulnerability management to fortify systems, protect data, and comply with evolving security standards, thereby driving significant market growth in the region.



Figure 4: US Security and Vulnerability Management Market (*Wadhvani, 2023*).

1.3 Problem Domain and Project as a Solution

Cybercrime is increasingly becoming a significant concern for small to medium-sized businesses (SMBs). According to the 2014 Year-End Economic Report by the National Small Business Association (NSBA), half of all small businesses reported being victims of cyber-attacks up from 44 percent just two years prior. Among those targeted, 68 percent reported being victims of cybercrime more than once (*Malla & Timalseña, 2022*). Despite the escalation in cybercrime incidents among small businesses, many remain vulnerable due to limited resources and, surprisingly, a lack of awareness regarding the threat.

Managing employee passwords presents a challenge for the majority of businesses in both the U.S. and globally. Maintaining robust password security measures incurs costs, yet the absence of such measures can lead to substantial financial losses. According to the latest Verizon Data Breach Investigations Report, more than 70% of employees reuse passwords in their work settings. The report highlights a concerning statistic, revealing that "81% of breaches related to hacking exploited either stolen or weak passwords (*Bank of North Dakota, 2018*).

The project aims to develop a network vulnerability assessment tool to address the pressing issues highlighted above. This tool, named "NetGuardian," is designed to help small to medium-sized businesses (SMBs) identify and fix potential security weaknesses in their computer networks. By proactively assessing vulnerabilities, the tool aims to fortify network security and reduce the risk of cyber-attacks. The tool provides a comprehensive solution by offering features such as user authentication, host discovery, port scanning, vulnerability detection, credential testing, and report generation. These functionalities empower small businesses to strengthen their network security posture and mitigate the risks associated with cyber threats.

Through its user-friendly interface and powerful capabilities, the tool aims to raise awareness among small business owners about the importance of cybersecurity and provide them with practical tools to safeguard their digital assets. By equipping businesses with the means to identify and address vulnerabilities, the tool strives to mitigate the financial and reputational losses resulting from cyber-attacks.

1.4 Aim and Objectives

The aim of the project "NetGuardian" is to develop a comprehensive network vulnerability assessment tool to enhance network security by proactively identifying and mitigating potential vulnerabilities. By leveraging modern technologies and methodologies, the tool aims to provide network users with an intuitive interface to assess network infrastructure, identify security weaknesses, and generate actionable insights to bolster network defenses.

The objectives of the project are:

- Create an intuitive Graphical User Interface (GUI) to facilitate ease of use and accessibility for network users.
- Incorporate login and signup functionalities with robust password policies to ensure secure access control and mitigate the risk of unauthorized entry.

- Identify active hosts within the network, along with their MAC addresses and operating system information, providing users with valuable insights into network topology.
- Identify open ports on active hosts and determine the services and service version running on each port, enhancing network visibility and aiding in the identification of potential entry points for cyber threats.
- Integrate with National Vulnerability Database (NVD) API to pinpoint vulnerabilities associated with specific services and versions, enabling users to prioritize and address security weaknesses.
- Provide users with the ability to evaluate the robustness of network credentials through brute-force attack simulations on port numbers like 21 and 22 using tools Hydra, and assessment of authentication mechanisms.
- Develop a reporting mechanism that generates clear, concise, and actionable reports.
- Provide cost-effective solutions to the organizations.

1.5 Structure of the Report

1.5.1 Background

The Background section provides information about the users who will use the NetGuardian tool, explains what the tool does, talks about other similar tools, and compares the NetGuardian tool with similar tools. This section helps readers understand who the tool is designed for, what it does, and how it differs from other options available.

1.5.2 Development

The development section encompasses several critical aspects of the project lifecycle, including the chosen methodology, its corresponding phase, project requirement analysis, project design, and project implementation. This segment serves as a comprehensive guide detailing the systematic approach adopted for the project's execution, ensuring clarity and coherence throughout its development process.

1.5.3 Testing and Analysis

The testing and analysis section is dedicated to examining the core features of the project, including testing the overall system. It also involves analyzing whether the project's development aligns with its intended goals and objectives. This phase aims to ensure that the project's functionalities meet the specified criteria and that its progress remains in line with its original purpose and objectives.

1.5.4 Conclusion

In the conclusion section, we delve into the legal, social, and ethical considerations that the tool may encounter. Additionally, we evaluate and highlight its strengths and limitations. Furthermore, we outline potential areas for future development and improvement within the project. This section serves as a comprehensive reflection on the broader implications and prospects for the tool's utilization and advancement.

2 Background

2.1 About the End Users

The end users of the NetGuardian tool primarily consist of small business owners, IT administrators, and network security professionals. Small business owners, often wearing multiple hats within their organizations, are tasked with overseeing various aspects of their operations, including cybersecurity. For these individuals, NetGuardian offers a valuable solution to enhance their understanding of network vulnerabilities and take proactive measures to protect their business assets.

IT administrators play a critical role in implementing and managing network infrastructure within small businesses. These professionals are responsible for ensuring the smooth operation of computer networks and safeguarding against potential security threats. NetGuardian provides IT administrators with a comprehensive toolkit to streamline the process of vulnerability assessment, enabling them to efficiently identify and mitigate security risks.

Additionally, network security professionals, whether employed internally or externally by small businesses, rely on tools like NetGuardian to conduct thorough security assessments and ensure compliance with industry standards and regulations. By leveraging NetGuardian's advanced features, security professionals can gain deeper insights into the security posture of small business networks and recommend appropriate measures to strengthen defenses.

Furthermore, the simplicity and effectiveness of NetGuardian's user interface make it accessible to users with varying levels of technical expertise. Small business owners and IT administrators who may not have extensive cybersecurity backgrounds can easily navigate the tool's functionalities to assess network vulnerabilities and take corrective actions as needed. This user-friendly approach empowers end users to actively participate in securing their networks without requiring specialized training or knowledge.

2.2 Understanding the Solution

The project, titled "NetGuardian," presents a comprehensive network vulnerability assessment tool designed to fortify network security through proactive identification and mitigation of potential vulnerabilities. Developed as a Graphical User Interface (GUI) application using Python's tkinter library, NetGuardian integrates a suite of functionalities aimed at enhancing network resilience and safeguarding against potential cyber threats.

The solution encompasses essential features essential for robust network defense. To ensure secure access, NetGuardian offers user authentication through login and signup functionalities, incorporating a robust password policy during user registration to bolster account security. This feature empowers users to create and manage their accounts with confidence, mitigating the risk of unauthorized access.

NetGuardian's host discovery capability leverages the power of the Python Nmap library to identify active hosts within the network. By discerning active hosts, along with their corresponding MAC addresses and operating system information, the tool equips administrators with vital insights into facilitating efficient network management and threat detection.

The port scanning functionality of NetGuardian further enhances network visibility by identifying open ports on active hosts. Leveraging the Python Nmap library, the tool meticulously scans network ports, and services running on each port along with their respective versions. This comprehensive port scanning capability empowers users to identify potential entry points for cyber threats, enabling proactive measures to reinforce network security.

Incorporating advanced vulnerability detection capabilities i.e., finding vulnerability features, NetGuardian enables users to identify potential security weaknesses within the network infrastructure. Leveraging data obtained from Nmap scans, the tool correlates identified services and their versions with the National Vulnerability Database (NVD) API to pinpoint vulnerabilities associated with specific services and versions. By presenting vulnerability information in a user-friendly format, NetGuardian facilitates informed decision-making regarding vulnerability remediation strategies.

The credential testing feature of NetGuardian provides users with a potent tool for evaluating the robustness of network credentials. Through a dedicated credential testing window, users can select target IP addresses and ports and their services. With the option to conduct brute-force attacks using the Hydra tool, NetGuardian enables users to assess the resilience of network credentials, identifying potential security gaps and reinforcing authentication mechanisms.

Finally, NetGuardian simplifies the process of report generation, empowering users to document assessment findings effectively. Utilizing Python Canvas, the tool generates comprehensive reports detailing assessment results, including identified vulnerabilities, network topology insights, and authentication assessment findings. By timestamping reports with current date and time, NetGuardian ensures the relevance and accuracy of assessment documentation, facilitating informed decision-making and ongoing network security maintenance.

2.3 Similar Projects

2.3.1 Nessus

Tenable Nessus is recognized as the industry's most widely deployed solution for vulnerability assessment and plays a crucial role in helping organizations reduce their attack surface and ensure compliance. Tenable Nessus features high-speed asset discovery, configuration auditing, target profiling, malware detection, sensitive data discovery, and more (Tenable, 2024).

Tenable Nessus supports more technologies than competitive solutions, scanning operating systems, network devices, hypervisors, databases, web servers, and critical infrastructure for vulnerabilities, threats, and compliance violations (Tenable, 2024). With the world's largest continuously updated library of vulnerability and configuration checks, and the support of Tenable, Inc.'s expert vulnerability research team, Tenable Nessus sets the standard for vulnerability scanning speed and accuracy.

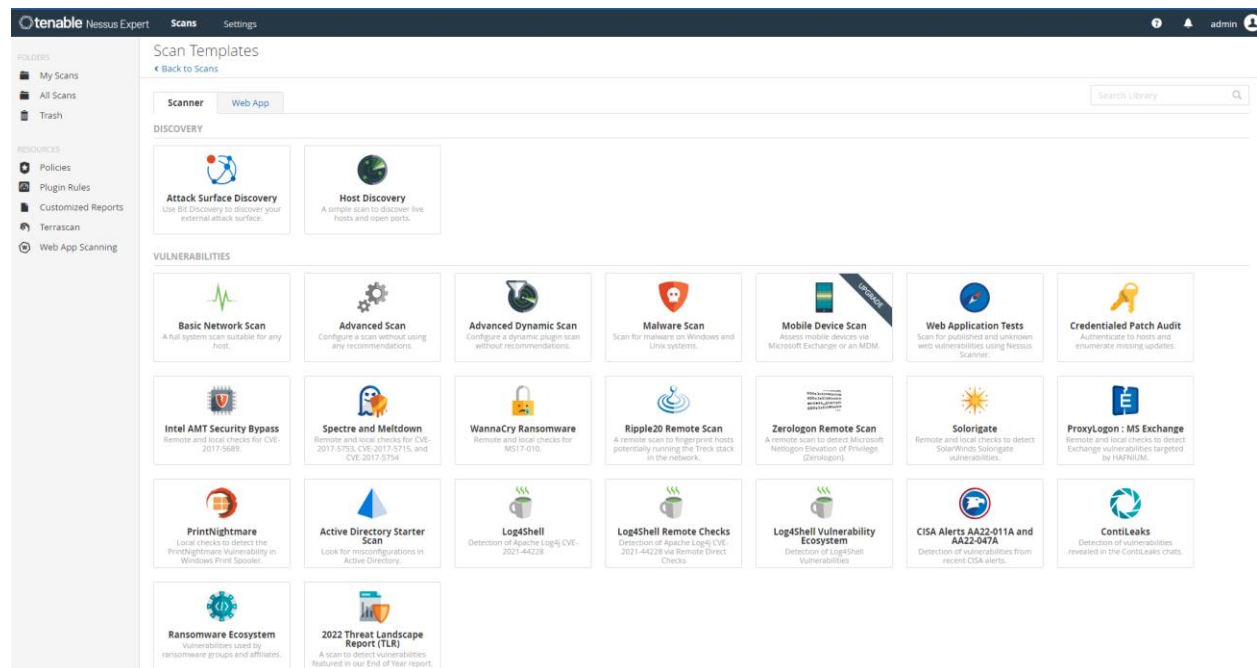


Figure 5: Nessus GUI (Tenable®, 2024).

2.3.2 Nmap

Nmap, also known as Network Mapper, is a free and open-source tool that is widely used for network discovery and security auditing. It is highly valued by system and network administrators for a variety of tasks. These include network inventory management, scheduling service upgrades, and monitoring the uptime of hosts or services (Nmap, 2022).

Nmap employs raw IP packets in innovative ways to ascertain a host of information about the network. It can identify available hosts on the network, the services they offer (including the application name and version), the operating systems they run (along with the OS versions), the type of packet filters or firewalls in use, and many other characteristics (Nmap, 2022). It is designed with the capability to swiftly scan large networks, Nmap is equally effective when used against single hosts. It is compatible with all major computer operating systems, and official binary packages are readily available for Linux, Windows, and Mac OS X.



Figure 6: Nmap Logo (Nmap, 2022).

2.3.3 Sparta

SPARTA is a GUI application, developed with Python, that serves as a network penetration testing tool for Kali Linux. It streamlines the scanning and enumeration phase, delivering faster results. One of the standout features of the SPARTA GUI toolkit is its ability to scan and detect the service running on the target port. Additionally, it offers a Brute force attack for scanned open ports and services, which forms an integral part of the enumeration phase (Ganesh, 2023).

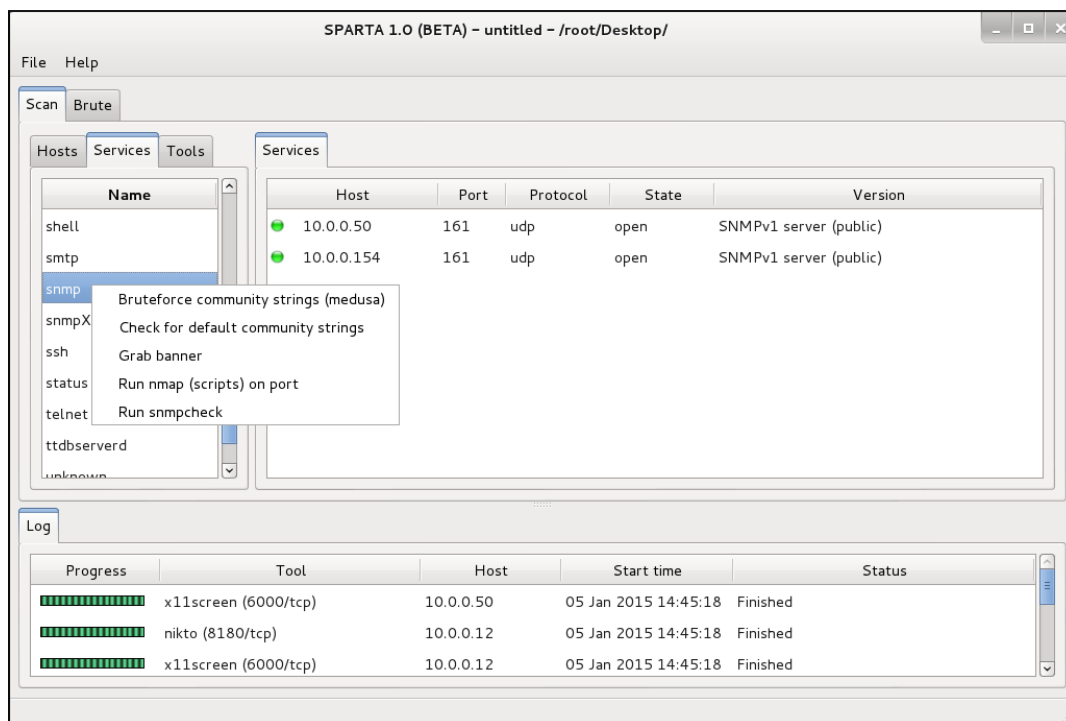


Figure 7: Sparta GUI (Black Hat Ethical Hacking, 2022).

2.3.4 OpenVAS (Open Vulnerability Assessment Scanner)

OpenVAS (Open Vulnerability Assessment Scanner) is a comprehensive vulnerability scanner renowned for its robust features. Its functionalities encompass both unauthenticated and authenticated testing, spanning various high-level and low-level internet and industrial protocols. Additionally, it offers performance tuning tailored for large-scale scans and boasts a powerful internal programming language, enabling the implementation of diverse vulnerability tests. The scanner accesses tests for identifying vulnerabilities from a feed with a rich historical database and daily updates (openvas, 2024).

Developed and continuously advanced by Greenbone since 2006, OpenVAS stands as a cornerstone within the commercial vulnerability management product line, Greenbone Enterprise Appliance (openvas, 2024). Within this ecosystem, it constitutes the Greenbone Community Edition alongside other open-source modules.



Greenbone OpenVAS

Open Vulnerability Assessment Scanner

Figure 8: OpenVAS Logo (openvas, 2024).

2.4 Comparisons

S. N	Features	Nessus	Nmap	Sparta-GUI	OpenVAS	NetGuardian
1	User-friendly GUI	Yes	No	Yes	Yes	Yes
2	User-authentication	Yes	No	Yes	Yes	Yes
3	Host Discovery	Yes	Yes	Yes	Yes	Yes
4	Port Scanning	Yes	Yes	Yes	Yes	Yes
5	Discover Vulnerability	Yes	Yes	Yes	Yes	Yes
6	Cost-effective	No	Yes	Yes	No	Yes
7	Open source	No	Yes	No	No	Yes
8	Generate Report	Yes	Yes	Yes	Yes	Yes

Table 1: Comparison Table

The table compares various network vulnerability assessment tools with the developed “NetGuardian” project based on several key features. Nessus, Sparta-GUI, OpenVAS, and NetGuardian all provide a user-friendly Graphical User Interface (GUI), making them accessible to users with varying levels of technical expertise. However, Nmap lacks this feature, requiring users to interact with it through a command-line interface. User authentication is a crucial security measure, and all tools except Nmap offer this feature. This ensures that only authorized users can access and utilize the tools' functionalities, enhancing overall security.

Host discovery, port scanning, and vulnerability discovery are fundamental functionalities present in all tools. These features enable users to identify active hosts on the network, scan open ports, and detect potential vulnerabilities, providing valuable insights for network security assessment and management.

In terms of cost-effectiveness, Nmap, Sparta-GUI, and NetGuardian stand out as they offer their functionalities at no cost or with minimal expenses. On the other hand, Nessus and OpenVAS are known for their high costs, which may limit their accessibility for smaller organizations with budget constraints. Open-source availability is another important aspect, with Nmap and NetGuardian being open-source tools. This means that users can access and modify the source code according to their specific requirements, fostering community collaboration and customization possibilities.

Finally, all tools are capable of generating reports summarizing the assessment findings. This feature is essential for documenting vulnerabilities, sharing insights with stakeholders, and informing decision-making regarding cybersecurity measures. Overall, each tool has its strengths and suitability depending on the organization's needs, budget, and technical preferences.

3 Development

3.1 Considered Methodologies

The considered methodologies are in the [appendix](#).

3.2 Selected Methodology

3.2.1 Evolutionary Prototyping

The Evolutionary Prototyping Model is a software development process where the prototype of a system is continually refined based on customer feedback until it evolves into the final product. This model is particularly useful when the customers do not know the exact project requirements beforehand (Guida, et al., 2012).

The reason for selecting evolutionary prototyping for this project is that it perfectly aligned with the project's requirements and objectives. This approach allows for the continuous refinement of the system based on user feedback, ensuring that the final product is tailored to meet the users' needs effectively. It also facilitates the early detection and resolution of potential issues, thereby reducing the risk of project failure (Guida, et al., 2012). Furthermore, evolutionary prototyping promotes active user involvement, fostering a sense of ownership and satisfaction among the stakeholders. Thus, this methodology not only enhances the quality of the deliverable but also ensures its acceptance and success in the target environment.

The advantages are:

- The tool can be refined based on user feedback, leading to a more effective and user-friendly tool.
- Since the prototype is tested at each iteration, errors can be detected and fixed early.
- The model is flexible, allowing new requirements to be easily accommodated on the project.
- Developers and users both understand the system better, which can lead to more effective network vulnerability assessments.

The disadvantages are:

- The model can be time-consuming due to the iterative nature of the process.
- The project may become complex due to continuous changes.
- There might be poor documentation because of continuously changing requirements.

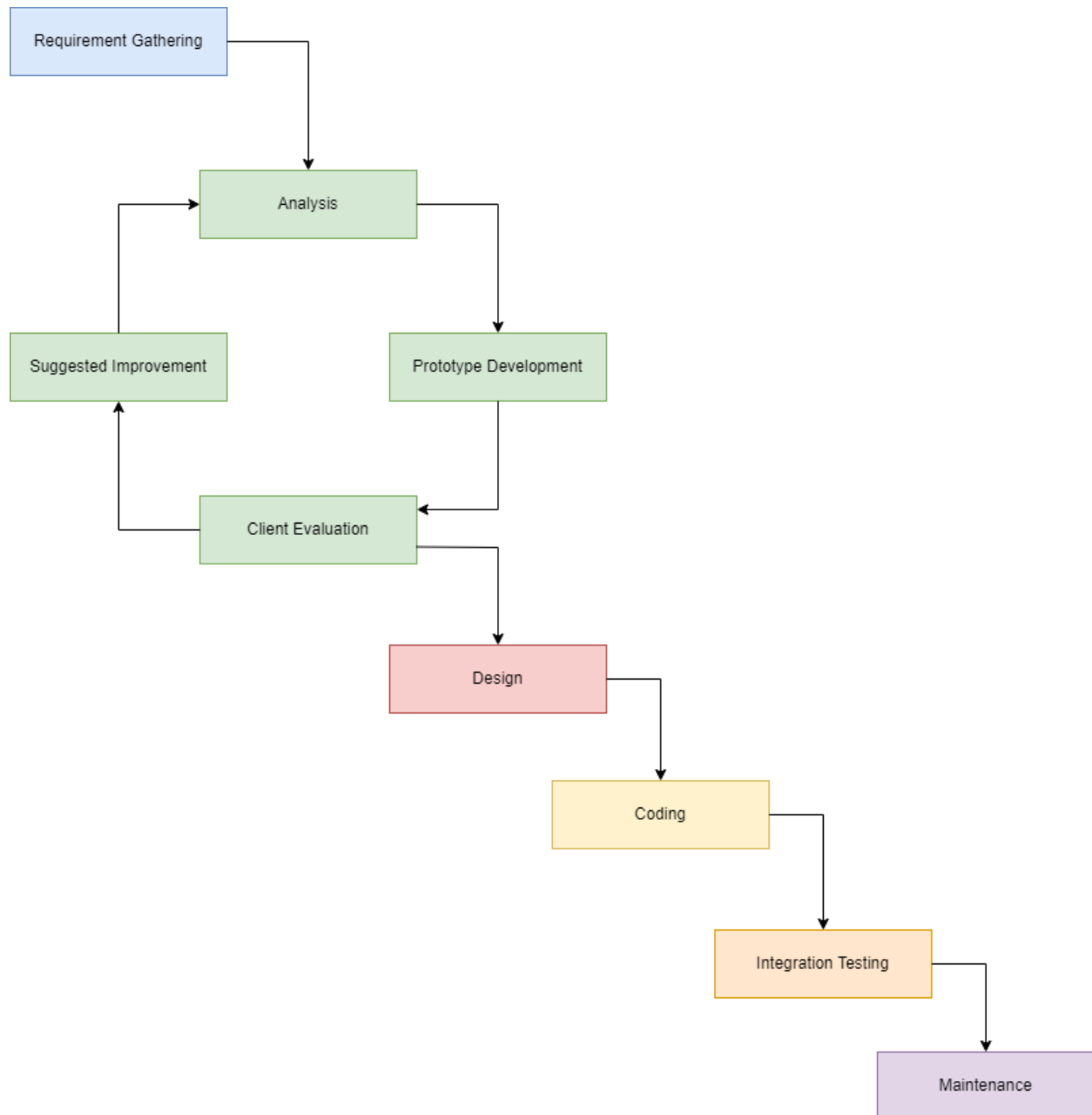


Figure 9: Evolutionary Prototype (*Lapada, 2019*).

3.3 Phases Of the Methodology

The development of the NetGuardian project, a network vulnerability assessment tool, follows a structured approach based on the phases of the evolutionary methodology. It begins with a thorough Requirement Gathering, where in-depth research is carried out to understand the specific needs and challenges faced by small businesses in managing network security and also conduct the survey. Through this process, a detailed understanding of the desired features, functionalities, and user expectations is obtained, laying the groundwork for the subsequent phases of the project.

Following Requirement Gathering, the Analysis phase entails a comprehensive review and prioritization of the gathered requirements. The identified needs are carefully examined to determine their relevance, feasibility, and potential impact on the project. Requirements are categorized and refined to establish a clear roadmap for development, ensuring that the NetGuardian tool addresses the most critical aspects of network security for small businesses. This phase serves as a crucial foundation for guiding the subsequent stages of the project and ensuring alignment with stakeholder expectations and project objectives.

Subsequently, the Prototype Development phase commences, focusing on the core functionalities identified during the requirement analysis. Using Python and the Tkinter library, a basic prototype of NetGuardian is crafted, with emphasis placed on essentials such as user authentication, network scanning, vulnerability detection, and credential testing. This prototype serves as a foundational framework upon which further iterations will build.

Once the prototype is ready, it undergoes Client Evaluation, where stakeholders test its usability and effectiveness. Feedback is gathered through various channels, including surveys and direct user testing sessions, providing invaluable insights into areas for improvement. Stakeholders' suggestions are carefully considered during the Suggested Improvement phase, where enhancements are identified and prioritized for inclusion in subsequent iterations.

In the Design phase, the NetGuardian tool's interface and functionality are refined based on stakeholder feedback and usability considerations. User interface (UI) and user

experience (UX) design principles are applied to ensure an intuitive and accessible user experience. The Coding phase follows, translating the refined design into functional code, leveraging Python and relevant libraries to implement host discovery, port scanning, credential testing, and vulnerability detection features.

Integration Testing is conducted to ensure that all components of the NetGuardian tool work seamlessly together and meet specified requirements. Through rigorous testing, compatibility issues are identified and addressed, ensuring the tool's reliability and effectiveness in real-world scenarios. Finally, the Maintenance phase ensures the ongoing relevance and reliability of NetGuardian through regular updates, bug fixes, and enhancements based on user feedback and emerging cybersecurity threats.

By following the evolutionary methodology, the NetGuardian project progresses iteratively, with each phase building upon the previous one. Through continuous feedback, refinement, and improvement, NetGuardian evolves into a robust and effective network vulnerability assessment tool, empowering small businesses to enhance their network security and mitigate cyber threats effectively.

3.4 Survey Results

3.4.1 Pre-Survey Results

Before starting the development, it is important to conduct the survey and gather comprehensive information about the network's size, device types, operating systems, familiarity with vulnerability assessment tools, current tool usage, preferred features in a tool, assessment frequency, performance satisfaction, confidence in assessments, presence of dedicated IT security personnel, and budget considerations.

The explanation of the pre-survey is in the [appendix](#).

3.4.2 Post-Survey Results

The post-survey was conducted after the development of the tool "NetGuardian" to assess the effectiveness, user satisfaction, and overall performance of the Network Vulnerability Assessment Tool, NetGuardian. By gathering feedback from users, this

evaluation provides valuable insights into the strengths and areas for improvement of NetGuardian, helping for further refinement and enhancement of the tool.

The explanation of the post-survey is in the [appendix](#).

3.5 Requirement Analysis

To develop NetGuardian effectively, a thorough requirement analysis was conducted to identify the key software, features, and functionalities necessary to address the cybersecurity needs of small businesses. The following requirements were identified based on the project objectives and user needs:

- Features Requirements:
 - Login User
 - Sign Up User
 - Host Discovery
 - Port Scanning
 - Find Vulnerability
 - Credential Testing
 - Generate Report
- Software Requirements:
 - Planning and Development track: Notion and spreadsheet
 - Design: Draw.io and balsamiq
 - IDE: Visual Studio Code and mousepad
 - Desktop application: Python and Python Tkinter
 - Operating System: Kali Linux and Windows
 - Virtualization: VMware Workstation Pro

3.6 Design

Design refers to the process of creating a plan or blueprint for building a software system or application. It involves making decisions about how the software will function, how it will be structured, and how different components will interact with each other to achieve the desired functionality (Pressman, 2010).

3.6.1 Mind Map

A Mind Map serves as a simple method for spontaneously generating and organizing thoughts, free from concerns about sequence and arrangement. By visually arranging your ideas, it aids in analysis and retention (MindMapping, 2024).

It is a graphical representation comprising tasks, words, concepts, or items connected to and positioned around a central idea or theme, employing a non-linear layout that enables users to construct an intuitive framework (MindMapping, 2024). Transforming extensive, mundane information into a vivid, memorable, and meticulously structured diagram, a Mind Map aligns with the innate cognitive processes of the brain.

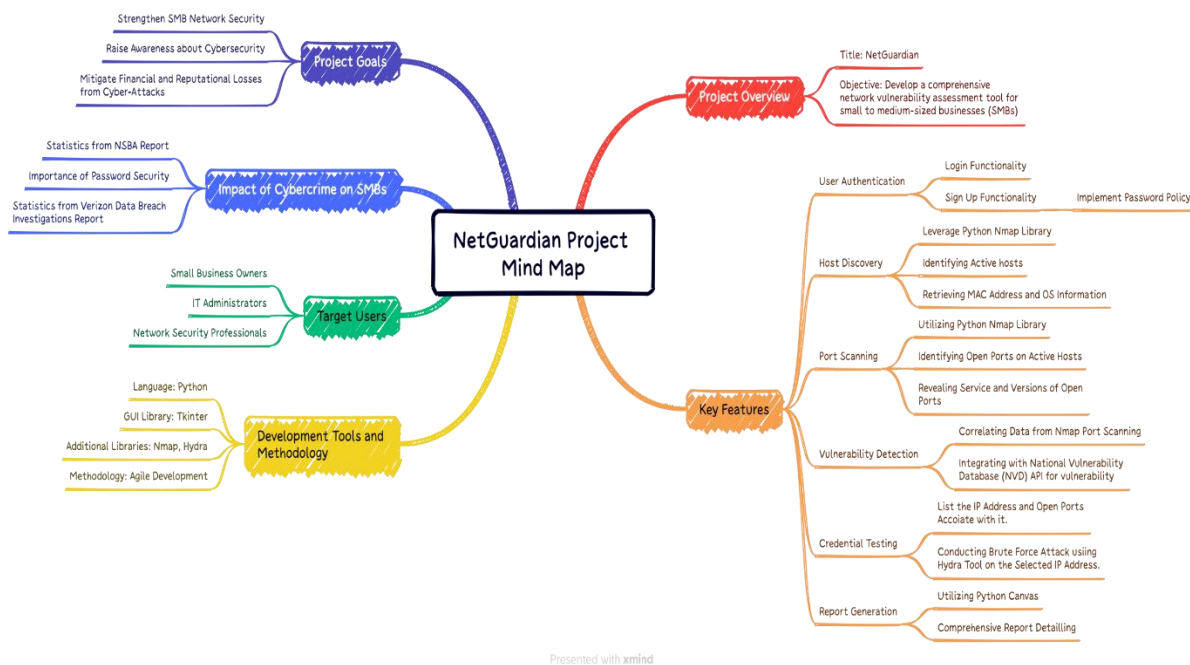


Figure 10: Project's Mind Map.

3.6.2 System Architecture

System Architecting is a method for enhancing the efficiency and effectiveness of systems by providing an overview, ensuring consistency and integrity, and maintaining balance. Essentially, the System Architect assists the development team in navigating a complex, dynamic, and uncertain environment.

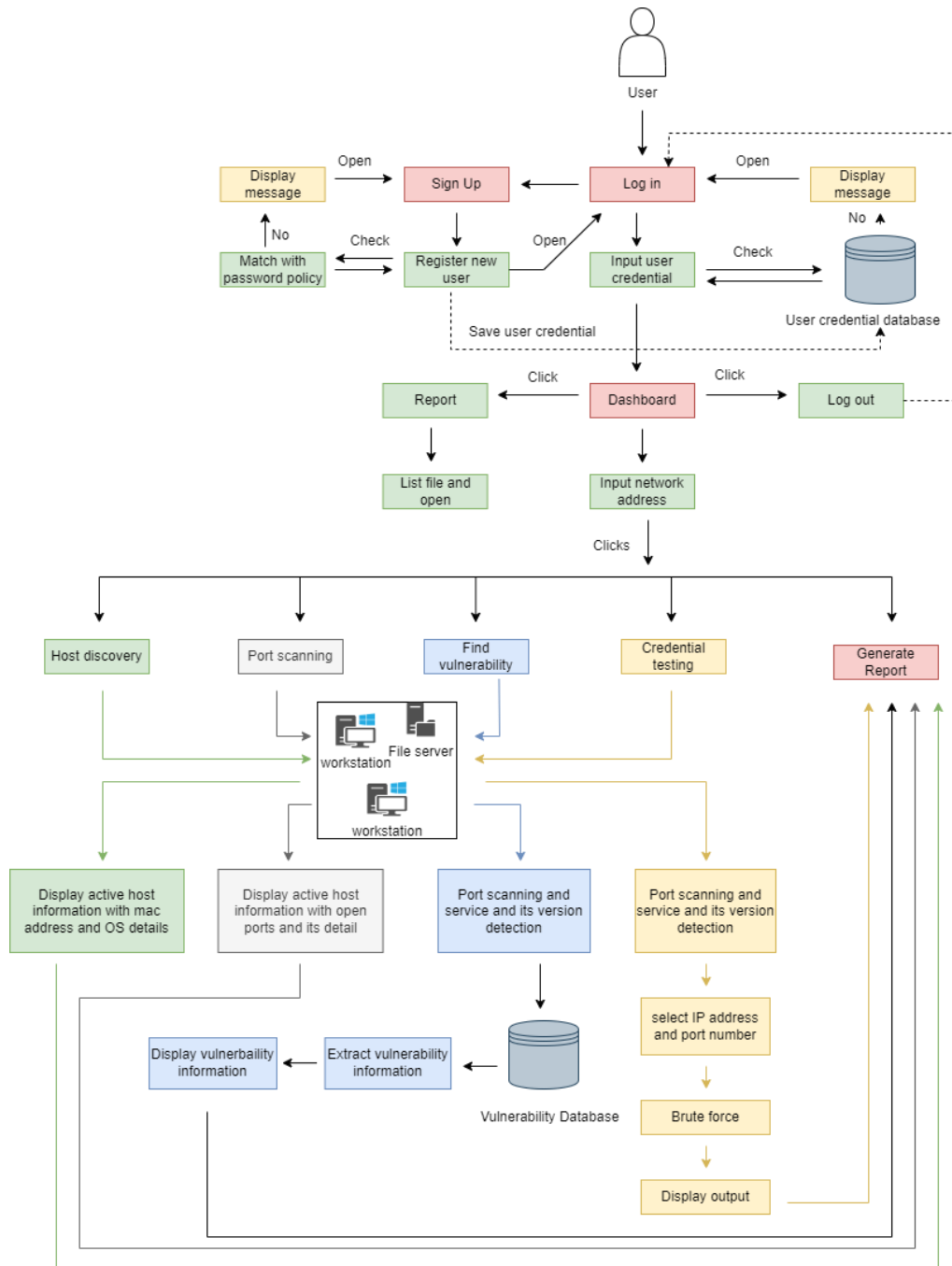


Figure 11: System Architecture

3.6.3 Use Case Diagram

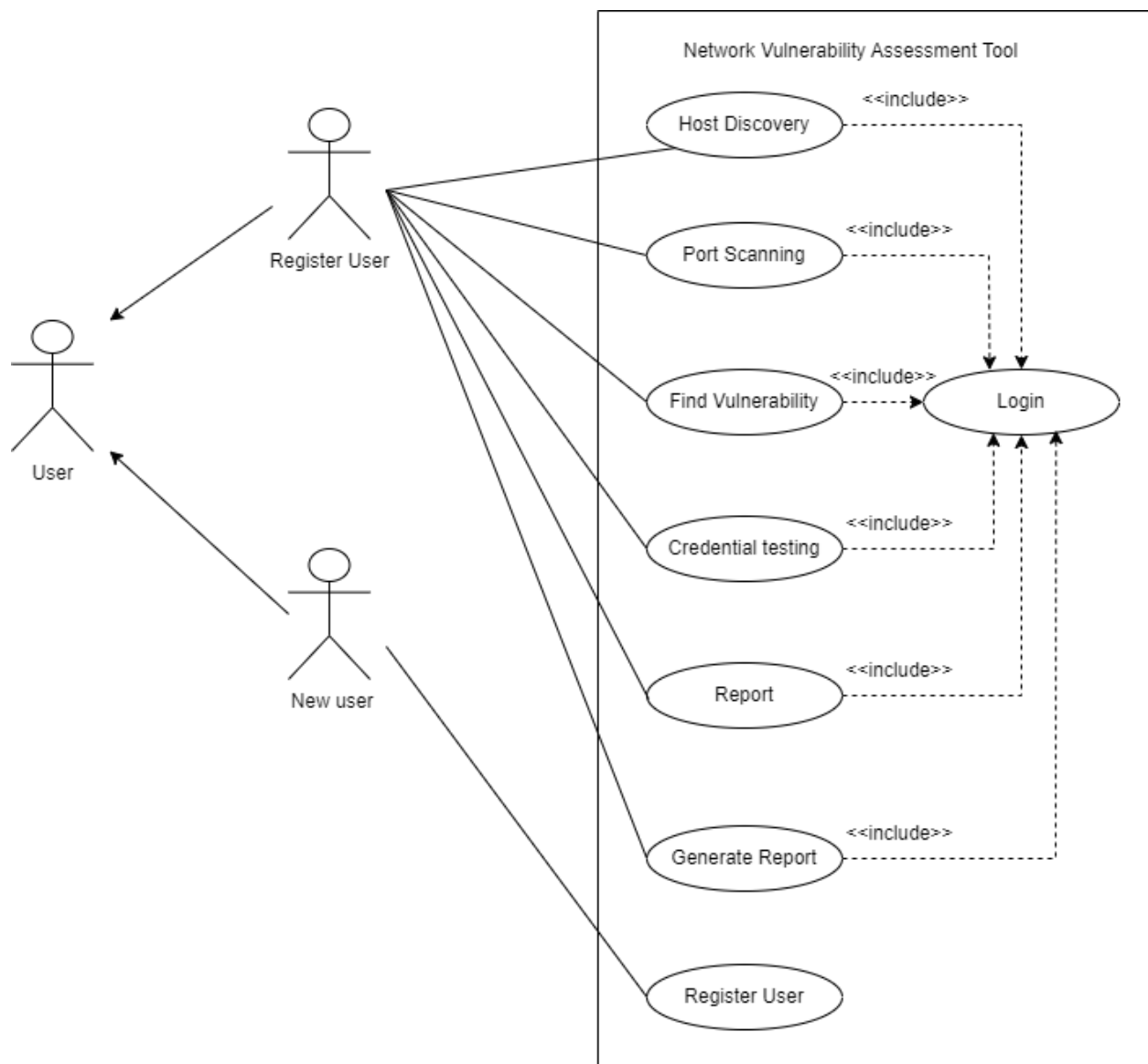
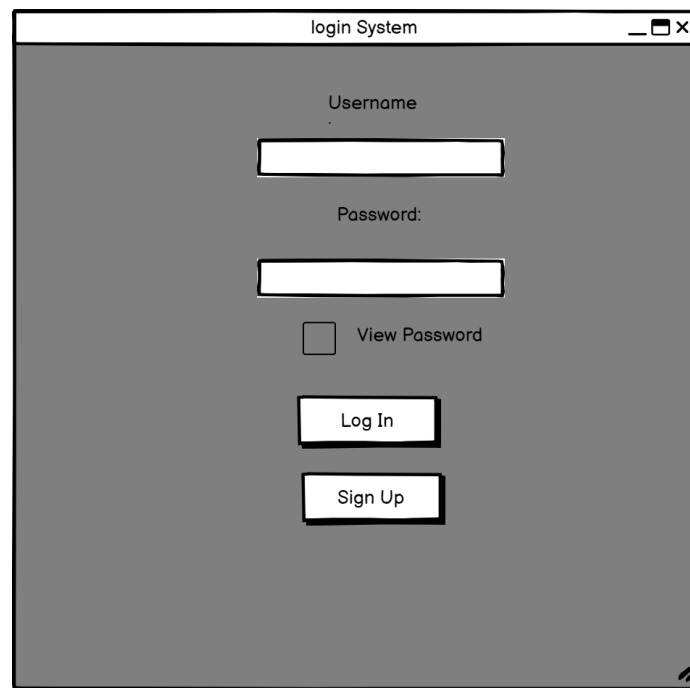


Figure 12: System Use Case Diagram.

3.6.4 Wireframe

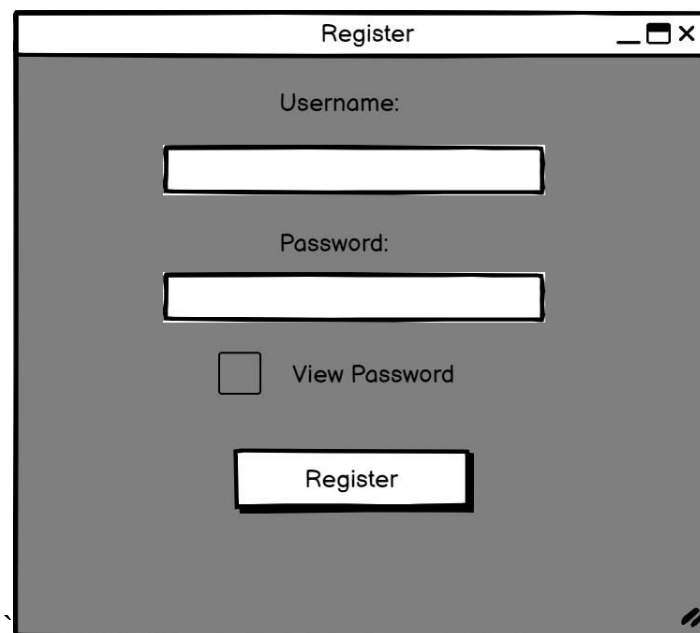
Wireframes serve as fundamental outlines aiding teams in agreeing on requirements, thereby concentrating and enhancing UX design discussions. Consider your wireframe as the foundational framework for your application, website, or any other

end product. It presents the design team and stakeholders with the basic skeletal structure of crucial web pages, components, and functionalities (Figma, 2024).



A wireframe of a login system window. The window has a title bar labeled "login System" with standard minimize, maximize, and close buttons. The main content area is dark gray and contains the following elements: a "Username" label above a white text input field; a "Password:" label above another white text input field; a checkbox labeled "View Password" below the password field; a "Log In" button below the checkbox; and a "Sign Up" button below the "Log In" button. A small cursor icon is visible in the bottom right corner of the window.

Figure 13: Login Wireframe.



A wireframe of a register window. The window has a title bar labeled "Register" with standard minimize, maximize, and close buttons. The main content area is dark gray and contains the following elements: a "Username:" label above a white text input field; a "Password:" label above another white text input field; a checkbox labeled "View Password" below the password field; and a "Register" button below the checkbox. A small cursor icon is visible in the bottom right corner of the window.

Figure 14: Signup Wireframe.

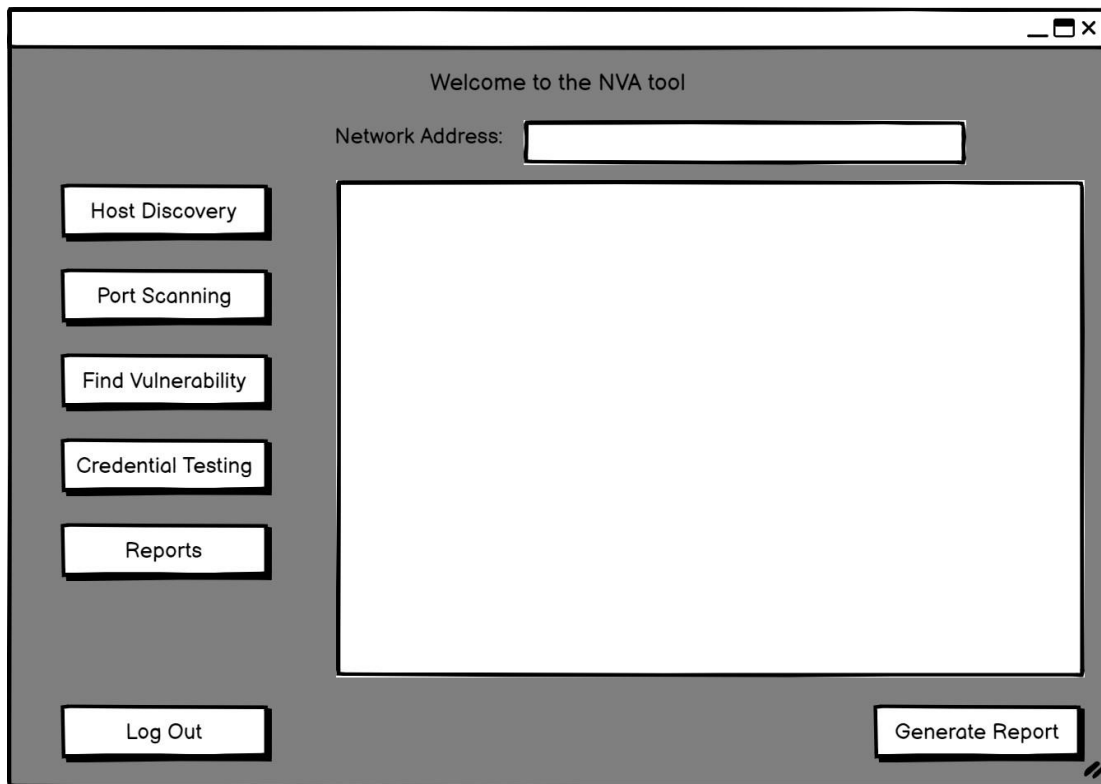


Figure 15: User Dashboard Wireframe

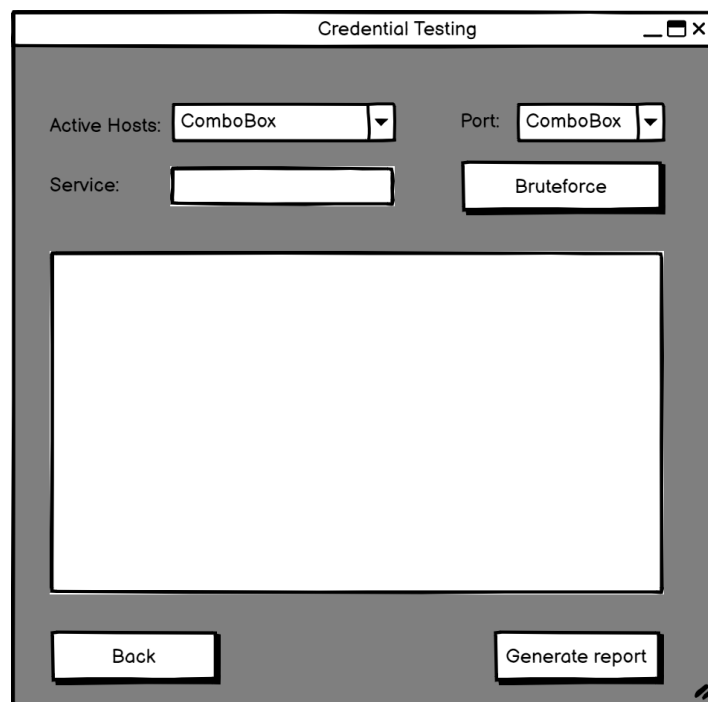


Figure 16: Credential Testing Wireframe.

3.6.5 Data Flow Diagrams

A data flow diagram (DFD) illustrates the movement of information within a process or system. It employs standardized symbols such as rectangles, circles, and arrows, alongside concise text labels, to depict data inputs, outputs, storage locations, and the pathways connecting them. These diagrams vary in complexity, ranging from basic hand-drawn outlines providing an overview of processes to intricate, multi-level DFDs that delve into the detailed handling of data. They serve purposes ranging from analyzing existing systems to modeling new ones (Lucidchart, 2024).

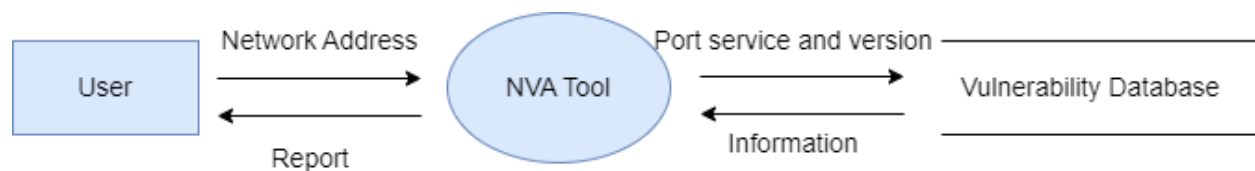


Figure 17: Level 0 System DFD

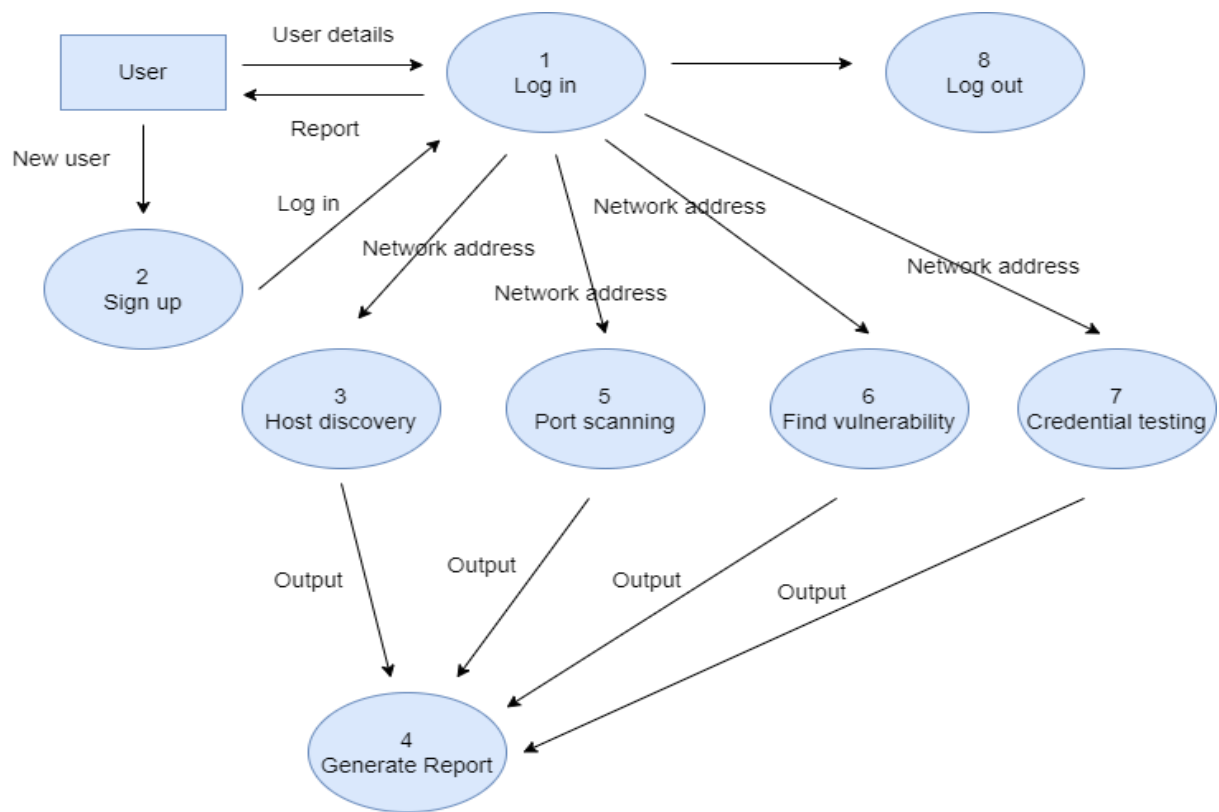


Figure 18: Level 1 DFD of System.

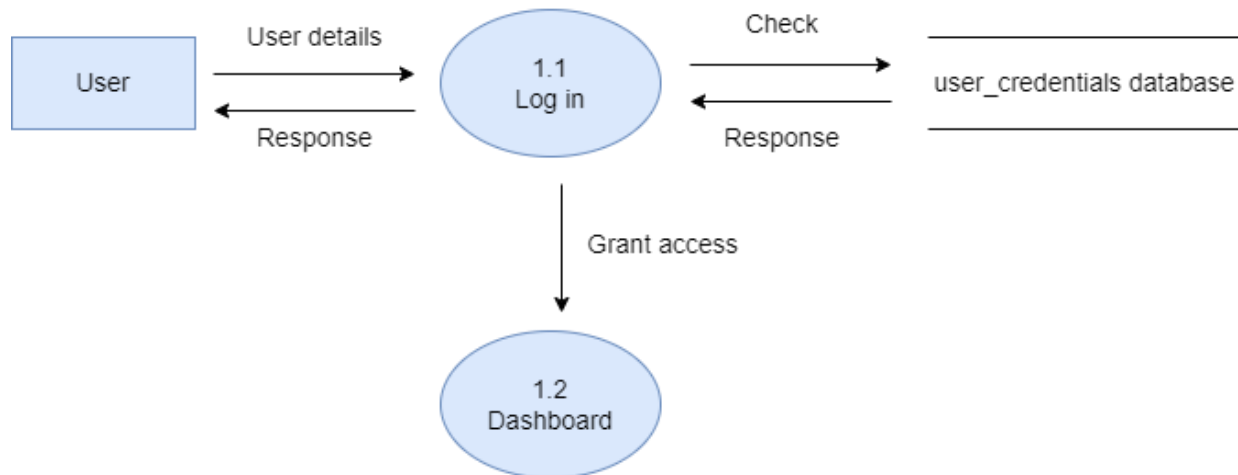


Figure 19: Level 2 DFD of Login.

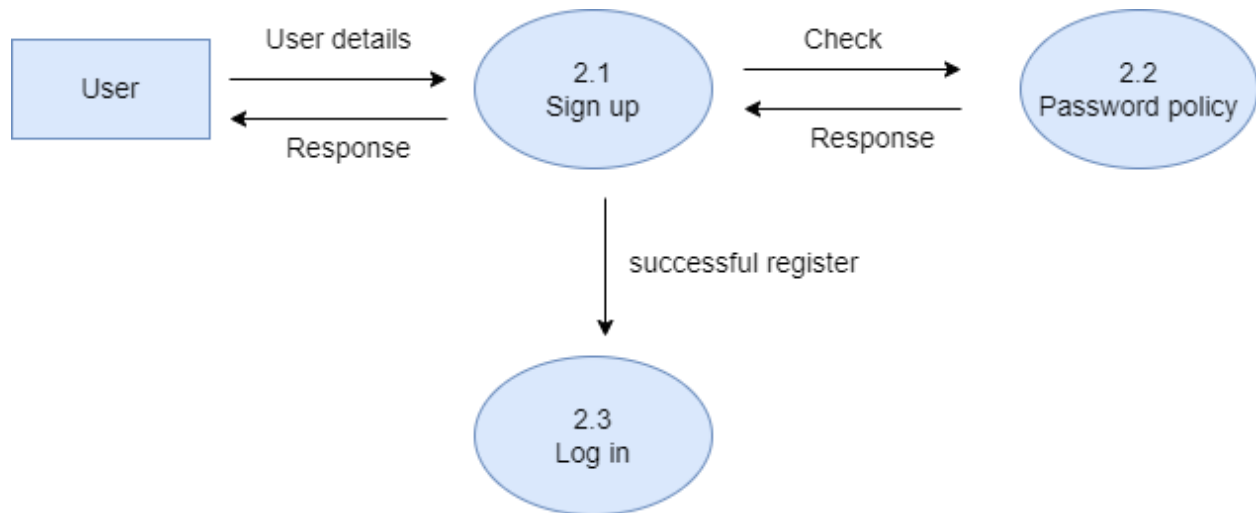


Figure 20: Level 2 DFD of Signup.

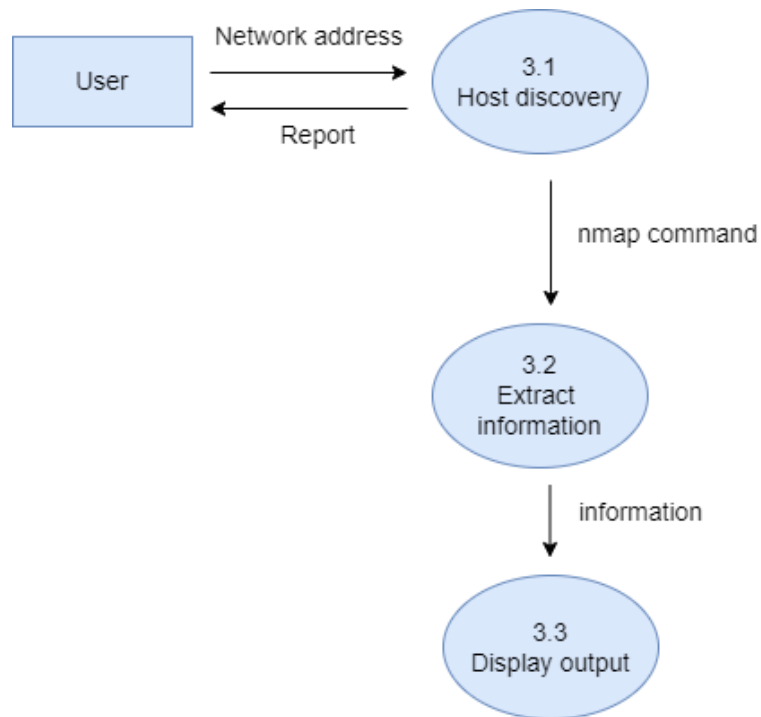


Figure 21: Level 2 DFD of Host Discovery.

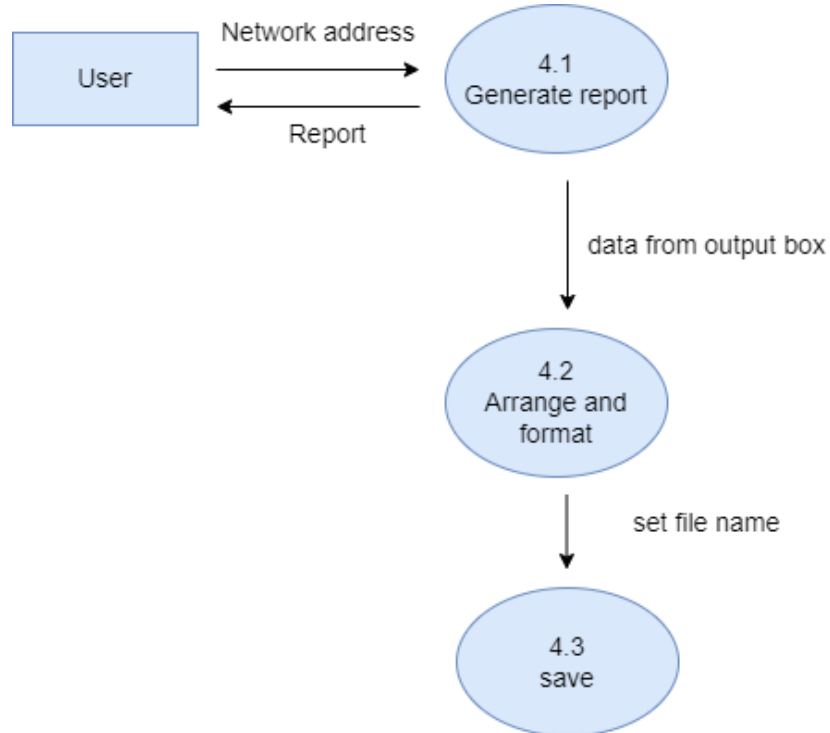


Figure 22: Level 2 DFD of Generate Report.

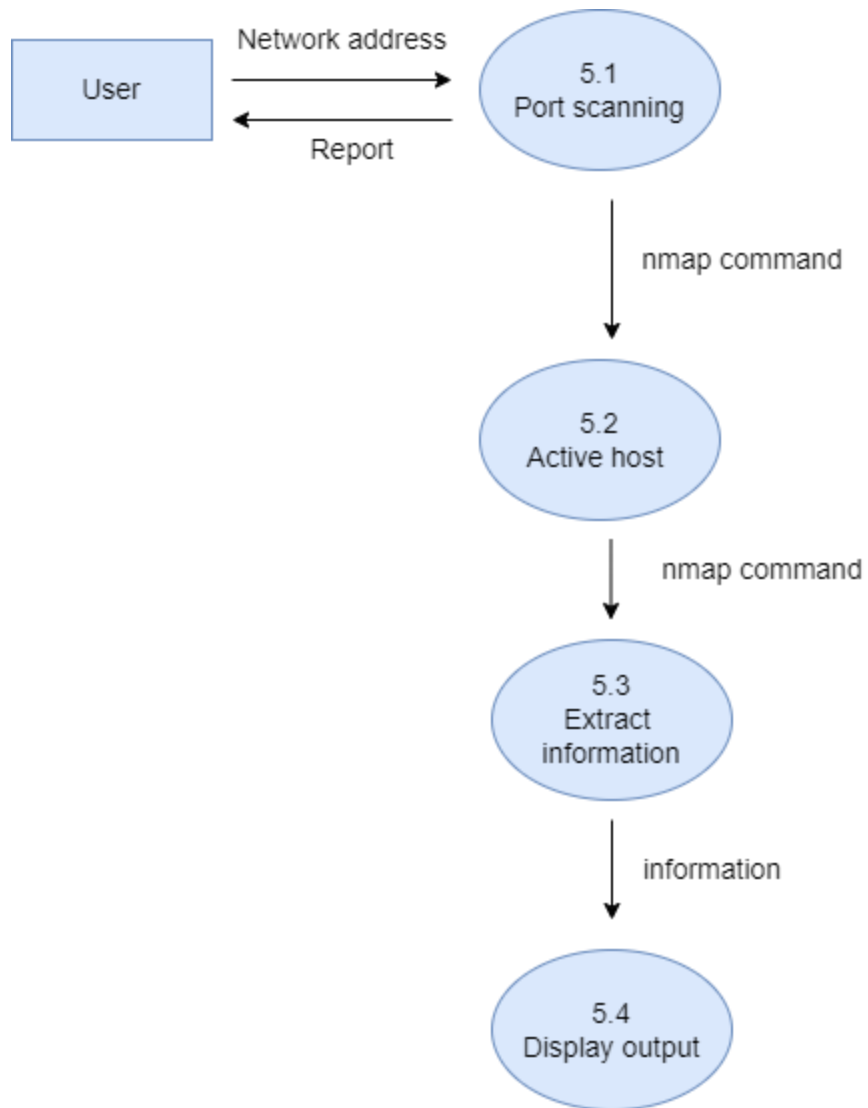


Figure 23: Level 2 DFD of Port Scanning.

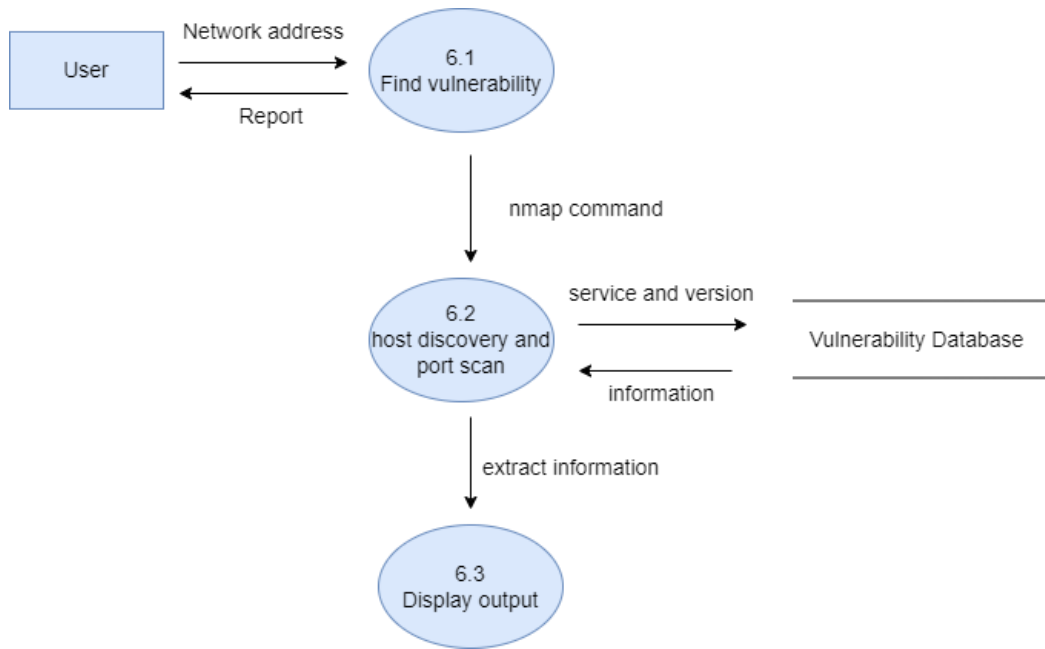


Figure 24: Level 2 DFD of Find Vulnerability.

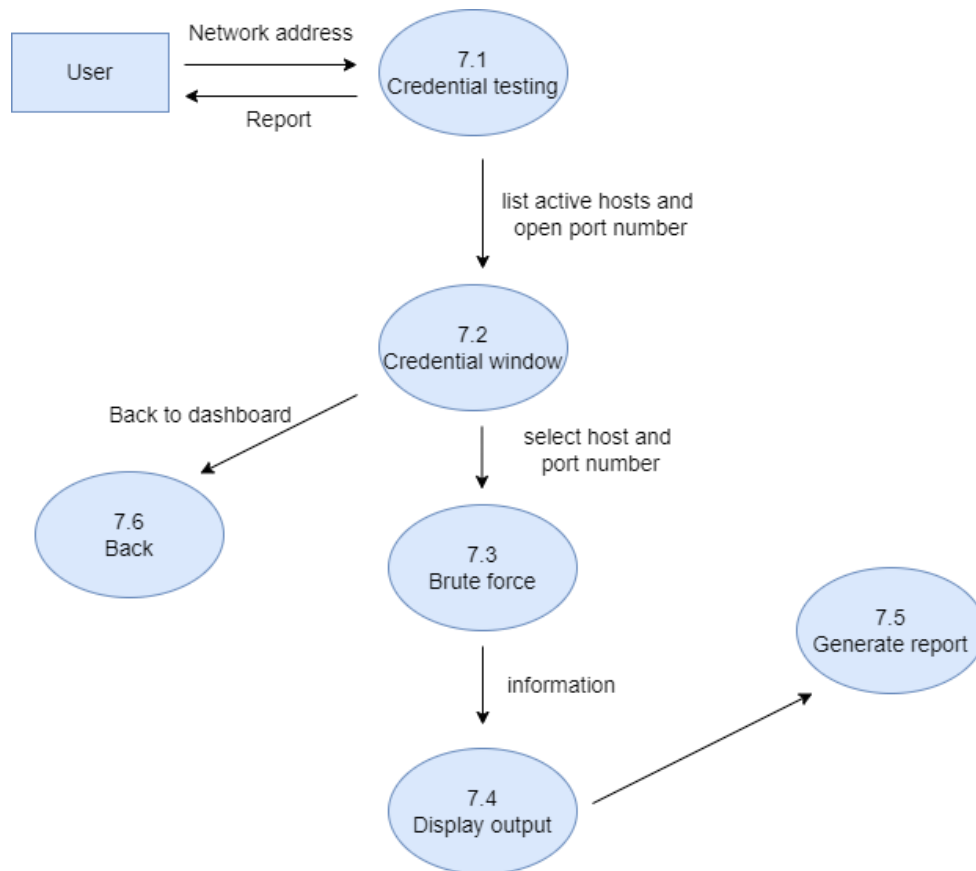


Figure 25: Level 2 DFD of Credential Testing.

3.6.6 Flowcharts

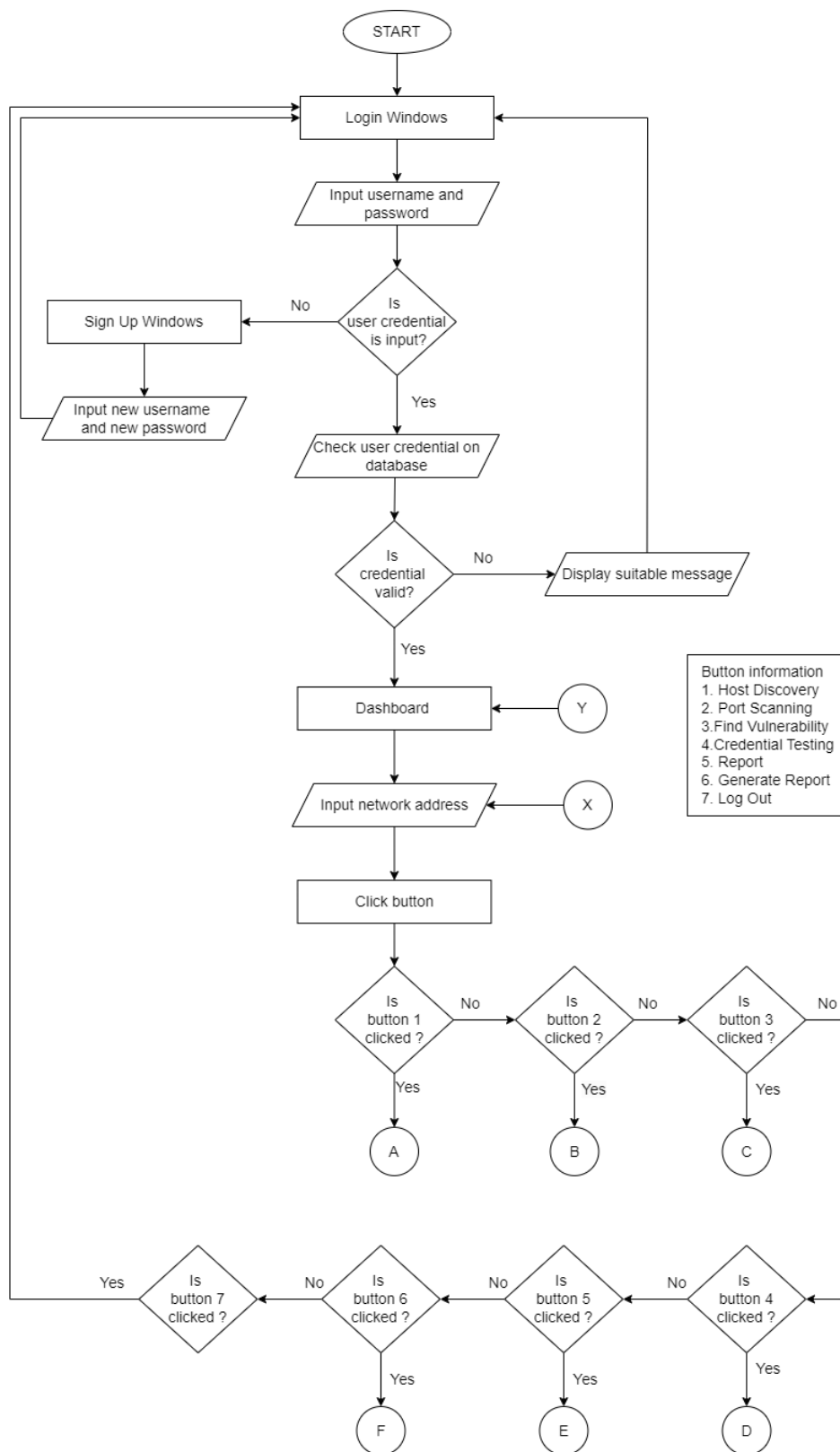


Figure 26: System Flowchart.

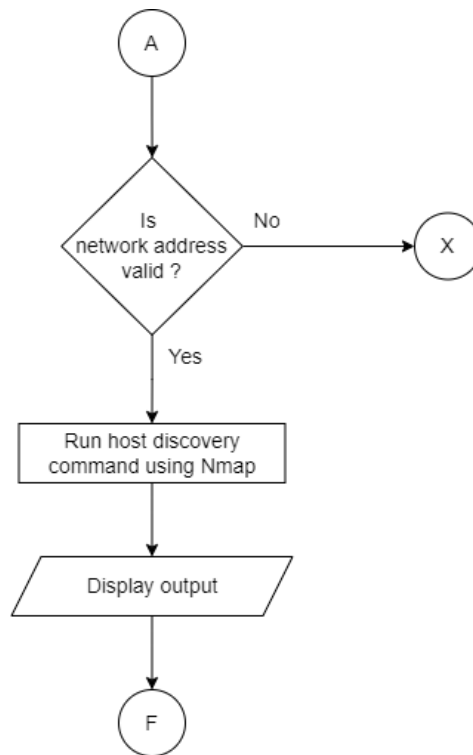


Figure 27: Host Discovery Flowchart.

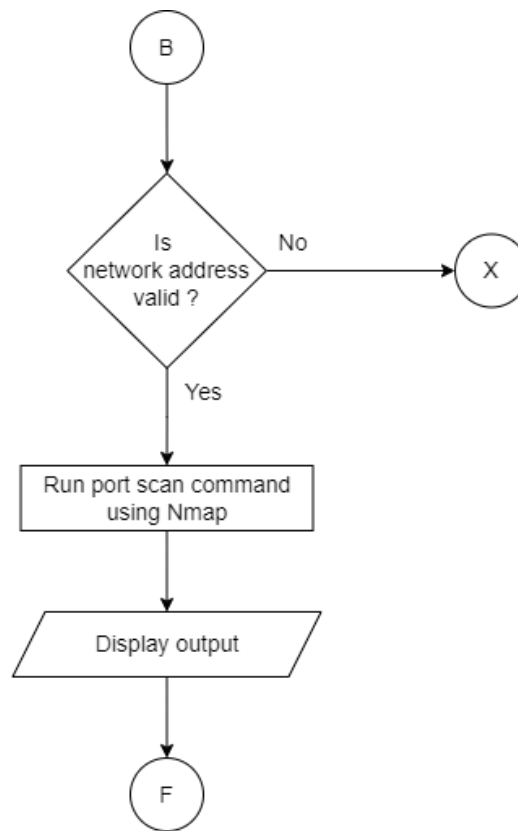


Figure 28: Port Scanning Flowchart.

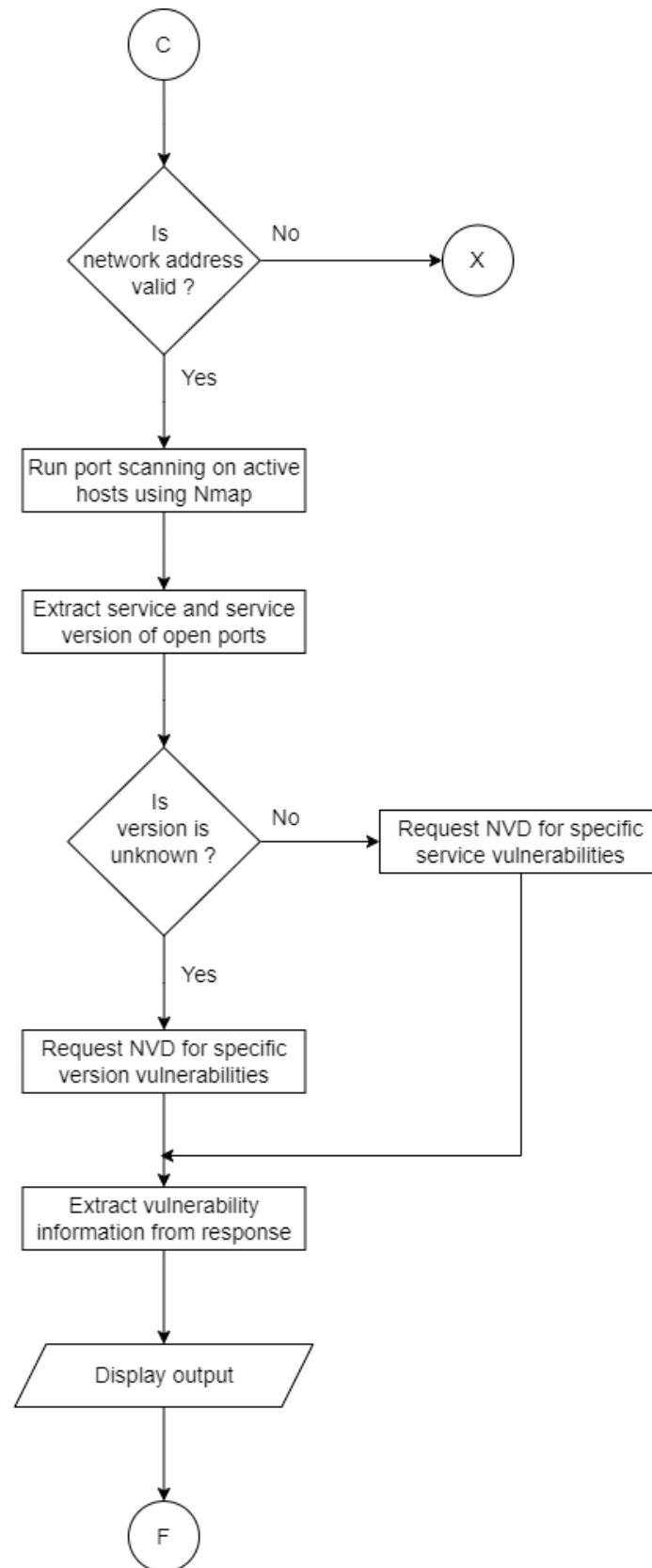


Figure 29: Find Vulnerability Flowchart.

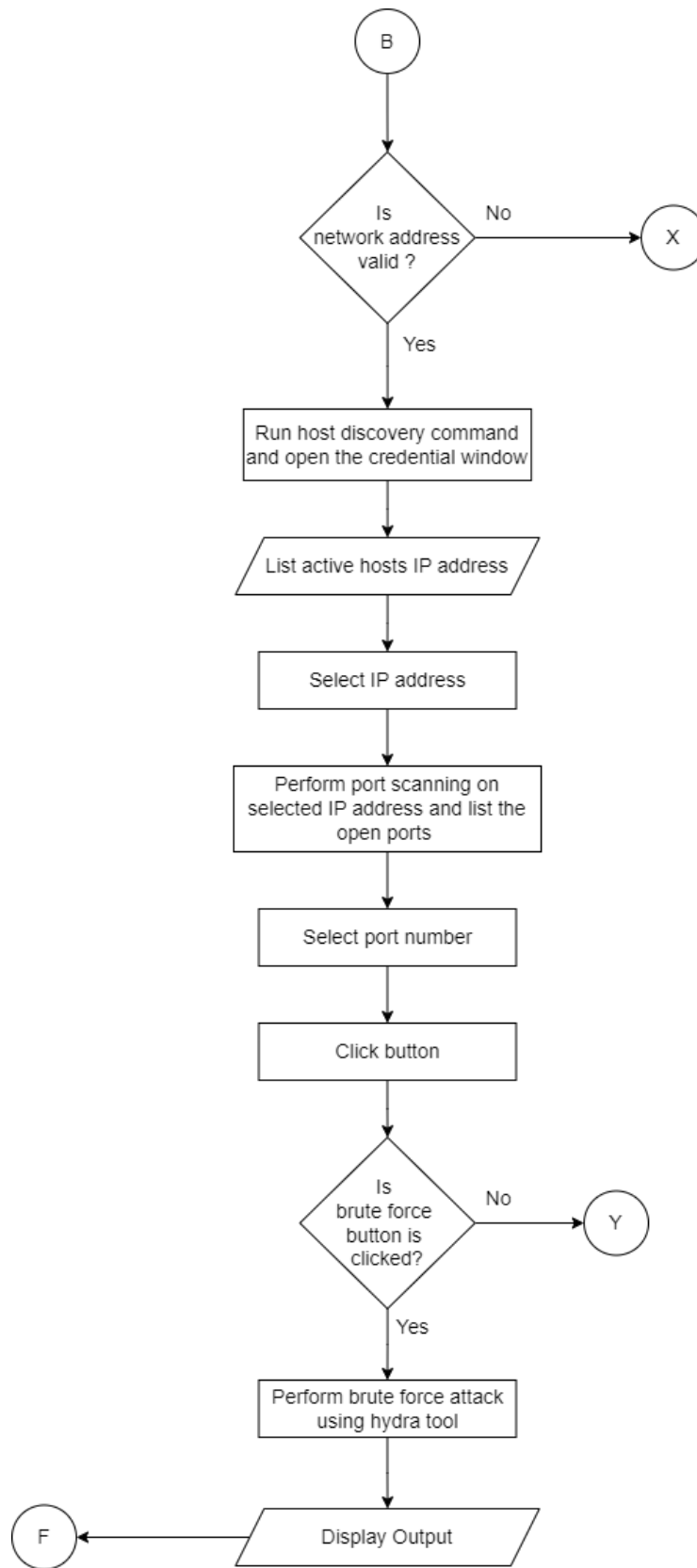


Figure 30: Credential Testing Flowchart.

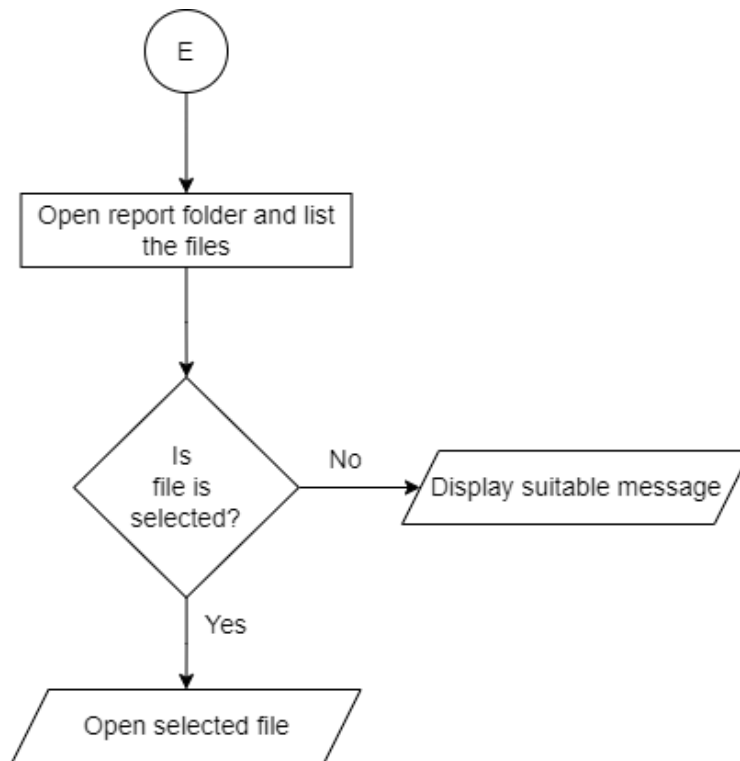


Figure 31: Report Flowchart.

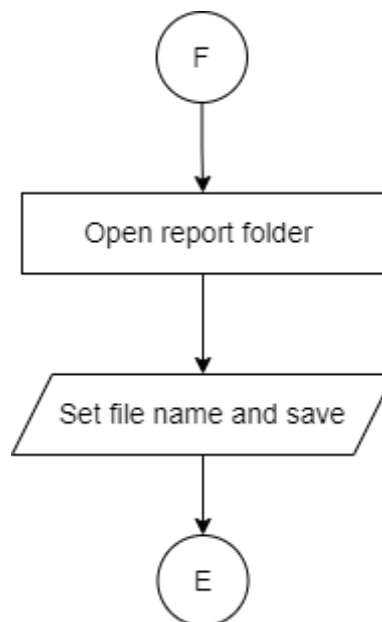


Figure 32: Generate Report Flowchart

For [more](#).

3.7 Implementation

3.7.1 Database

The database used to store user credentials for logging in and registering users plays a critical role in managing user authentication and authorization processes. It serves as the repository for sensitive information such as usernames and passwords for granting access to the system.

```
708
709
710 conn = sqlite3.connect('../database/user_credentials.db')
711 cursor = conn.cursor()
712
713 cursor.execute('''
714     CREATE TABLE IF NOT EXISTS users (
715         username TEXT PRIMARY KEY,
716         password TEXT NOT NULL
717     )
718 ''')
719 conn.commit()
720
```

Figure 33: Code to Store User Credentials on Database.

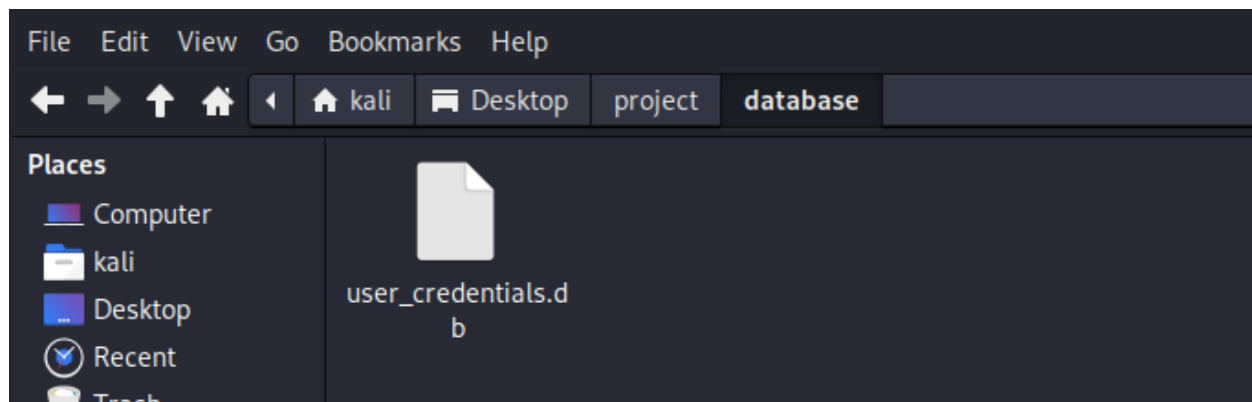


Figure 34: User Credentials Database.

3.7.2 Login GUI

The login GUI serves as the entry point for users to access a system or application by providing a secure interface for entering their credentials, typically a username and password.

```

23
24 def login():
25     username = username_entry.get()
26     password = password_entry.get()
27
28     if not username or not password:
29         messagebox.showerror("Error", "Please enter both username and password.")
30         return
31
32     cursor.execute('SELECT * FROM users WHERE username = ? AND password = ?', (username, password))
33     user = cursor.fetchone()
34
35     if user:
36         messagebox.showinfo("Success", "Login successful!")
37         # Clear the username and password entries
38         username_entry.delete(0, tk.END)
39         password_entry.delete(0, tk.END)
40         open_main_window(username)
41     else:
42         messagebox.showerror("Error", "Invalid username or password.")
43
44 #for login
45 def toggle_view_password_login():
46     # Toggle between password visibility and hidden
47     current_state = password_entry["show"]
48     if current_state:
49         password_entry["show"] = ""
50     else:
51         password_entry["show"] = "*"
52

```

Figure 35: Login Code Part 1.

```

703
704 root = tk.Tk()
705 root.title("Login System")
706 root.geometry("400x300")
707 #root.iconbitmap(r"../images/icon.ico")
708
709

```

Figure 36: Login Code Part 2.

```

720
721 tk.Label(root, text="Username:").pack()
722 username_entry = tk.Entry(root)
723 username_entry.pack()
724
725 tk.Label(root, text="Password:").pack()
726 password_entry = tk.Entry(root, show="*")
727 password_entry.pack()
728
729 view_password_checkbox = tk.Checkbutton(root, text="View Password", command=toggle_view_password_login)
730 view_password_checkbox.pack()
731
732 # Left column buttons
733 button_width = 6 # Set the width of all buttons
734
735 login_button = tk.Button(root, text="Log In", command=login, font=("Arial", 12), activebackground="#45a049", width=button_width)
736 login_button.place(relx=0.5, rely=0.45, anchor='center')
737
738 register_button = tk.Button(root, text="Sign Up", command=open_register_window, font=("Arial", 12), activebackground="#45a049", width=button_width)
739 register_button.place(relx=0.5, rely=0.60, anchor='center')
740
741 root.mainloop()
742
743 conn.close()
744

```

Figure 37: Login Code Part 3.

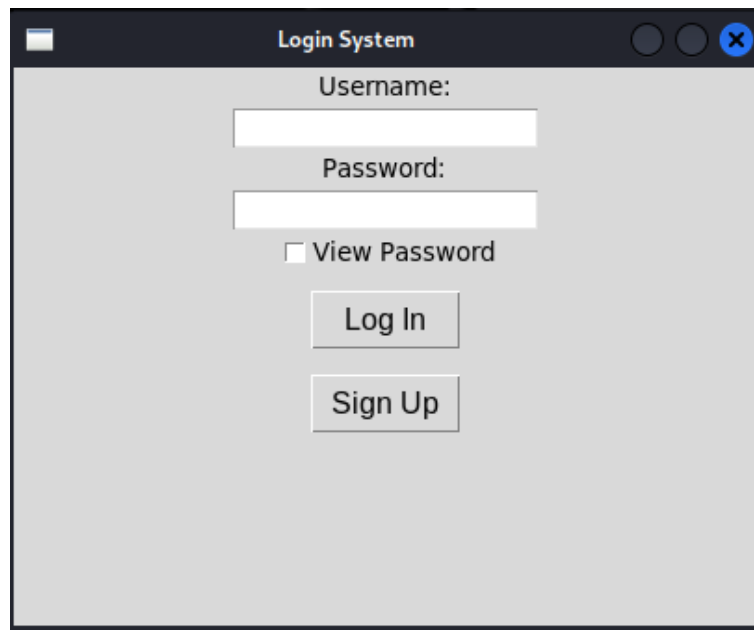


Figure 38: Login GUI

3.7.3 Sign Up GUI

The function of a signup GUI is to facilitate the process of creating a new user account within a system or application. It typically includes fields for users to input their personal information, such as username, and password. Once users fill out the required information and submit it, the signup GUI processes the data and creates a new user account, often with validation checks to ensure data accuracy and security.

```

62 def open_register_window():
63     register_window = tk.Toplevel(root)
64     register_window.title("Register")
65     register_window.geometry("300x200")
66
67     tk.Label(register_window, text="Username:").pack()
68     new_username_entry = tk.Entry(register_window)
69     new_username_entry.pack()
70
71     tk.Label(register_window, text="Password:").pack()
72     new_password_entry = tk.Entry(register_window, show="*")
73     new_password_entry.pack()
74
75     view_password_checkbox = tk.Checkbutton(register_window, text="View Password", command=lambda: toggle_view_password(new_password_entry))
76     view_password_checkbox.pack()
77
78     def submit_registration():
79         new_username = new_username_entry.get()
80         new_password = new_password_entry.get()
81
82         if not new_username or not new_password:
83             messagebox.showerror("Error", "Please enter both username and password.")
84             return
85
86         # Password policy requirements
87         if not (re.search(r"[A-Z]", new_password) and # At least one uppercase letter
88               re.search(r"[a-z]", new_password) and # At least one lowercase letter
89               re.search(r"[0-9]", new_password) and # At least one digit
90               re.search(r"[!@#$%^&*]", new_password) and # At least one special character
91               len(new_password) >= 8): # Minimum length of 8 characters
92             messagebox.showerror("Error", "Password must be at least 8 characters long and contain at least one uppercase letter, one lowercase letter, one digit, and one special character.")
93             return
94
95         try:
96             cursor.execute('INSERT INTO users VALUES (?, ?)', (new_username, new_password))
97             conn.commit()
98             messagebox.showinfo("Success", "Registration successful!")
99             register_window.destroy()
100         except sqlite3.IntegrityError:
101             messagebox.showerror("Error", "Username already exists. Please choose a different username.")
102
103     submit_button = tk.Button(register_window, text="Submit", command=submit_registration)
104     submit_button.pack(pady=10)
105
106

```

Figure 39: Code for Sign Up GUI.

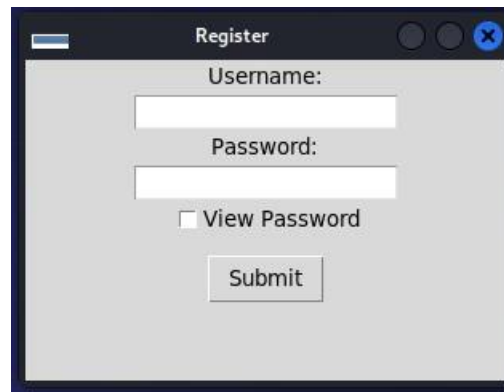


Figure 40: Sign Up GUI

3.7.4 User Dashboard

Dashboard serves as a centralized platform for displaying key metrics and data, providing users with a quick and easy way to analyze and interpret information relevant to their objectives or activities.

```

129 def open_main_window(username):
130     main_window = tk.Toplevel(root)
131     main_window.title(f"NetGuardian")
132     main_window.geometry("700x500")
133
134     # Create and place labels
135     welcome_label = tk.Label(main_window, text="The Guardian of your Network")
136     login_user = tk.Label(main_window, text=f"Hi, {username}")
137     welcome_label.pack(pady=20) # Add vertical gap (20 pixels) below the label
138     login_user.place(x=10, y=12)
139
140     # Left column buttons
141     button_width = 18 # Set the width of all buttons
142
143     # Network Address Label and Entry
144     network_address_label = tk.Label(main_window, text="Network Address:")
145     network_address_label.place(relx=0.34, rely=0.15, anchor='center') # Place at the center
146
147     network_address_entry = tk.Entry(main_window, width=35)
148     network_address_entry.place(relx=0.64, rely=0.15, anchor='center')
149
150     # Output Box
151     output_box = tk.Text(main_window, width=60, height=20, wrap=tk.WORD)
152     output_box.place(relx=0.6, rely=0.55, anchor='center')
153     output_box.configure(state='disabled') # Set the state to disabled
154
155     # buttons
156     host_discovery_button = tk.Button(main_window, text="Host Discovery", command=lambda: host_discovery_wrapper(network_address_entry, output_box), width=button_width)
157     host_discovery_button.place(relx=0.12, rely=0.25, anchor='center')
158
159     port_scanning_button = tk.Button(main_window, text="Port Scanning", command=lambda: port_scanning_wrapper(network_address_entry, output_box), width=button_width)
160     port_scanning_button.place(relx=0.12, rely=0.33, anchor='center')
161
162     find_vulnerability_button = tk.Button(main_window, text="Find Vulnerability", command=lambda: find_vulnerability_wrapper(network_address_entry, output_box), width=button_width)
163     find_vulnerability_button.place(relx=0.12, rely=0.41, anchor='center')
164
165     credential_testing_button = tk.Button(main_window, text="Credential Testing", command=lambda: credential_wrapper(network_address_entry, username), width=button_width)
166     credential_testing_button.place(relx=0.12, rely=0.49, anchor='center')
167
168     report_button = tk.Button(main_window, text="Report", command=report_history, width=button_width)
169     report_button.place(relx=0.12, rely=0.57, anchor='center')
170
171     # Log Out button
172     logout_button = tk.Button(main_window, text="Log Out", command=main_window.destroy, width=button_width)
173     logout_button.place(relx=0.12, rely=0.94, anchor='center')
174
175     # Report button
176     generate_report_button = tk.Button(main_window, text="Generate report", command=lambda: generate_report(network_address_entry, output_box, username), width=18)
177     generate_report_button.place(relx=0.82, rely=0.94, anchor='center')
178
179     main_window.mainloop()
180

```

Figure 41: Code for User Dashboard.

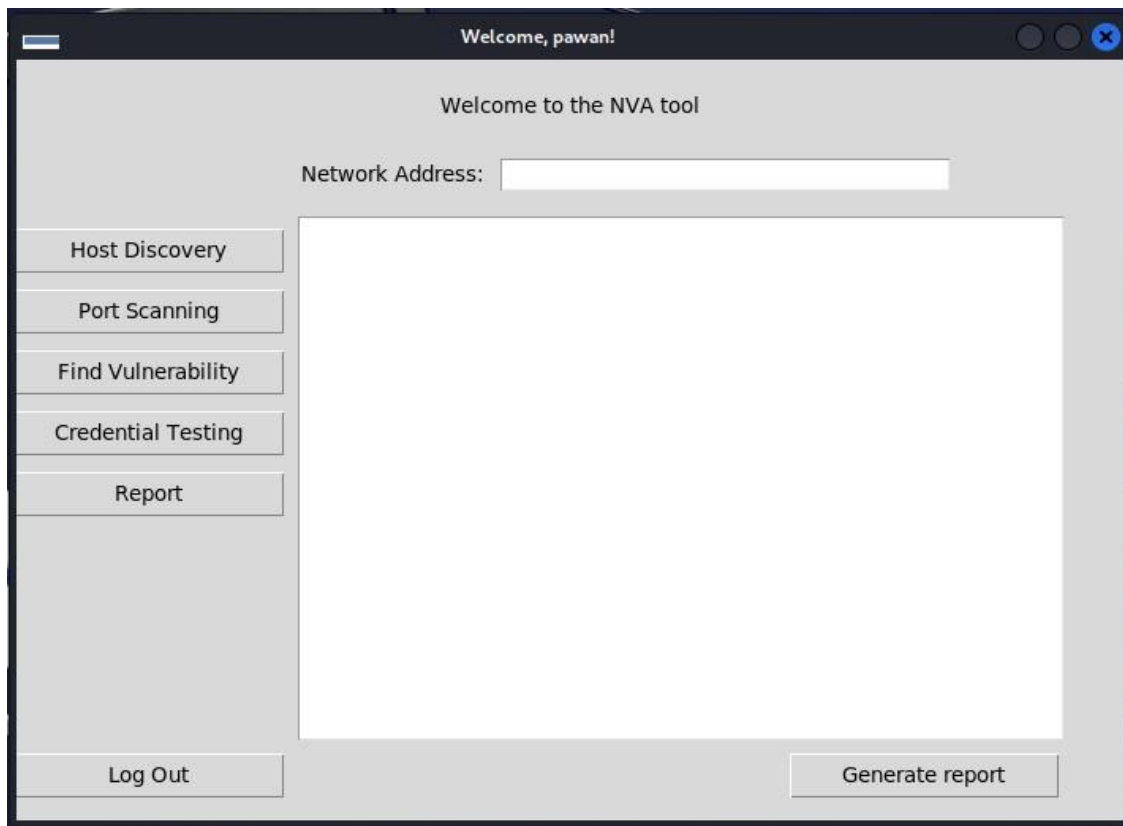


Figure 42: User Dashboard.

3.7.5 Credential Testing GUI

Credential Testing involves attempting to access network-connected devices by systematically trying various username and password combinations, often targeting default or commonly used credentials. This process commonly includes brute-force attacks on specific ports such as 21 (FTP) and 22 (SSH), where default credentials are frequently left unchanged by users or system administrators. By exploiting weak or default authentication information, attackers seek unauthorized access to these devices, potentially compromising the security of the network.

```

432
433 def scan_ports(ip_dropdown,port_dropdown,service_text):
434     selected_ip = ip_dropdown.get()
435     nm = nmap.PortScanner()
436     nm.scan(hosts=selected_ip, arguments='-p 1-1000')
437
438     open_ports = []
439     for host in nm.all_hosts():
440         for proto in nm[host].all_protocols():
441             ports = nm[host][proto].keys()
442             open_ports.extend(ports)
443
444     port_dropdown['values'] = sorted(open_ports)
445
446     service_text.delete(1.0, tk.END) # Clear previous text
447
448 def show_services(ip_dropdown,port_dropdown, service_text):
449     selected_ip = ip_dropdown.get()
450     selected_port = port_dropdown.get()
451
452     nm = nmap.PortScanner()
453     nm.scan(hosts=selected_ip, arguments='-sV -p ' + selected_port)
454
455     service_info = nm[selected_ip]['tcp'][int(selected_port)]
456     service_name = service_info['name']
457     service_text.delete(1.0, tk.END) # Clear previous text
458     service_text.insert(tk.END, service_name)
459

```

Figure 43: Credential Testing Code Part 1.

```

461 def brute_force_attack(ip_dropdown, port_dropdown, output_box, service_text):
462     selected_ip = ip_dropdown.get()
463     selected_port = port_dropdown.get()
464     enable_output_box(output_box)
465     # Clear the output box before displaying new results
466     output_box.delete(1.0, tk.END)
467     # Retrieve the text from the service_text widget
468     selected_service = service_text.get("1.0", "end-1c") # Get all text excluding the trailing newline character
469
470     usernames_list = "../credential/usernames.txt" # Replace with the actual username
471     password_list = "../credential/passwords.txt" # Replace with the path to your password list
472
473     # Execute hydra command for SSH brute force attack
474     command = f"hydra -L {usernames_list} -P {password_list} {selected_ip} -s {selected_port} {selected_service}"
475     output_box.insert(tk.END, f"Performing brute force attack on {selected_ip} and the port is {selected_port}... \n\n")
476     try:
477         result = subprocess.run(command, shell=True, capture_output=True, text=True)
478         if result.returncode == 0:
479             output_box.insert(tk.END, "Brute force attack successful.\n\n")
480
481             # Extract and display usernames and passwords
482             matches = re.findall(r"login:\s+(\S+)\s+password:\s+(\S+)", result.stdout)
483             if matches:
484                 output_box.insert(tk.END, "Found credentials:\n")
485                 for match in matches:
486                     brute_username, password = match
487                     output_box.insert(tk.END, f"Username: {brute_username}, Password: {password}\n")
488             else:
489                 output_box.insert(tk.END, "No credentials found.\n")
490         else:
491             output_box.insert(tk.END, f"Error: {result.stderr}\n")
492     except Exception as e:
493         output_box.insert(tk.END, f"Error: {e}\n")
494
495     output_box.insert(tk.END, "Brute force attack completed.\n")
496     disable_output_box(output_box)
497

```

Figure 44: Credential Testing Code Part 2.

```

498 def open_credential_window(network_address_entry, active_hosts, username):
499     credential_window = tk.Toplevel(root)
500     credential_window.title("Credential Testing")
501     credential_window.geometry("400x400")
502
503     #set for ip dropdown and lable
504     ip_dropdown = ttk.Combobox(credential_window, width = 13, state="readonly")
505     #ip_dropdown.set(active_hosts[0]) # Set default active host
506     ip_dropdown.place(relx=0.27, rely=0.1)
507     ip_label = tk.Label(credential_window, text="Active Hosts:")
508     ip_label.place(relx=0.02, rely=0.10)
509     ip_dropdown.bind("<<ComboboxSelected>>", lambda event: scan_ports(ip_dropdown, port_dropdown, service_text))
510     ip_dropdown['values'] = active_hosts
511
512
513     #setting for port dropdown and label
514     port_dropdown = ttk.Combobox(credential_window, width = 9, state="readonly") # Set the state to readonly
515     port_dropdown.set('') # Initially empty until host selected
516     port_dropdown.place(relx=0.7, rely=0.1)
517     port_label = tk.Label(credential_window, text="Port:")
518     port_label.place(relx=0.6, rely=0.1)
519     port_dropdown.bind("<<ComboboxSelected>>", lambda event: show_services(ip_dropdown, port_dropdown, service_text))
520
521
522     #setting for service lable
523     service_label = tk.Label(credential_window, text="service:")
524     service_label.place(relx=0.02, rely=0.2)
525     service_text = tk.Text(credential_window, height=1, width=14, state="normal")
526     service_text.place(relx=0.27, rely=0.2)
527     #service_text.configure(state="disable")
528
529     # Output Box
530     output_box = tk.Text(credential_window, width=40, height=13, wrap=tk.WORD)
531     output_box.place(relx=0.5, rely=0.58, anchor='center')
532     output_box.configure(state='disabled') # Set the state to disabled
533
534     # Bruteforce button
535     Brute_force = tk.Button(credential_window, text="Bruteforce", command=lambda: brute_force_attack(ip_dropdown, port_dropdown, output_box, service_text), width=12)
536     Brute_force.place(relx=0.77, rely=0.22, anchor='center')
537
538     # Generate report button
539     credential_report_button = tk.Button(credential_window, text="Generate report", command=lambda: generate_report(network_address_entry, output_box, username), width=12)
540     credential_report_button.place(relx=0.78, rely=0.94, anchor='center')
541
542     # back button
543     back_button = tk.Button(credential_window, text="Back", command=credential_window.destroy, width=12)
544     back_button.place(relx=0.2, rely=0.94, anchor='center')
545

```

Figure 45: Credential Testing Code Part 3.

```

534     # Bruteforce button
535     Brute_force = tk.Button(credential_window, text="Bruteforce", command=lambda: brute_force_attack(ip_dropdown, port_dropdown, output_box, service_text), width=12)
536     Brute_force.place(relx=0.77, rely=0.22, anchor='center')
537
538     # Generate report button
539     credential_report_button = tk.Button(credential_window, text="Generate report", command=lambda: generate_report(network_address_entry, output_box, username), width=12)
540     credential_report_button.place(relx=0.78, rely=0.94, anchor='center')
541
542     # back button
543     back_button = tk.Button(credential_window, text="Back", command=credential_window.destroy, width=12)
544     back_button.place(relx=0.2, rely=0.94, anchor='center')
545
546
547 def credential_wrapper(network_address_entry, username):
548     if validate_network_address(network_address_entry):
549         scan_network(network_address_entry, username)
550         #disable_output_box(output_box)
551

```

Figure 46: Credential Testing Code Part 4.

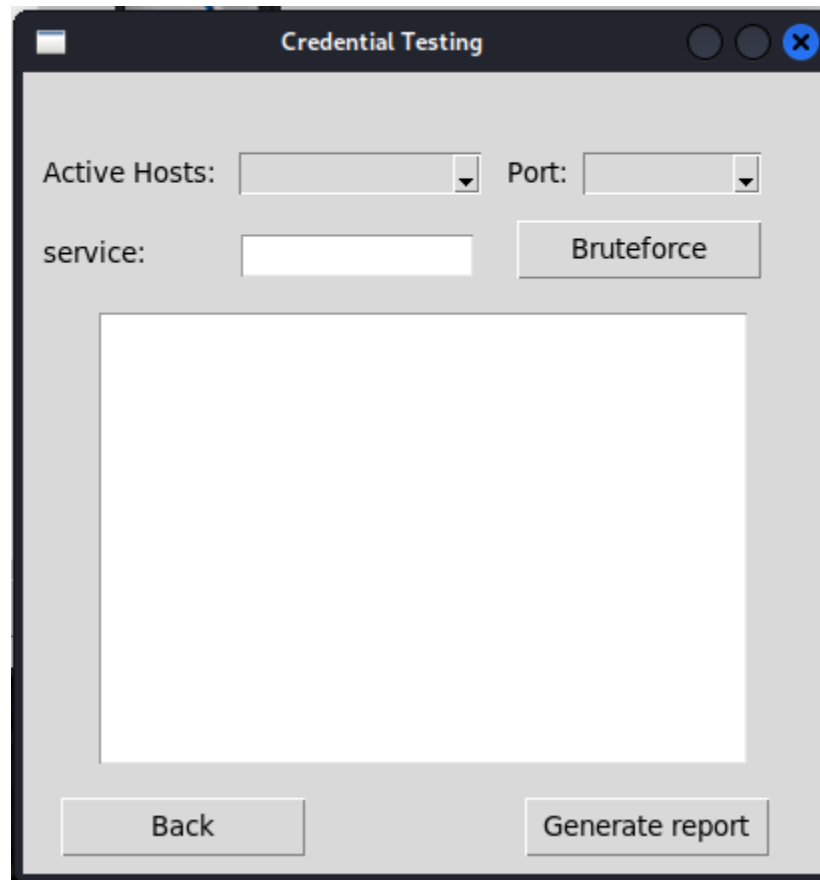


Figure 47: Credential Testing GUI.

3.7.6 Log Out Button

```
# Log Out button
logout_button = tk.Button(main_window, text="Log Out", command=main_window.destroy, width=button_width)
logout_button.place(relx=0.12, rely=0.94, anchor='center')
# Report button
```

Figure 48: Code of Logout Button.

3.7.7 Back Button

```
# back button
back_button = tk.Button(credential_window, text="Back", command=credential_window.destroy, width=12)
back_button.place(relx=0.2, rely=0.94, anchor='center')
```

Figure 49: Code of Back Button.

3.7.8 Generate Report Button

Generating the report involves creating the report based on the provided input data. This process entails arranging and formatting the output in a structured manner, ultimately converting it into a PDF file for ease of dissemination and presentation.

```

3 def generate_report(network_address_entry, output_box, username):
4     # Get the current date
5     current_date = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
6
7     # Get the network address and output text
8     network_address = network_address_entry.get().replace('/', '-')
9     report_text = output_box.get("1.0", tk.END)
10
11     # Check if the output box is empty
12     if not report_text.strip():
13         messagebox.showerror("Error", "Output box is empty. Please perform actions before generating a report.")
14         return
15     report_folder = '../reports'
16
17     try:
18         # List all files in the reports folder
19         pdf_files = [f for f in os.listdir(report_folder) if f.endswith(".pdf")]
20         # Prompt the user for the file name and location
21         file_path = filedialog.asksaveasfilename(
22             initialdir=report_folder,
23             defaultextension=".pdf",
24             filetypes=[("PDF files", "*.pdf")],
25             title="Save Report As"
26         )
27
28     if file_path:
29         # Create the PDF document
30         pdf_canvas = canvas.Canvas(file_path, pagesize=letter)
31
32         # Set margins
33         left_margin = 72
34         right_margin = letter[0] - 72
35         top_margin = letter[1] - 72
36         bottom_margin = 72
37
38         # Add metadata to the PDF
39         pdf_canvas.setTitle(f"Report - {network_address}")
40
41         # Add header only on the first page
42         pdf_canvas.setFont("Helvetica-Bold", 14)
43
44         # Draw company logo in the left section of the header
45         draw_company_logo(pdf_canvas, left_margin, top_margin)
46
47         # Continue with the rest of the header
48         pdf_canvas.drawCentredString((left_margin + right_margin) / 2, top_margin - 20, "Report")
49         top_margin -= 40

```

Figure 50: Generate Report code Part-1.

```

600         top_margin -= 40
601
602         pdf_canvas.setFont("Helvetica", 12)
603
604         # Add username, network address, and date only on the first page
605         pdf_canvas.drawString(left_margin, top_margin, f"User: {username}")
606         top_margin -= 15
607         pdf_canvas.drawString(left_margin, top_margin, f"Network Address: {network_address}")
608         top_margin -= 15
609         pdf_canvas.drawString(left_margin, top_margin, f>Date: {current_date}")
610         top_margin -= 20
611
612         # Add page number in the right section of the footer on the first page
613         pdf_canvas.drawRightString(right_margin, bottom_margin - 20, "Page 1")
614
615         # Split the report text into lines and add them to the PDF
616         lines = report_text.split("\n")
617
618         line_spacing = 1.5 # Adjust this value for the desired line spacing
619
620         page_number = 1
621         for line in lines:
622             # Check if there's enough space for the current line
623             if top_margin - bottom_margin <= 0:
624                 pdf_canvas.showPage() # Start a new page
625                 page_number += 1
626                 top_margin = letter[1] - 25 # Reset the position for the new page
627
628                 # Add metadata to the PDF for subsequent pages
629                 pdf_canvas.setTitle(f"Report - {network_address}")
630
631                 # Add header for the new page
632                 pdf_canvas.setFont("Helvetica-Bold", 14)
633                 top_margin -= 40
634
635                 pdf_canvas.setFont("Helvetica", 12)
636
637                 # Add page number in the right section of the footer for subsequent pages
638                 pdf_canvas.drawRightString(right_margin, bottom_margin - 20, f"Page {page_number}")
639
640             # Calculate the width of the line
641             line_width = pdf_canvas.stringWidth(line.strip(), "Helvetica", 12)
642             # Check if the line exceeds the right margin
643             if left_margin + line_width > right_margin:
644                 # Split the line into words
645                 words = line.split()
646                 wrapped_line = ""
647                 for word in words:
648                     # Check if adding the word exceeds the right margin

```

Figure 51: Generate Report code Part-2

```

648                 # Check if adding the word exceeds the right margin
649                 if pdf_canvas.stringWidth(wrapped_line + word, "Helvetica", 12) <= right_margin - left_margin:
650                     wrapped_line += word + " "
651                 else:
652                     # Draw the wrapped line
653                     pdf_canvas.drawString(left_margin, top_margin, wrapped_line.strip())
654                     top_margin -= 12 * line_spacing # Adjusting line spacing
655                     wrapped_line = word + " "
656                 # Draw the remaining wrapped line
657                 pdf_canvas.drawString(left_margin, top_margin, wrapped_line.strip())
658                 top_margin -= 12 * line_spacing # Adjusting line spacing
659             else:
660                 pdf_canvas.drawString(left_margin, top_margin, line.strip())
661                 top_margin -= 12 * line_spacing # Adjusting line spacing
662
663         pdf_canvas.save()
664
665         messagebox.showinfo("Report", f"Report generated successfully: {file_path}")
666
667     except Exception as e:
668         messagebox.showerror("Error", f"Failed to generate report: {e}")
669

```

Figure 52: Generate Report code Part-3.

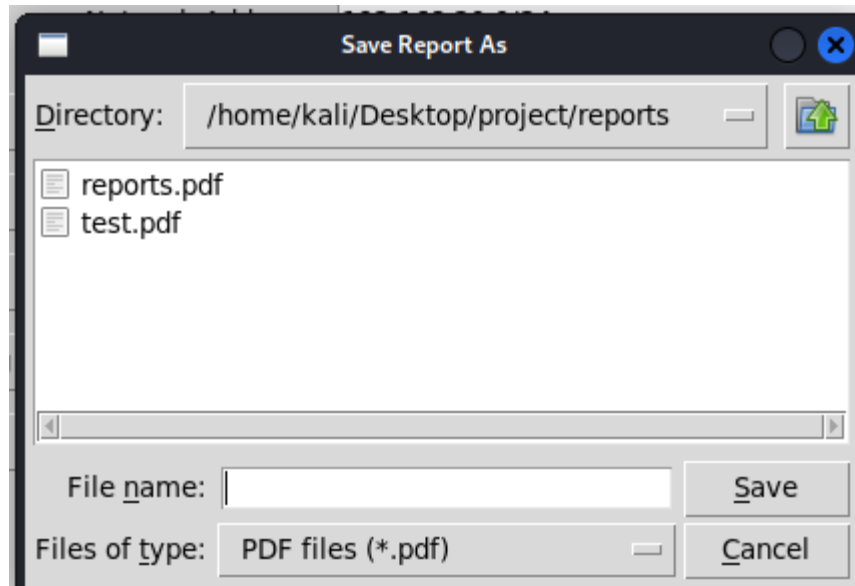


Figure 53: Save Report As.

3.7.9 Report Button

'Report' button will display a list of all the PDF files located within the 'reports' folder.

```

672
673 def report_history():
674     report_folder = r"../reports"
675     try:
676         # List all files in the reports folder
677         pdf_files = [f for f in os.listdir(report_folder) if f.endswith(".pdf")]
678
679         if pdf_files:
680             file_path = filedialog.askopenfilename(
681                 initialdir=report_folder,
682                 title="Select a PDF file",
683                 filetypes=[("PDF files", "*.pdf")],
684                 defaultextension=".pdf"
685             )
686
687             if file_path:
688                 # Determine the platform (Windows or Unix-like)
689                 if platform.system().lower() == 'windows':
690                     # On Windows, use the "start" command
691                     subprocess.run(["start", "", file_path], shell=True)
692                 else:
693                     # On Unix-like systems, use the "xdg-open" command
694                     subprocess.run(["xdg-open", file_path])
695             else:
696                 messagebox.showinfo("No File Selected", "No PDF file selected.")
697         else:
698             messagebox.showinfo("No PDF Files", f"No PDF files found in {report_folder}")
699
700     except Exception as e:
701         messagebox.showerror("Error", f"Failed to list report files: {e}")
702

```

Figure 54: Code to List the Files.

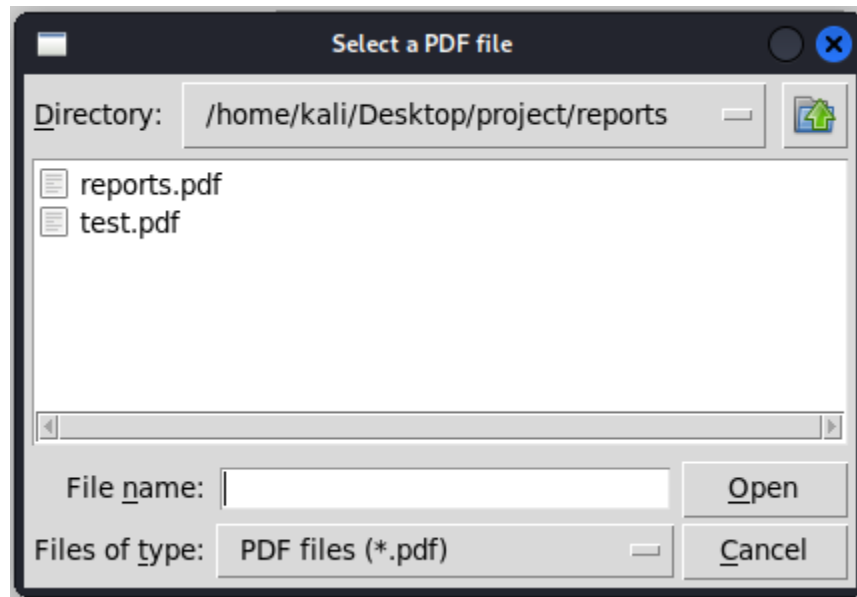


Figure 55: Select a PDF File

4 Testing and Analysis

4.1 Test Plan

4.1.1 Unit Testing, Test Plan

A unit test is a form of software testing aimed at assessing specific elements within a software system. These elements, termed 'units,' encompass functions, procedures, methods, objects, or other components found in the source code of an application. The determination of which unit to prioritize for comprehension and testing rests with each development team, tailored to suit their system's needs (Bakharev, 2024).

The plan is to prioritize testing critical functionalities such as login, sign-up, host discovery, port scanning, find vulnerability, report generation, credential testing, and navigation controls like the back button. Test cases will be designed to validate input validation, error handling, boundary conditions, and expected outcomes.

4.1.2 System Testing, Test Plan

System testing refers to the testing of a fully integrated software product in its entirety. This form of testing operates within the framework of black-box testing, meaning that prior knowledge of the code's internal design is not necessary, and it is conducted independently by the testing team (The Economic Times, 2024).

4.2 Unit Testing

4.2.1 Evolution 1

4.2.1.1 Error Handling on Sign Up Button (1)

Objective	To test the “Sign Up” button to show the error if the username and password are empty.
Action	Click the “Sign Up” button and then the Register windows will open and click the “Submit” button.
Expected Results	The button should be able to show an error message if the username and the password are empty.
Actual Results	The button was able to show an error message.
Conclusion	The test was successful.

Table 2: Error Handling on Sign Up Button (1).

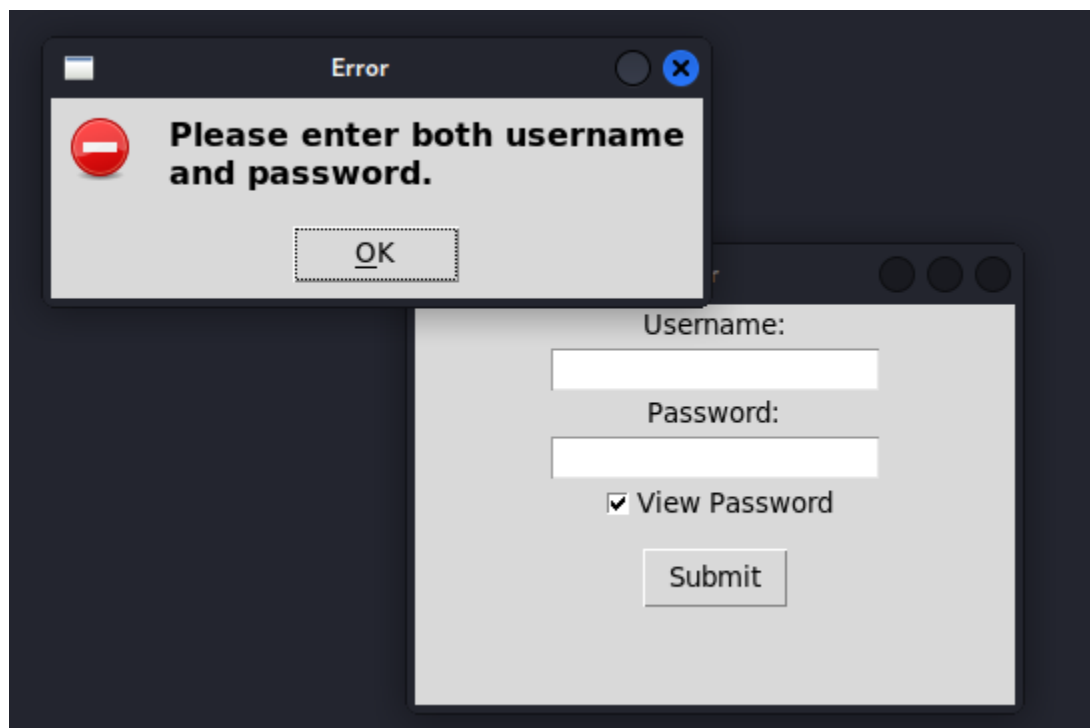


Figure 56: Error Handling on Sign Up Button (1).

4.2.1.2 Error Handling on Sign Up Button (2)

Objective	To test the "Sign Up" button to show the error if the password does not match the password policy.
Action	Click the "Sign Up" button and the Register window will open then enter username and password and then click the "Submit" button.
Expected Results	The button should be able to pop up an error message if the password doesn't match the password policy.
Actual Results	The button was able to pop up an error message.
Conclusion	The test was successful.

Table 3: Error Handling on Sign Up Button (2).

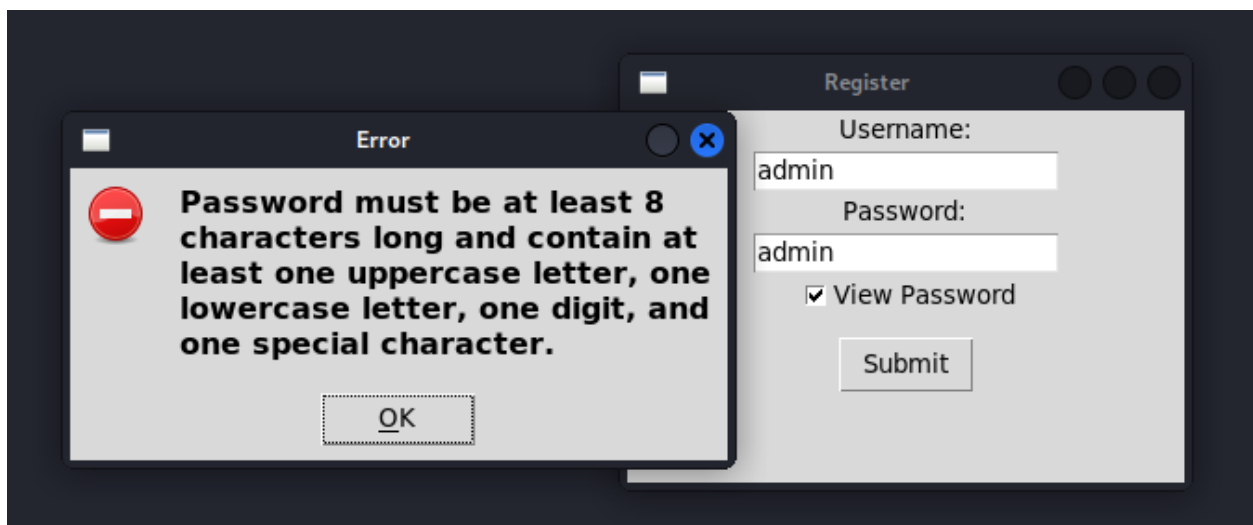


Figure 57: Error Handling on Sign Up Button (2).

4.2.1.3 Sign Up Button

Objective	To test the “Sign Up” button to register a new user.
Action	Click the “Sign Up” button and the Register window will open then enter username and password and click the “Submit” button.
Expected Results	The new user should be registered on the system and a “Registration successful” message should pop up.
Actual Results	The new user was able to register on the system and the “Registration successful” message popped up.
Conclusion	The test was successful.

Table 4: Register New User.

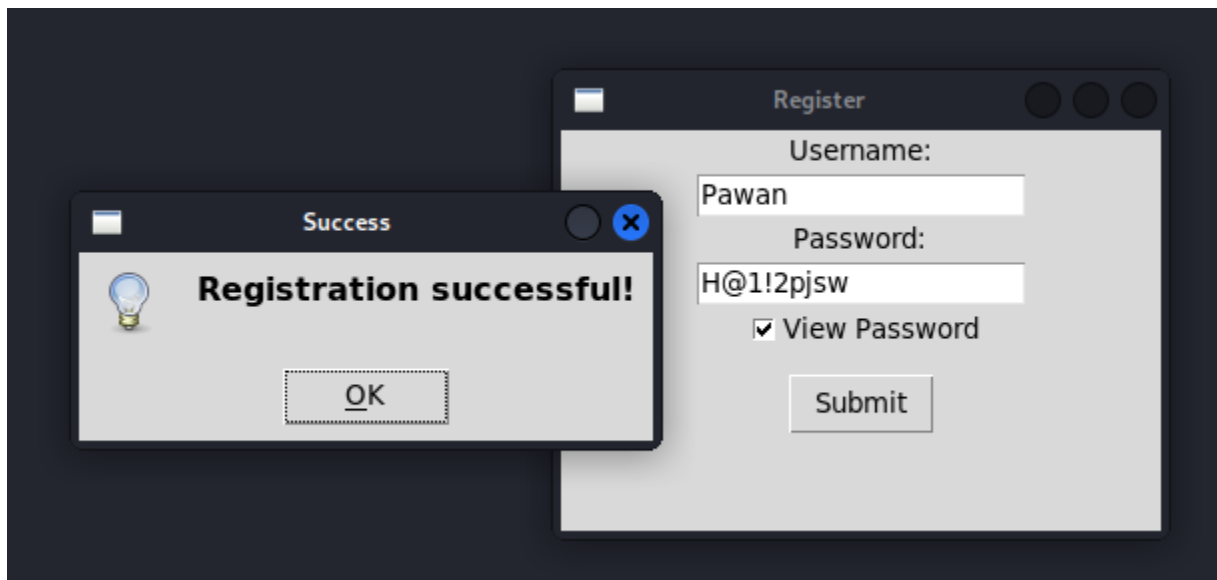


Figure 58: Register New User.

4.2.1.4 Error Handling on Login Button (1)

Objective	To test the “Login” button to show the error if the username and password are empty.
Action	Click the “Login” button.
Expected Results	The button should be able to show an error message if the username and the password are empty.
Actual Results	The button was able to show an error.
Conclusion	The test was successful.

Table 5: Error Handling on Login Button (1).

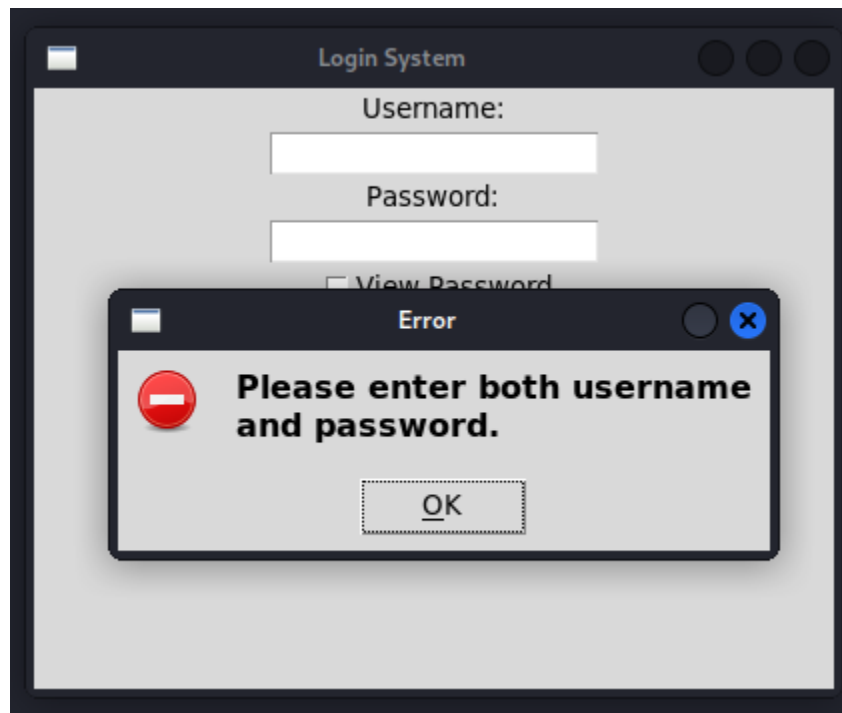


Figure 59: Error Handling on Login Button (1).

4.2.1.5 Error Handling on Login Button (2)

Objective	To test the “Login” button to show the error if the password is incorrect.
Action	Enter the username and incorrect password and then click the “Login” button.
Expected Results	The button should be able to show an error message if the password is incorrect.
Actual Results	The button was able to show an error.
Conclusion	The test was successful.

Table 6: Error Handling on Login Button (2)

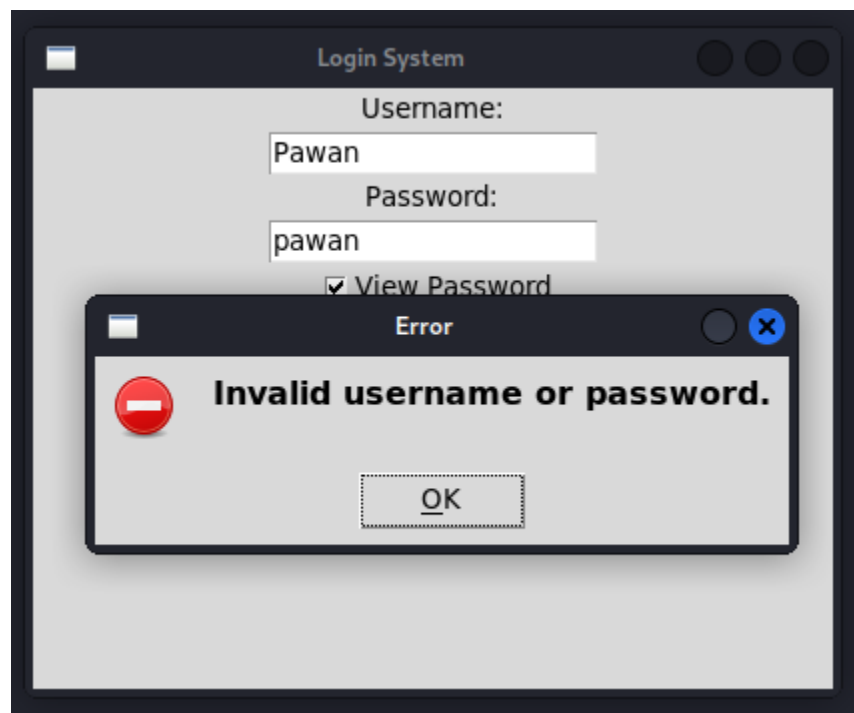


Figure 60: Error Handling on Login Button (2).

4.2.1.6 Login button

Objective	To test the “Login” button to access the system.
Action	Enter the user credentials and click the “Login” button.
Expected Results	The user should be able to access the system dashboard.
Actual Results	The user was able to access the system dashboard.
Conclusion	The test was successful.

Table 7: Login Button Testing Table.

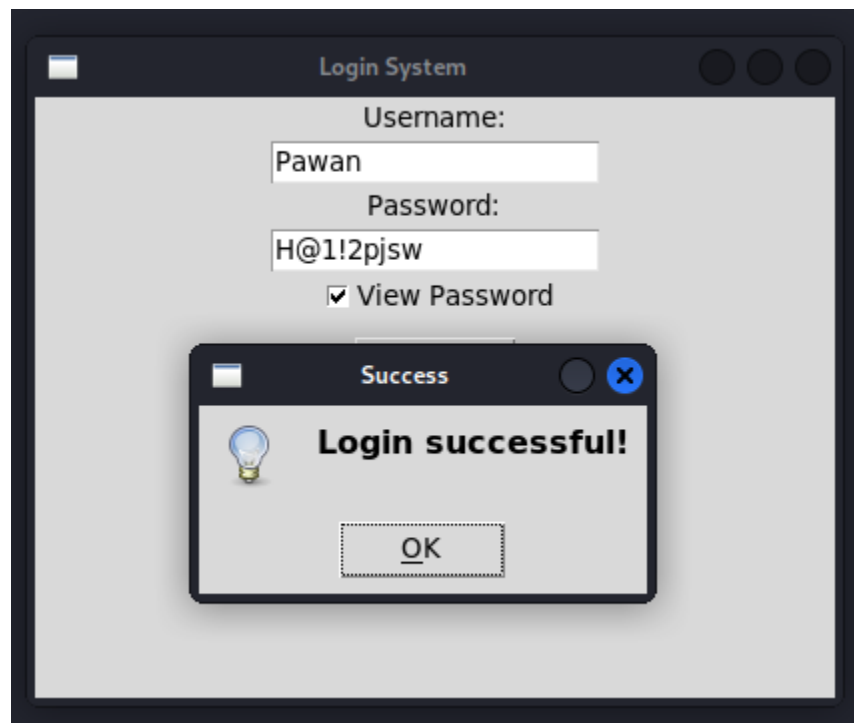


Figure 61: Login Successful.

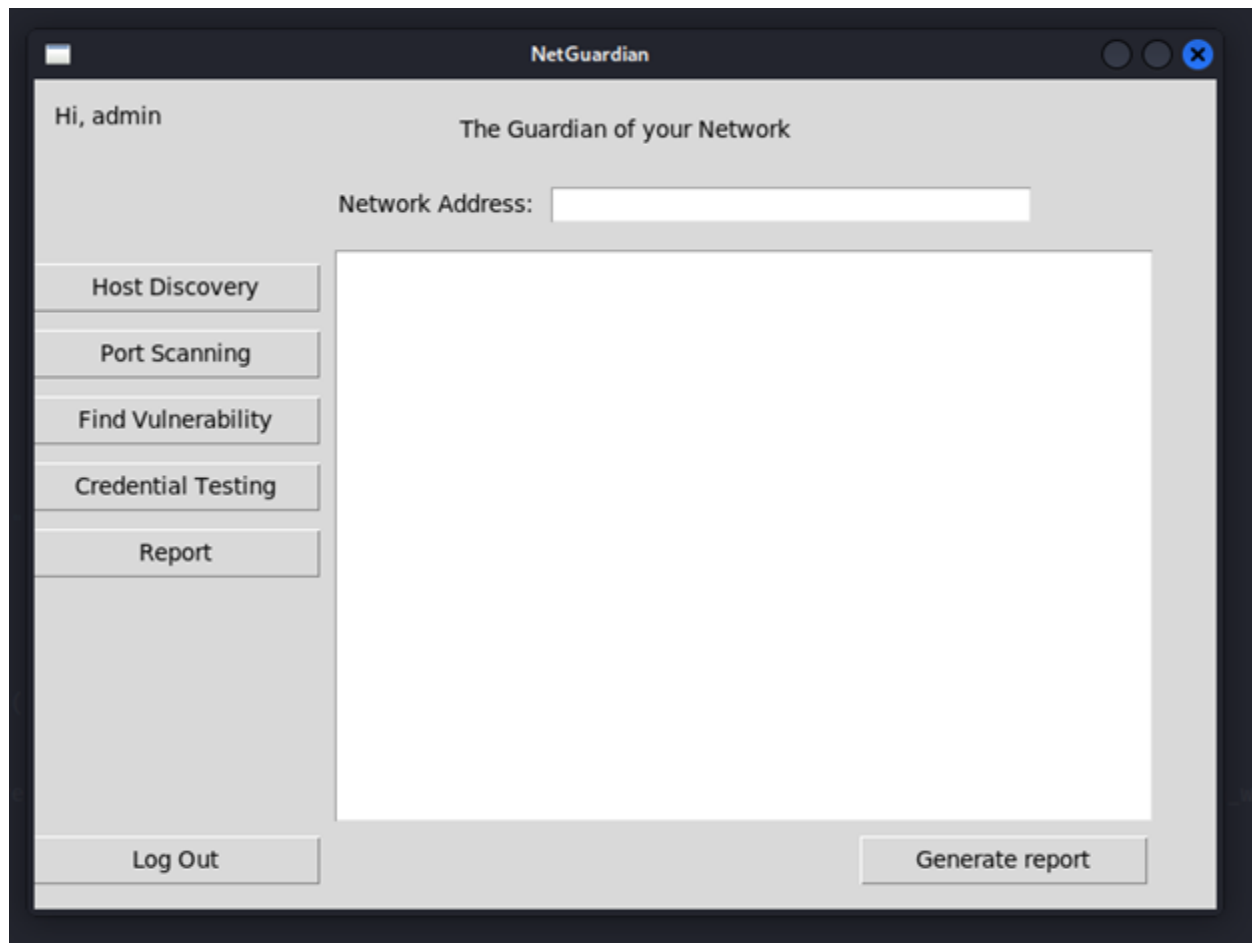


Figure 62: Redirect to User Dashboard.

4.2.1.7 Host Discovery Button

Objective	To test the “Host Discovery” button to discover active hosts on the network.
Action	Enter the network address and click the “Host Discovery” button.
Expected Results	The button should be able to discover active hosts on the network and display the output on the output box.
Actual Results	The button was able to discover active hosts on the network and display the output on the output box.
Conclusion	The test was successful.

Table 8: Host Discovery Button Testing Table.

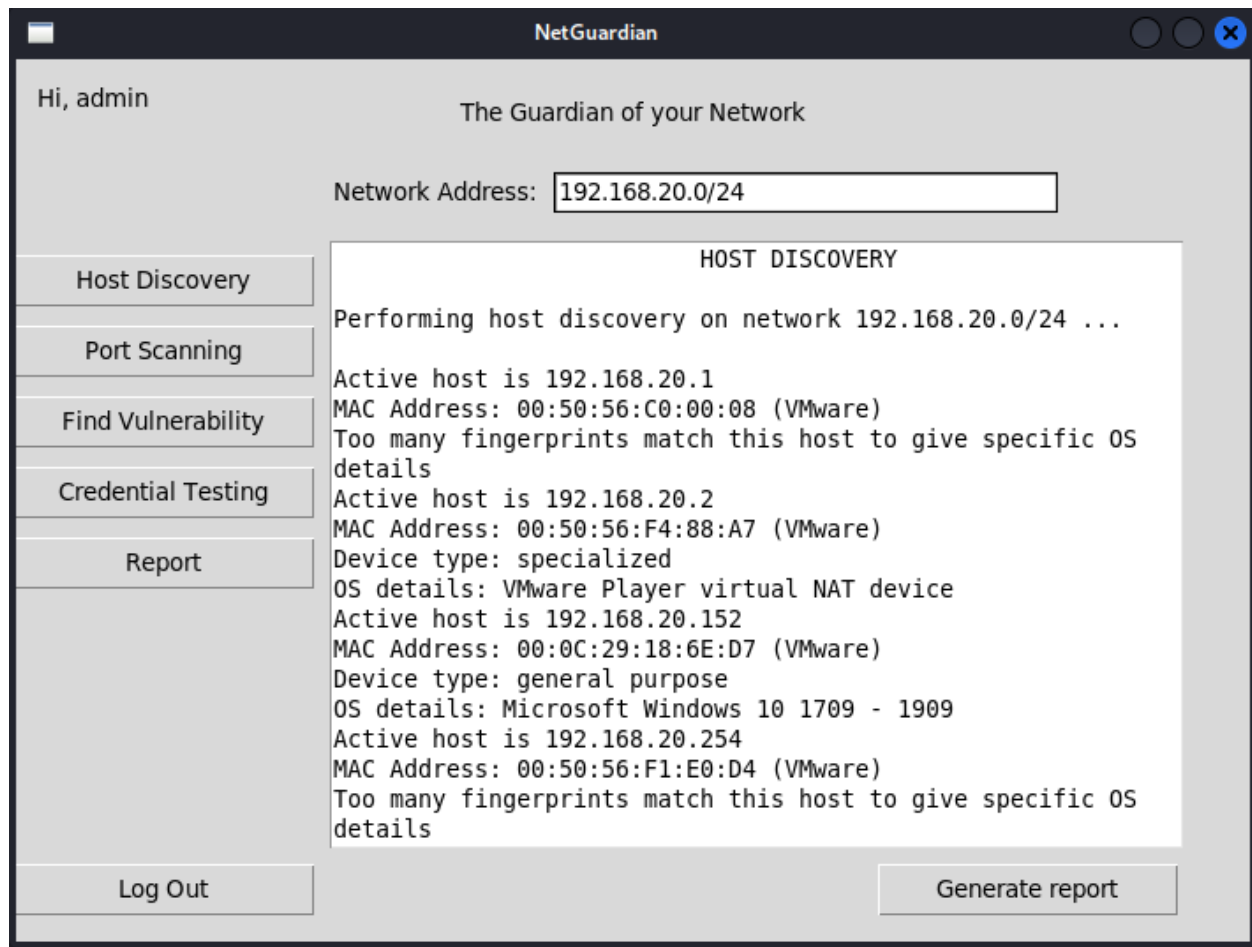


Figure 63: Host Discovery Button Output.

4.2.2 Evolution 2

4.2.2.1 Port Scanning Button

Objective	To test the “Port Scanning” button to discover active hosts including open ports and services and their version on the network.
Action	Enter the network address and click the “Port Scanning” button.
Expected Results	The button should be able to discover active hosts including open ports and services and their versions on the network and also, display the output on the output box.
Actual Results	The button was able to discover active hosts including open ports and services and their versions on the network and also, display the output on the output box.
Conclusion	The test was successful.

Table 9: Port Scanning Button Testing Table.

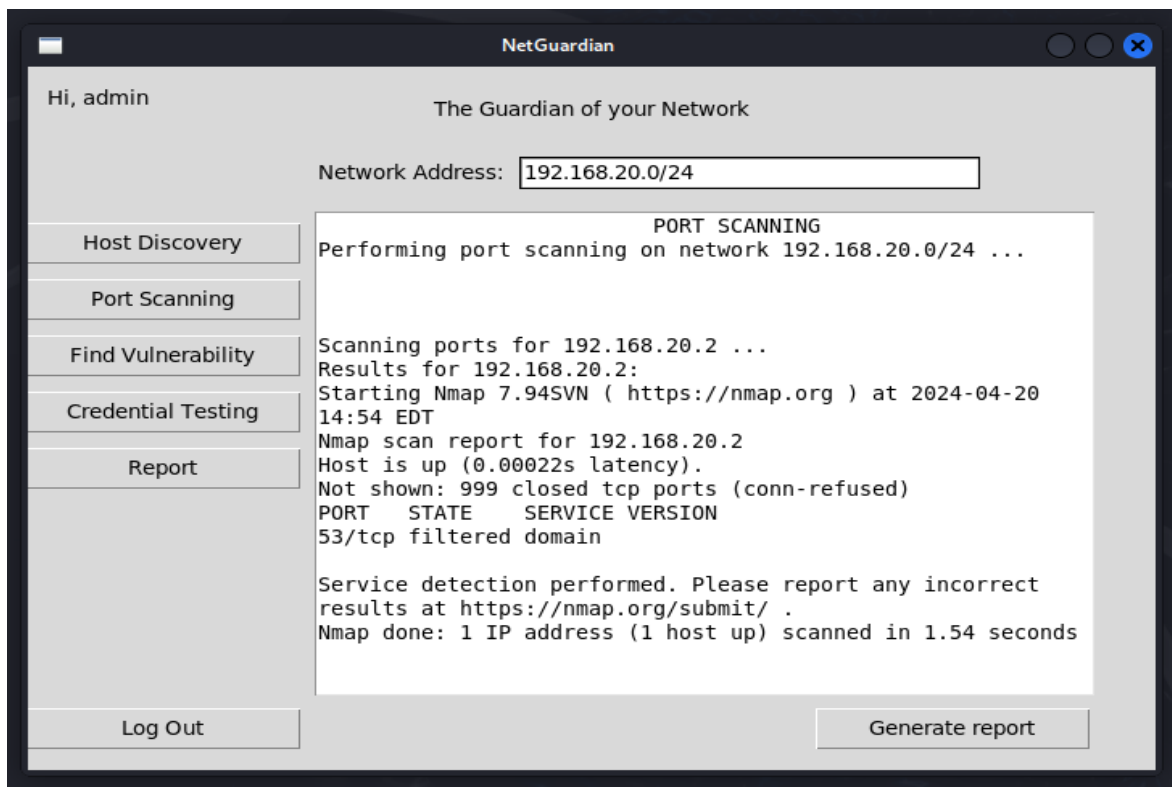


Figure 64: Port Scanning Button Output.

4.2.2.2 Find Vulnerability

Objective	To test the “Find Vulnerability” button to discover specific vulnerabilities related to service and version of open ports of the active hosts on the network.
Action	Enter the network address and click the “Find Vulnerability” button.
Expected Results	The button should be able to discover specific vulnerabilities related to the service and version of open ports of the active hosts on the network and also, display the output on the output box.
Actual Results	The button was able to discover specific vulnerabilities related to the service and version of open ports of the active hosts on the network and also, display the output on the output box.
Conclusion	The test was successful.

Table 10: Find the Vulnerability Button Testing table.

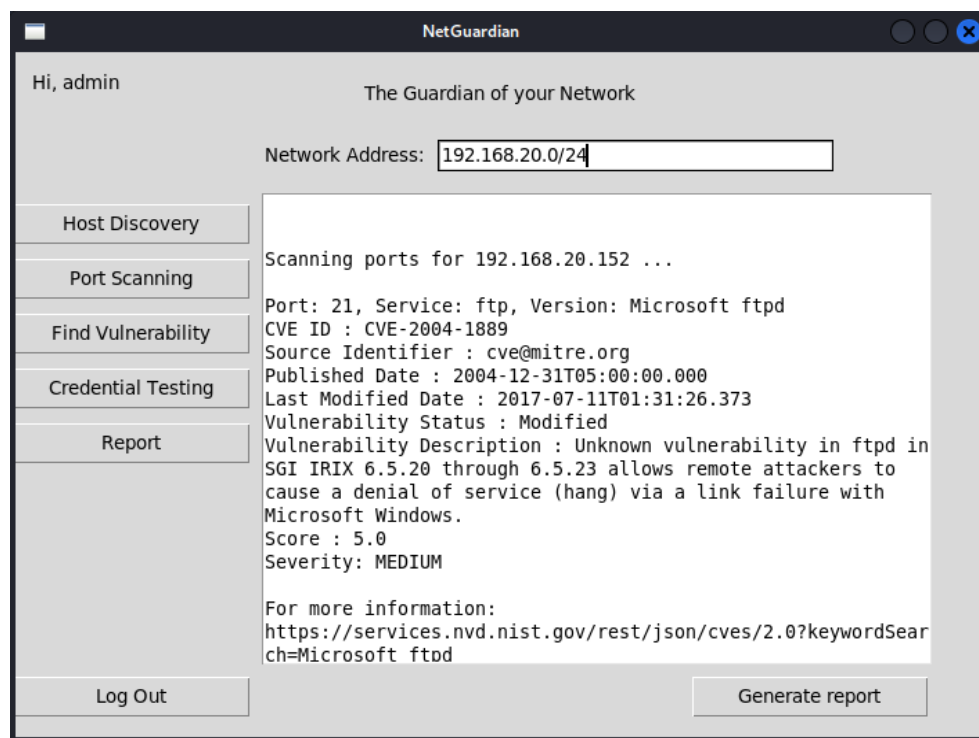


Figure 65: Find Vulnerability Button Output.

4.2.2.3 Generate Report Button

Objective	To test the “Generate Report” button to generate the report from output in pdf format.
Action	After performing the task, click the “Generate Report” button and name the PDF file, and then save the file.
Expected Results	The button should be able to generate a report from output and name it and save the file in the reports folder.
Actual Results	The button was able to generate a report from the output and name it and save the file in the reports folder.
Conclusion	The test was successful.

Table 11: Generate Report Button Testing Table.

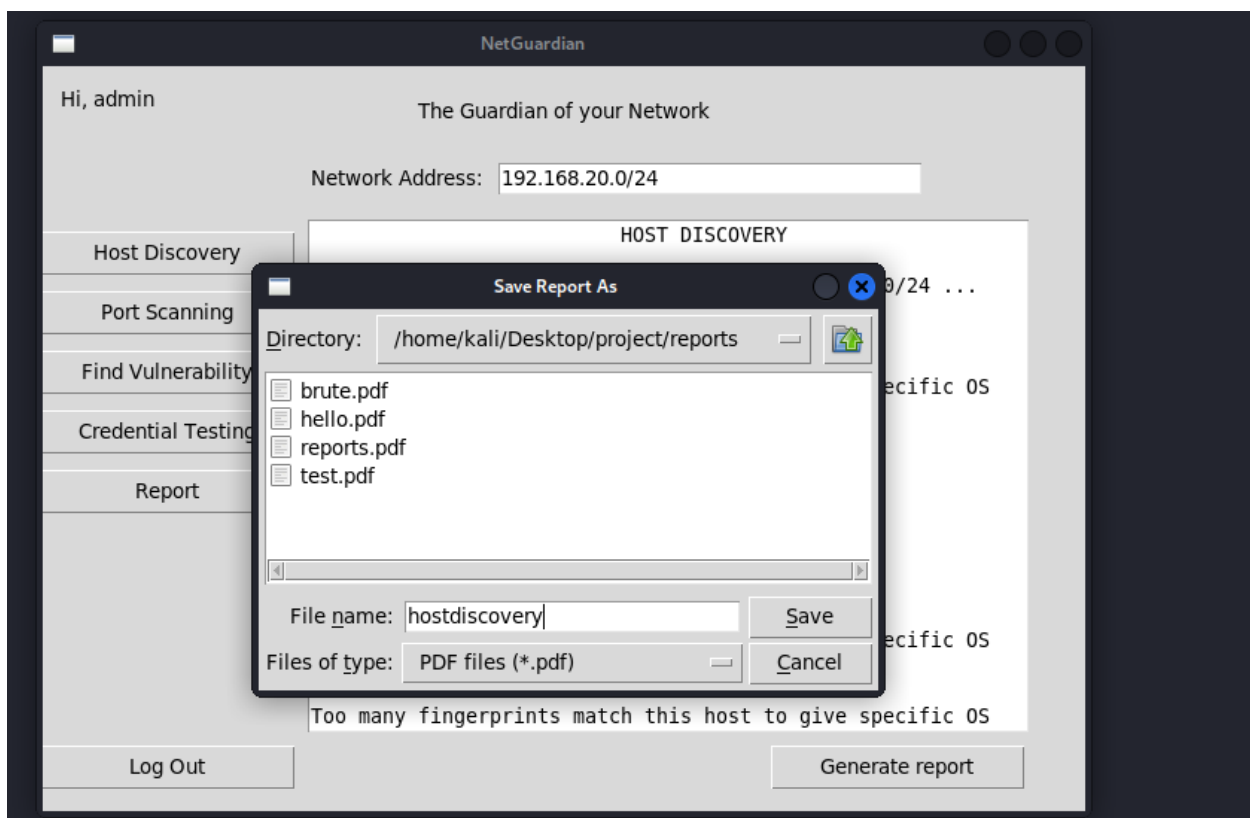


Figure 66: Naming File as "hostdiscovery".

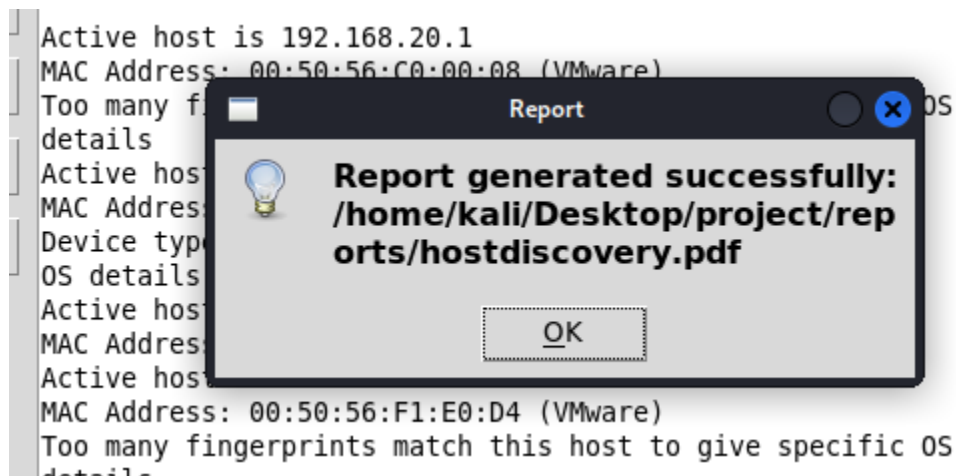


Figure 67: Report Generated Successfully.

4.2.2.4 Report Button

Objective	To test the “Report” button to list all the pdf files in the report folder.
Action	Click the “Report” button and select the pdf file from the list and then click on the “Open” button.
Expected Results	The button should be able to list and open the selected PDF file.
Actual Results	The button was able to list and open the selected PDF file.
Conclusion	The test was successful.

Table 12: Report Button Testing Table.

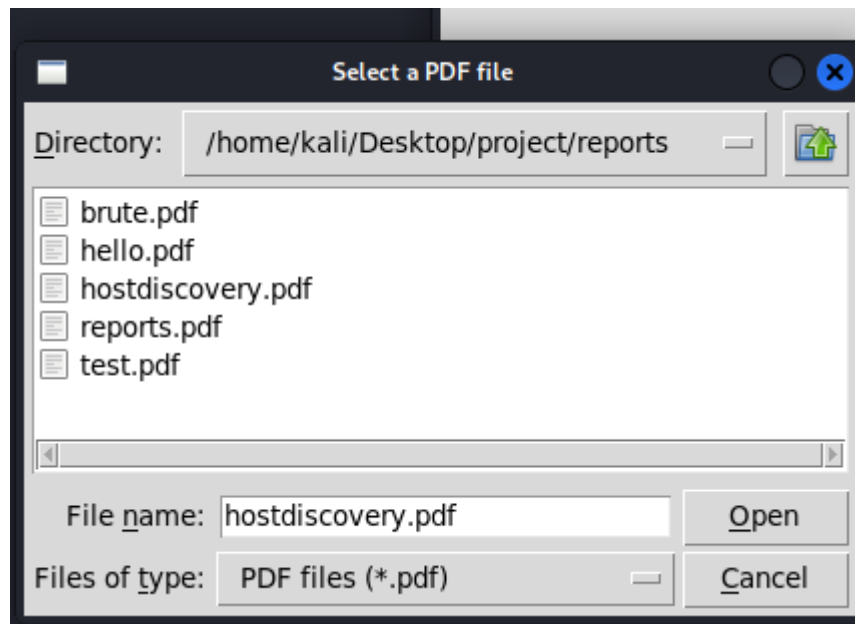


Figure 68: Selecting the "hostdiscovery" File.

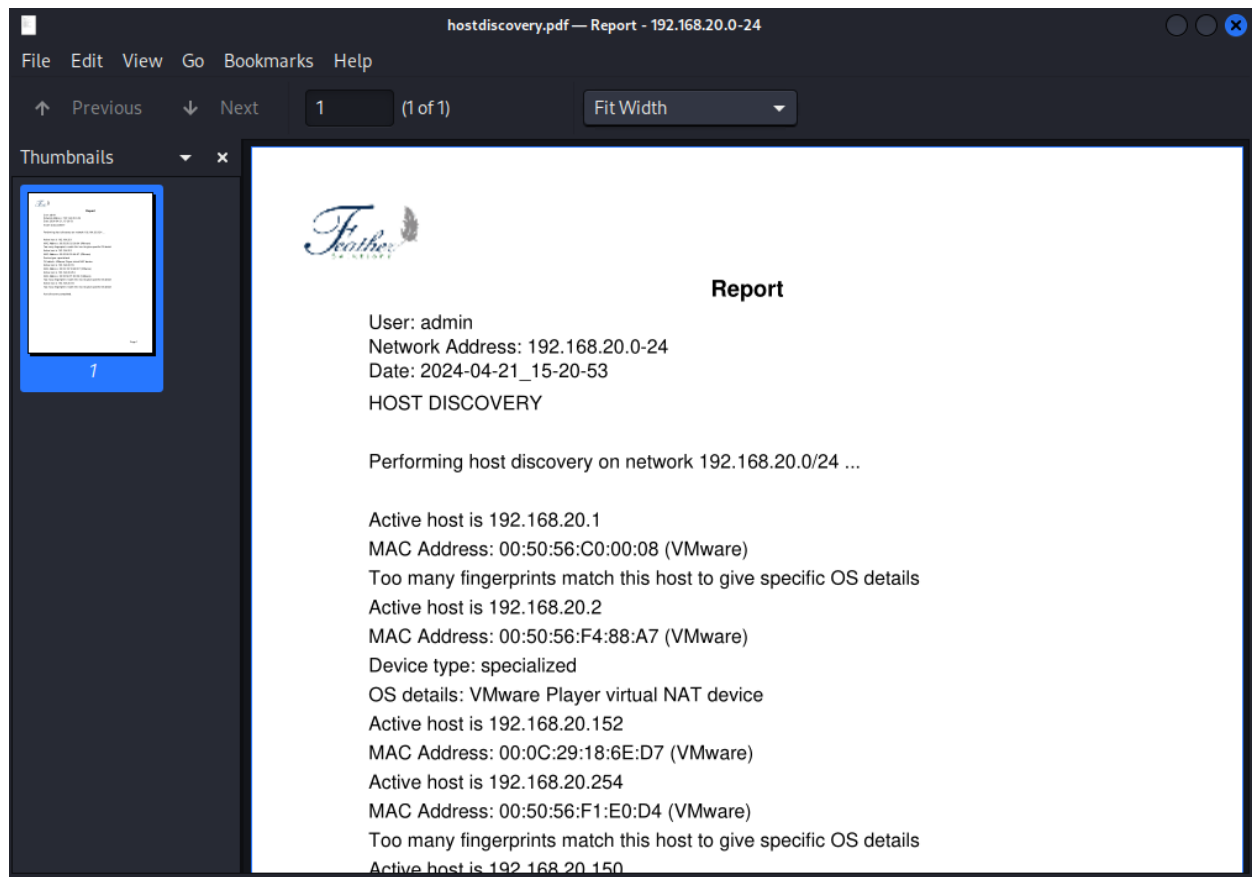


Figure 69: Open "hostdiscovery" File.

4.2.2.5 Error Handling on Generate Report Button

Objective	To test the Output Box.
Action	Click the “Generate” Button.
Expected Results	If the output box is empty then the error message should pop up.
Actual Results	If the output box was empty then the error message popped up.
Conclusion	The test was successful.

Table 13: Error Handling on the Generate Report Button.

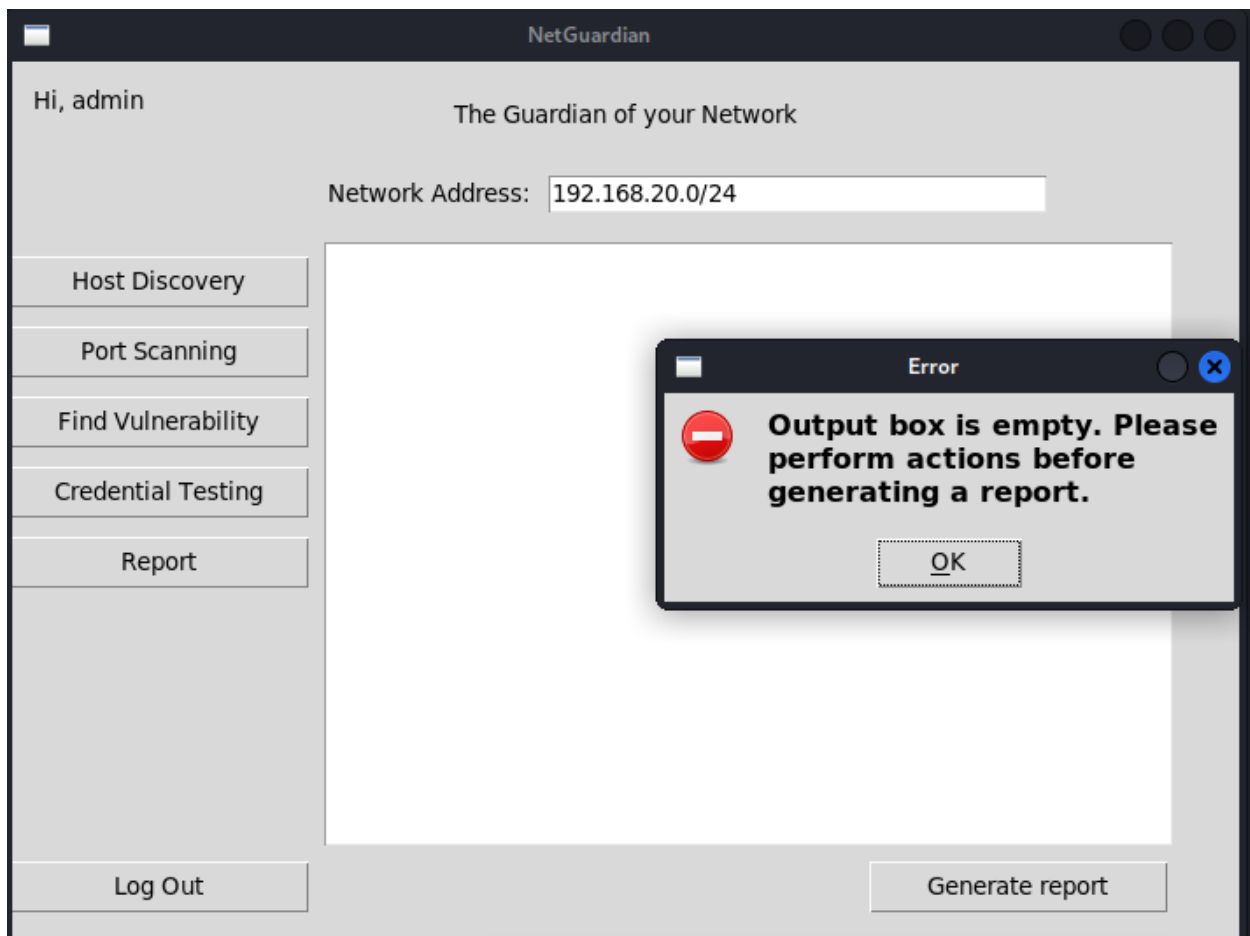


Figure 70: Error Handling on Output Box.

4.2.2.6 Log Out Button

Objective	To test the “Logout” button to log out the current user from the system.
Action	Click the “Log Out” Button.
Expected Results	The dashboard should be terminated and the “login System” window should be open.
Actual Results	The dashboard was terminated and the “login System” window should be open.
Conclusion	The test was successful.

Table 14: Logout Button Testing Table.

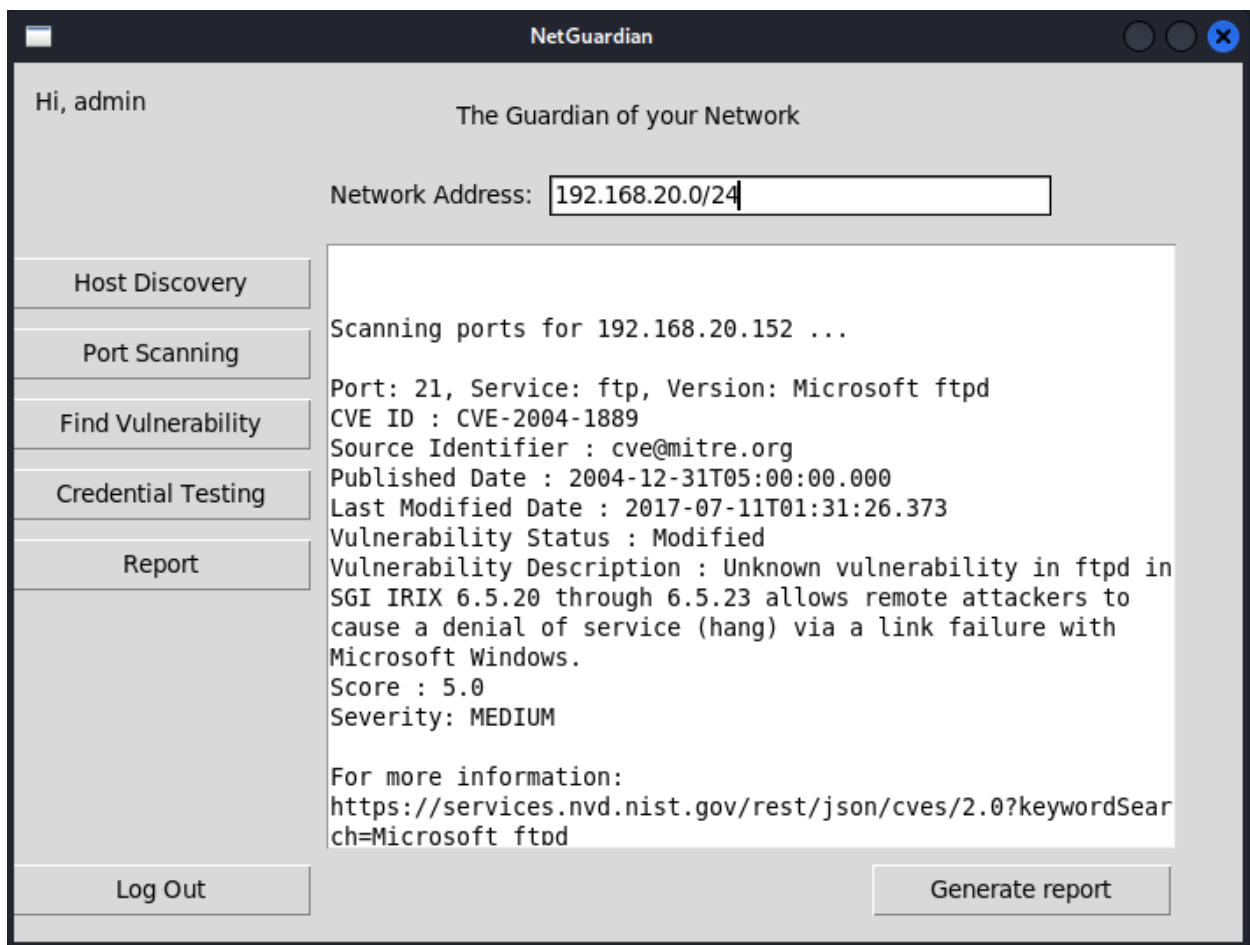


Figure 71: Redirecting to Login Window.

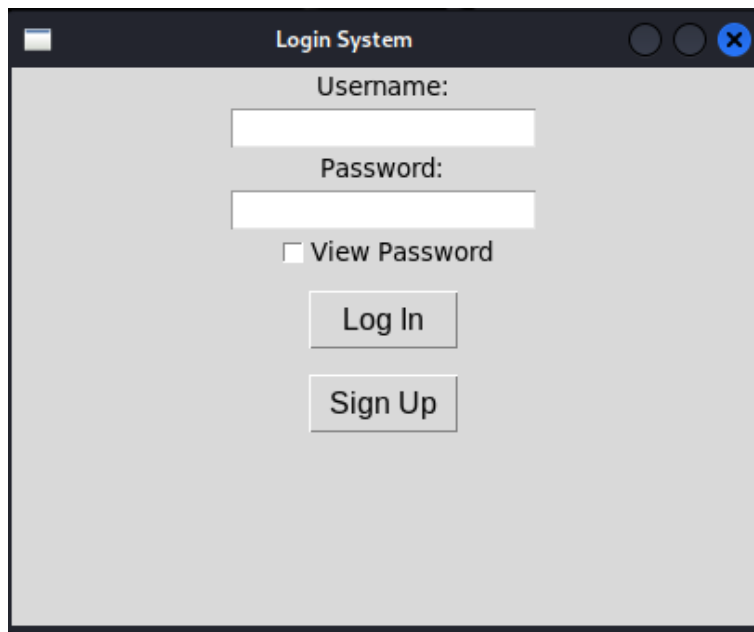


Figure 72: Successfully Redirect to Login Window.

4.2.3 Evolution 3

4.2.3.1 Credential Testing Button

Objective	To test the “Credential Testing” button to discover default or weak passwords in the active hosts.
Action	Enter the network address and click the “Credential Testing” button. The credential Testing window will be open and select the IP address and the port number. Then, click the “Brute force” button and the result will be displayed in the output box.
Expected Results	The button should be able to discover default or weak passwords in the selected IP address.
Actual Results	The button should be able to discover default or weak passwords in the selected IP address.
Conclusion	The test was successful.

Table 15: Credential Testing Button Testing Table.

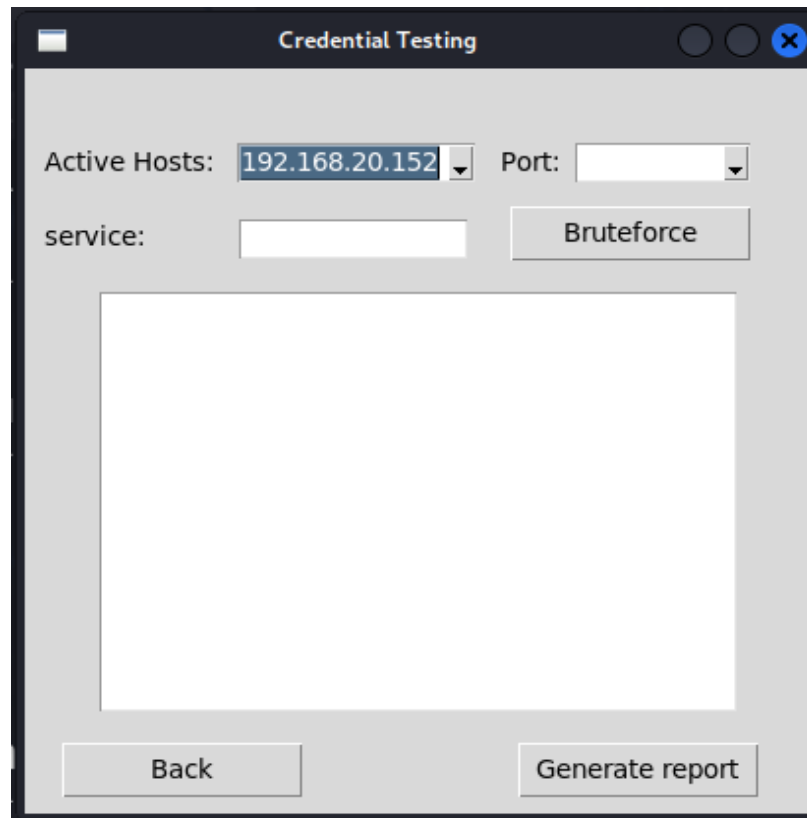


Figure 73: Selecting IP address.

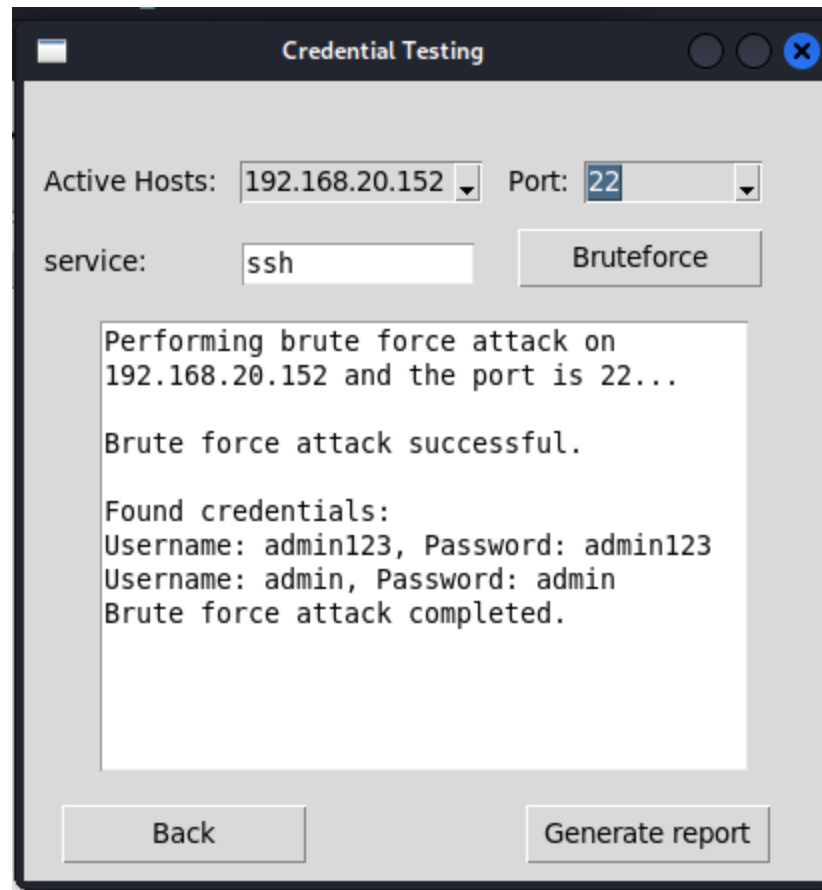


Figure 74: Successfully Brute force the Selected IP Address.

4.2.3.2 Generate Report Button of Credential Testing Window

Objective	To test the “Generate Report” button to generate the report from output in pdf format.
Action	After performing the task, click the “Generate Report” button and name the PDF file, and then save the file.
Expected Results	The button should be able to generate a report from output and name and save the file in the reports folder.
Actual Results	The button was able to generate a report from output and name and save the file in the reports folder.
Conclusion	The test was successful.

Table 16: Generate report Button Testing Table.

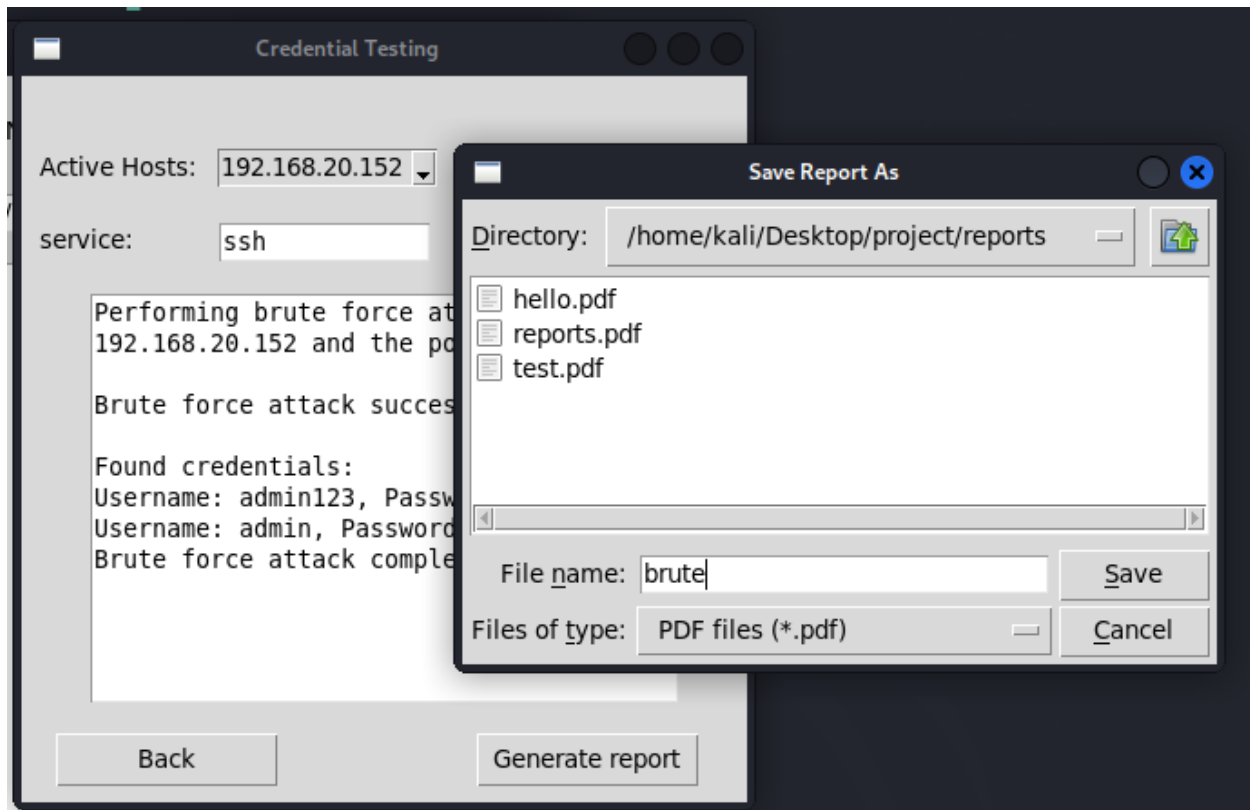


Figure 75: Naming file as "brute".

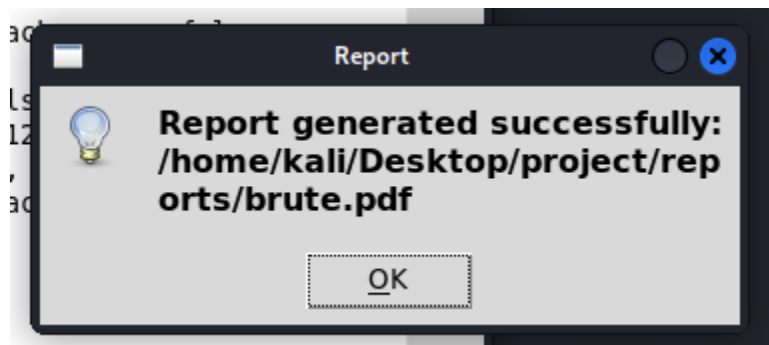


Figure 76: Successfully Generated Report.

4.2.3.3 Back Button

Objective	To test the “Back” button to redirect the previous window i.e dashboard.
Action	Click the “Back” Button.
Expected Results	The user should be redirected to the system dashboard.
Actual Results	The user was redirected to the system dashboard.
Conclusion	The test was successful.

Table 17: Back Button Testing Table.

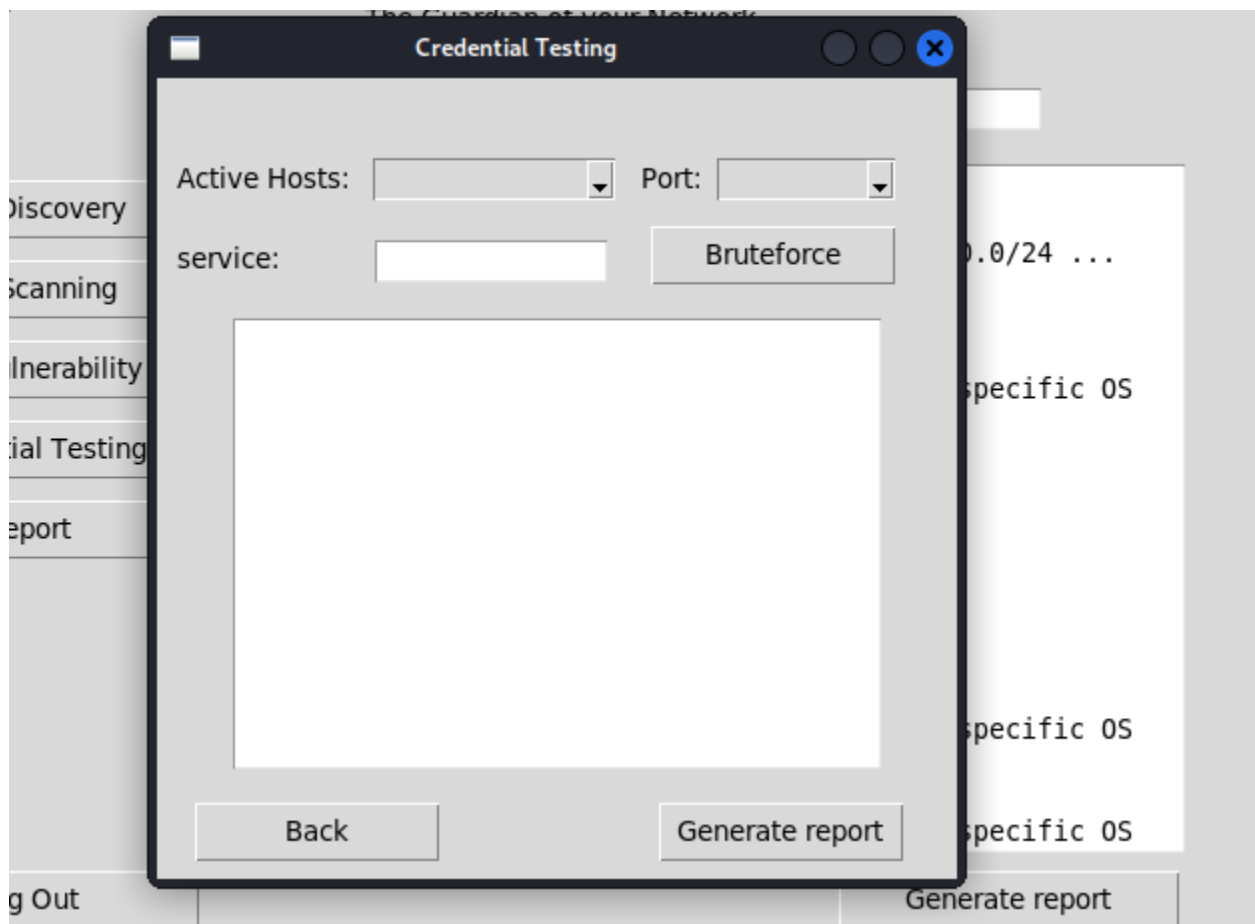


Figure 77: Redirecting to the Dashboard.

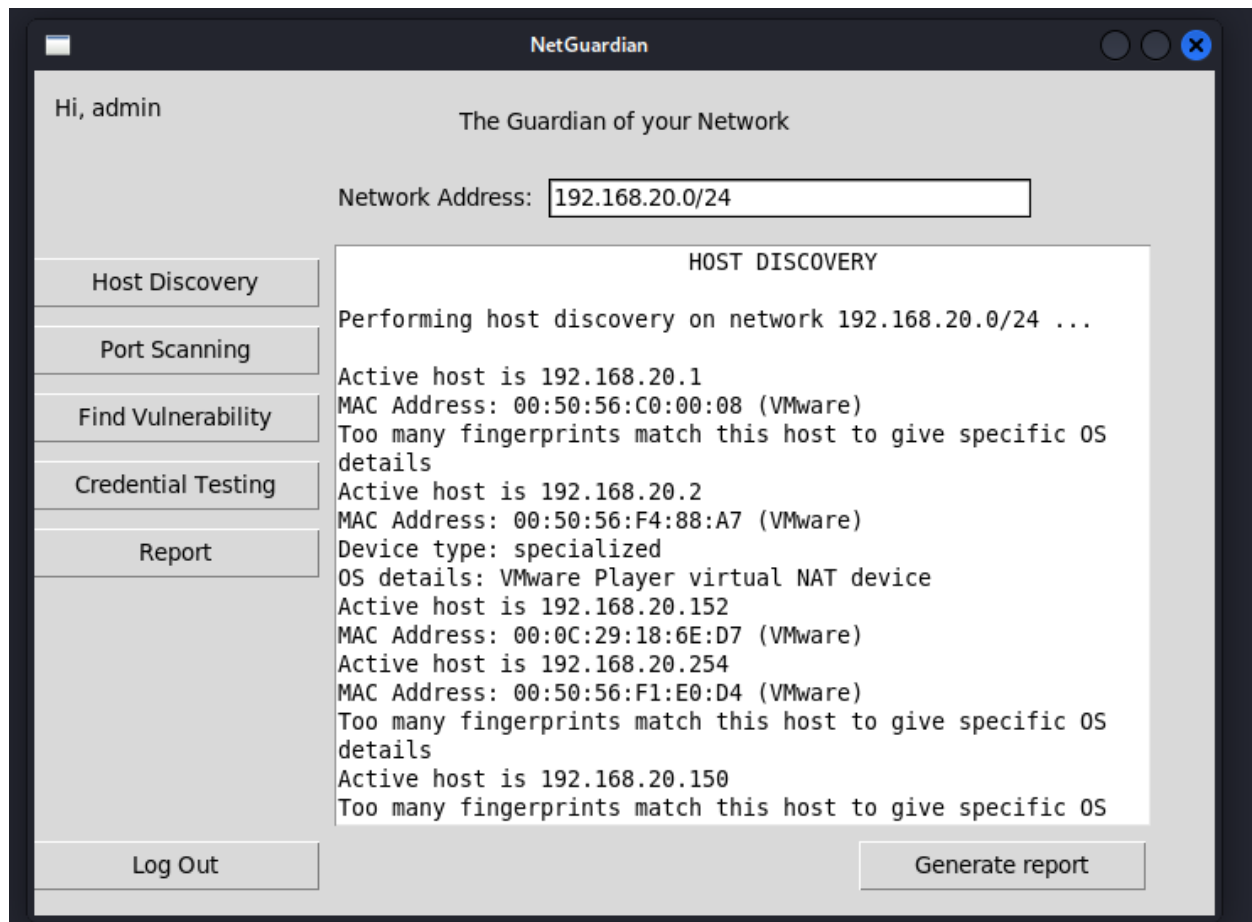


Figure 78: Successfully Redirect to the Dashboard.

4.2.3.4 Output Box

Objective	To test the result displayed in the output box should not be modified by the user.
Action	Click on the output box and try to modify the result.
Expected Results	The user should not be able to modify the results.
Actual Results	The user was not able to modify the results.
Conclusion	The test was successful.

Table 18: Output Box Testing Table.

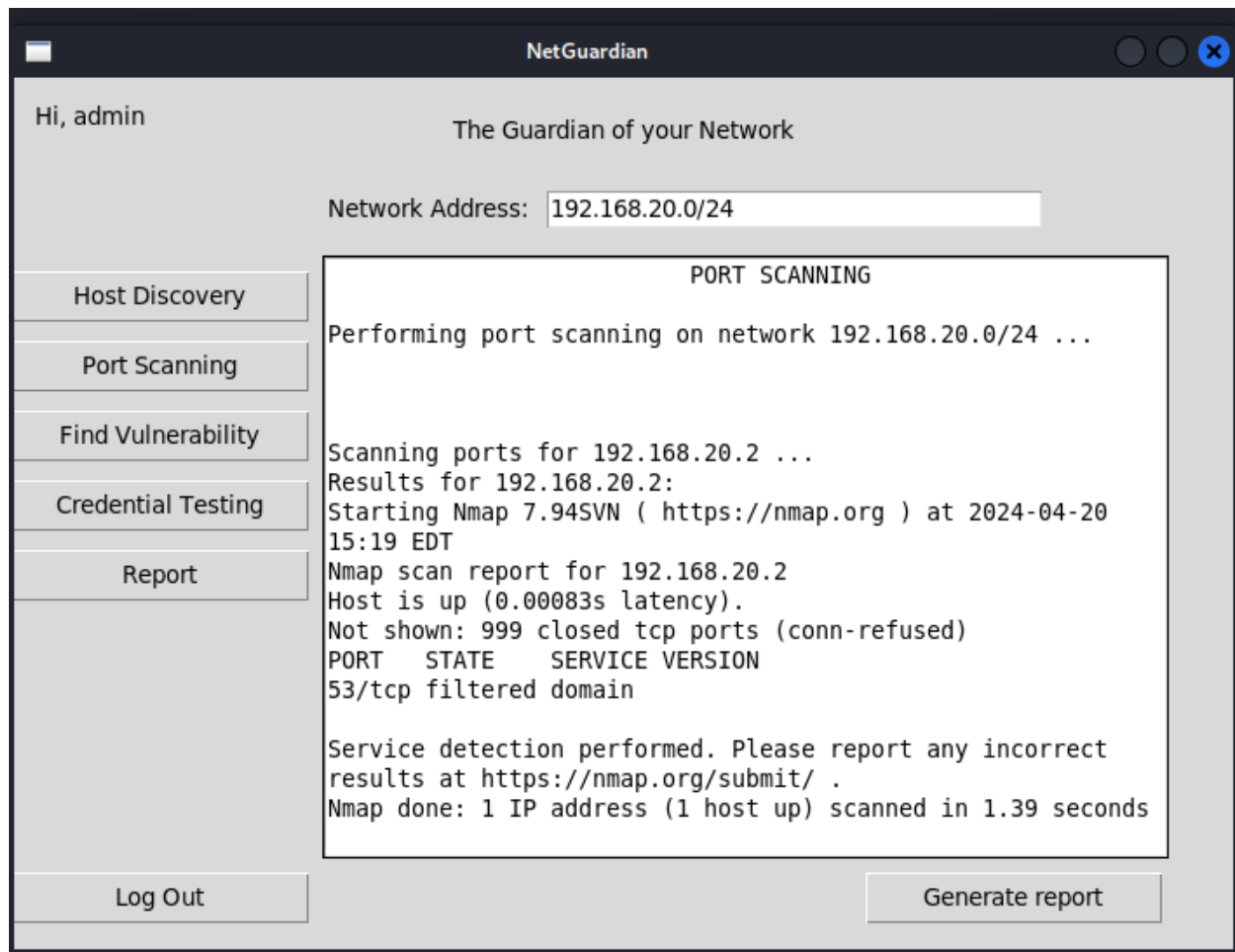


Figure 79: Trying to Edit the Output Box.

4.2.3.5 Error Handling for Network Address Entry

Objective	To test the Network Address Entry.
Action	Keep the network address entry empty and click any button.
Expected Results	The system should be able to pop up an error message if the network address entry is empty.
Actual Results	The system was able to pop up an error message.
Conclusion	The test was successful.

Table 19: Network Address Entry Testing Table.

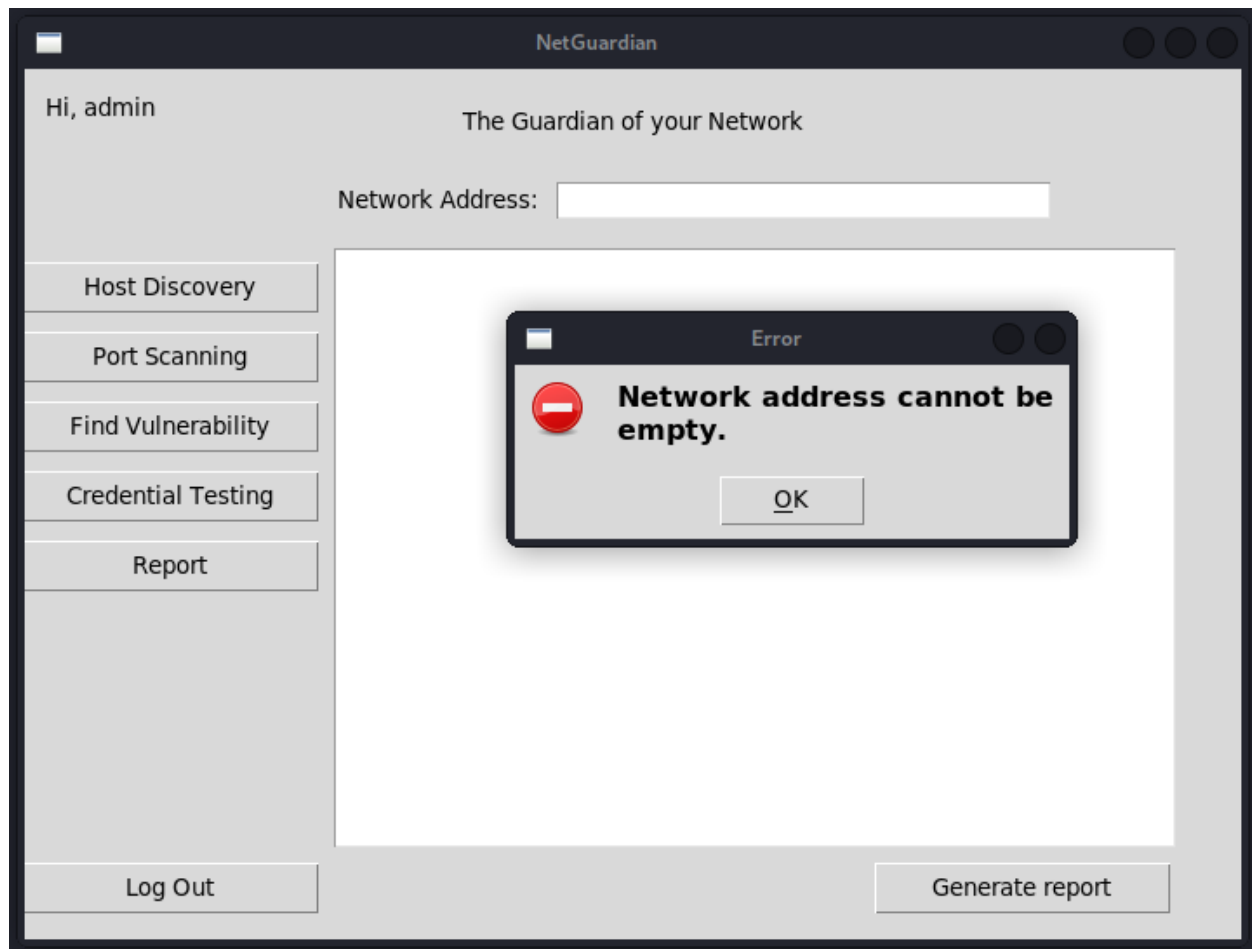


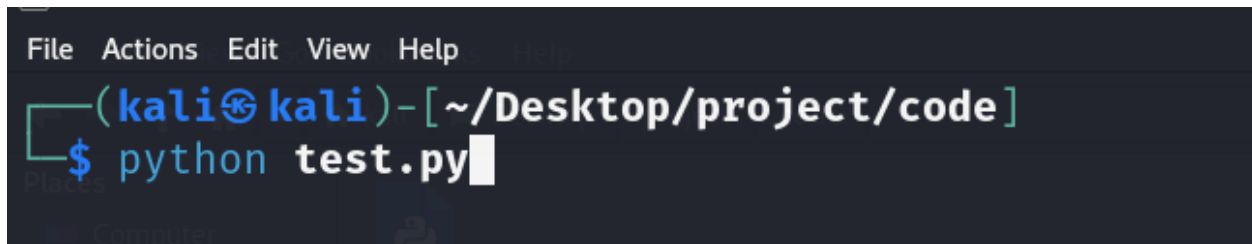
Figure 80: Error Handling on Network Address Entry.

4.3 System Testing

4.3.1 Run Application

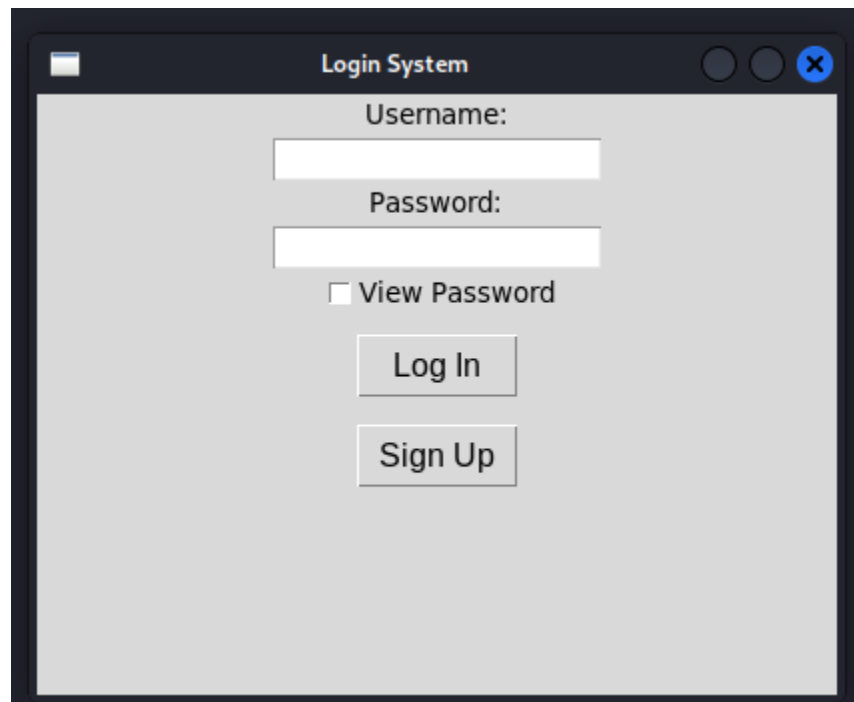
Objective	To run the application in the interpreter mode of the terminal.
Action	Open a terminal and run the Python script.
Expected Results	The application should run and be able to open the Login window.
Actual Results	The application was running and was able to open the Login window.
Conclusion	The test was successful.

Table 20: Run Application.



```
File Actions Edit View Help
(kali㉿kali)-[~/Desktop/project/code]
$ python test.py
```

Figure 81: Running Python File.



A window titled "Login System" with a standard macOS-style title bar (red, yellow, and green buttons). The window has a light gray background. It contains the following elements:

- Username:** A text label followed by a white rectangular input field.
- Password:** A text label followed by a white rectangular input field.
- View Password:** A checkbox with the label "View Password".
- Log In:** A rectangular button with the text "Log In".
- Sign Up:** A rectangular button with the text "Sign Up".

Figure 82: Login System Open.

4.3.2 Sign Up

Objective	To test the “Sign Up” button to register a new user and store user credentials on the database.
Action	Click the “Sign Up” button and the Register window will open then enter username and password and click the “Submit” button.
Expected Results	The new user should be registered and stored on the database and a “Registration successful” message should pop up.
Actual Results	The new user was registered and stored on the database and a “Registration successful” message should pop up.
Conclusion	The test was successful.

Table 21: Sing Up feature Testing.

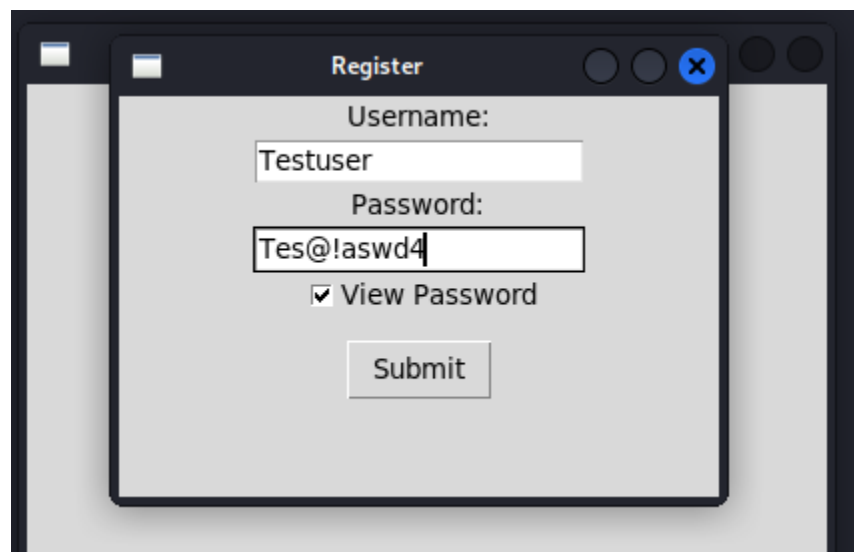


Figure 83: Registering User.

	username	password
	Filter	Filter
1		
2	pawan	pawan
3	admin	admin
4	lost	12345
5	samrat	1212
6	sahil	S@hil1!11
7	abc	Abc@123#
8	Pawan	H@1!2pjsw
9	Testuser	Tes@!aswd4

Figure 84: Store User Credential Testing on Database.

4.3.3 Login

Objective	To test the “Login” button to access the system.
Action	Enter the user credentials and click the “Login” button.
Expected Results	The user should be able to access the system after matching the user credentials on the database.
Actual Results	The user was able to access the system dashboard after matching the user credentials on the database.
Conclusion	The test was successful.

Table 22: Login Features Testing.

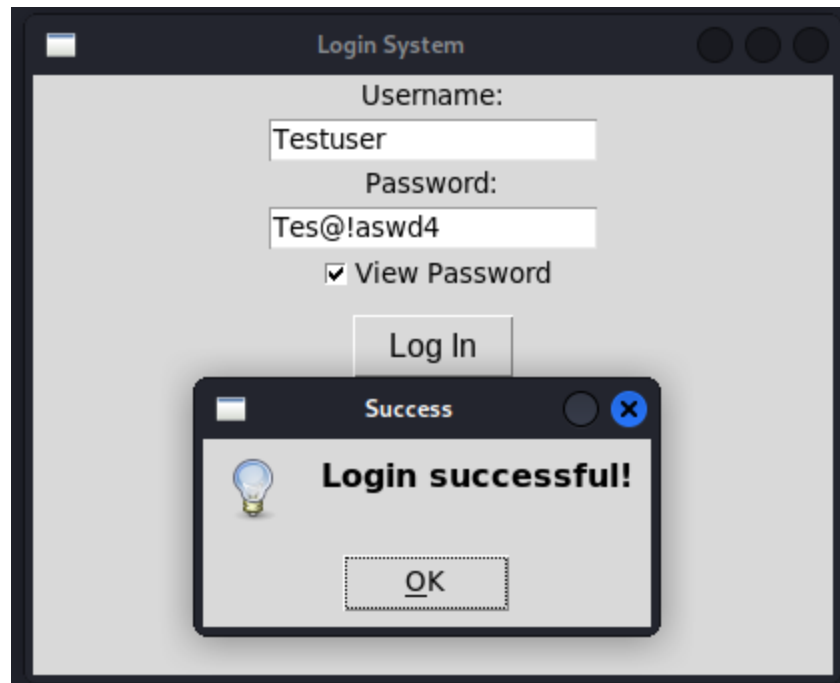


Figure 85: Login Successful.

4.3.4 Host Discovery

Objective	To test the “Host Discovery” button to discover active hosts along with their Mac address and OS details using Nmap.
Action	Enter the network address and click the “Host Discovery” button.
Expected Results	The button should be able to discover active hosts on the network along with their Mac address and OS details using Nmap and display the results on the output box.
Actual Results	The button was able to discover active hosts on the network along with their Mac address and OS details using Nmap and display the results on the output box.
Conclusion	The test was successful.

Table 23: Host Discovery Feature Testing.

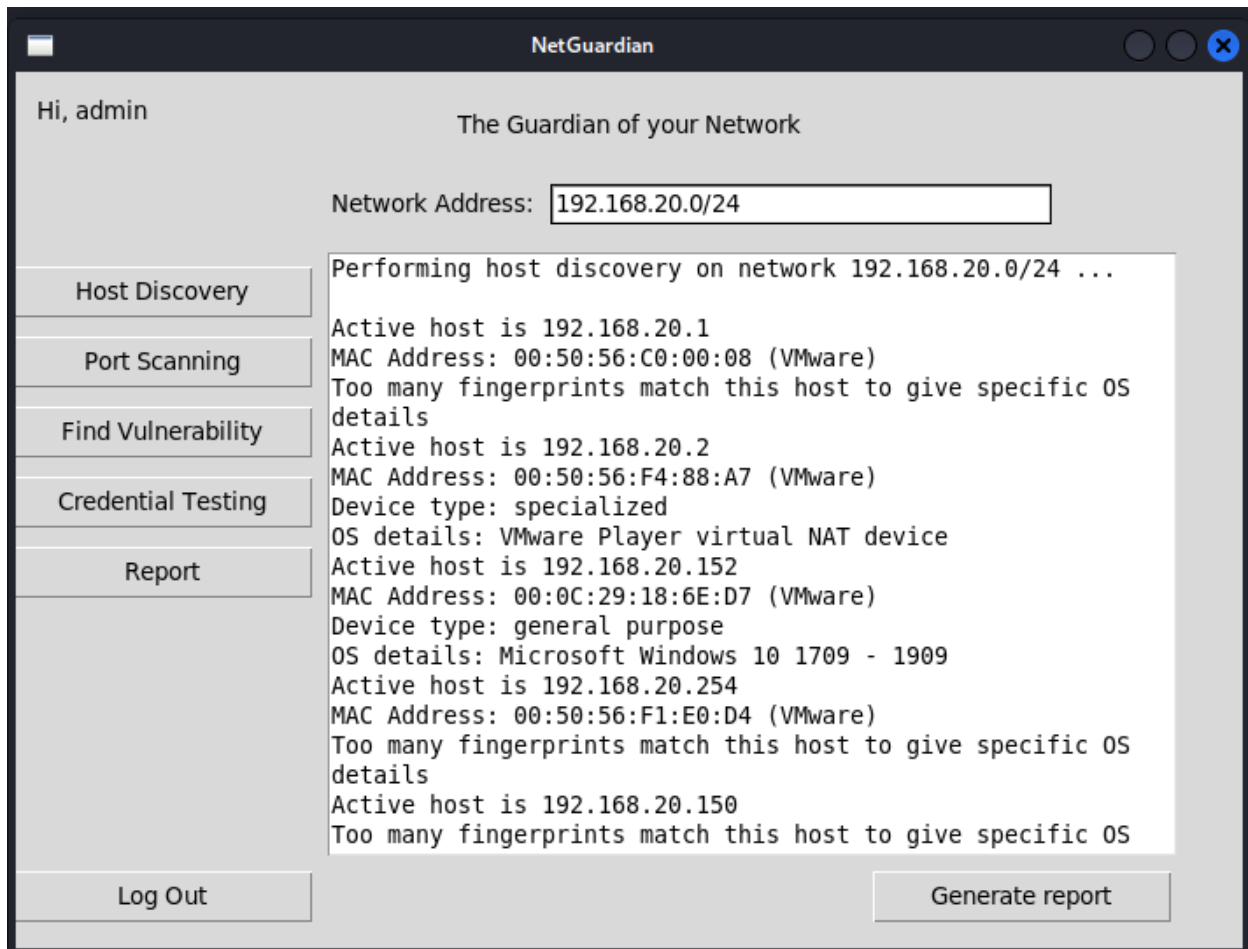


Figure 86: Result of Host Discovery.

4.3.5 Port Scanning

Objective	To test the “Port Scanning” button to discover active hosts including open ports and services and their version on the network using Nmap.
Action	Enter the network address and click the “Port Scanning” button.
Expected Results	The button should be able to discover active hosts including open ports and services and their versions on the network using Nmap and also, display the results on the output box.
Actual Results	The button was able to discover active hosts including open ports and services and their versions on the network using Nmap and also, display the results on the output box.
Conclusion	The test was successful.

Table 24: Port Scanning Feature Testing.

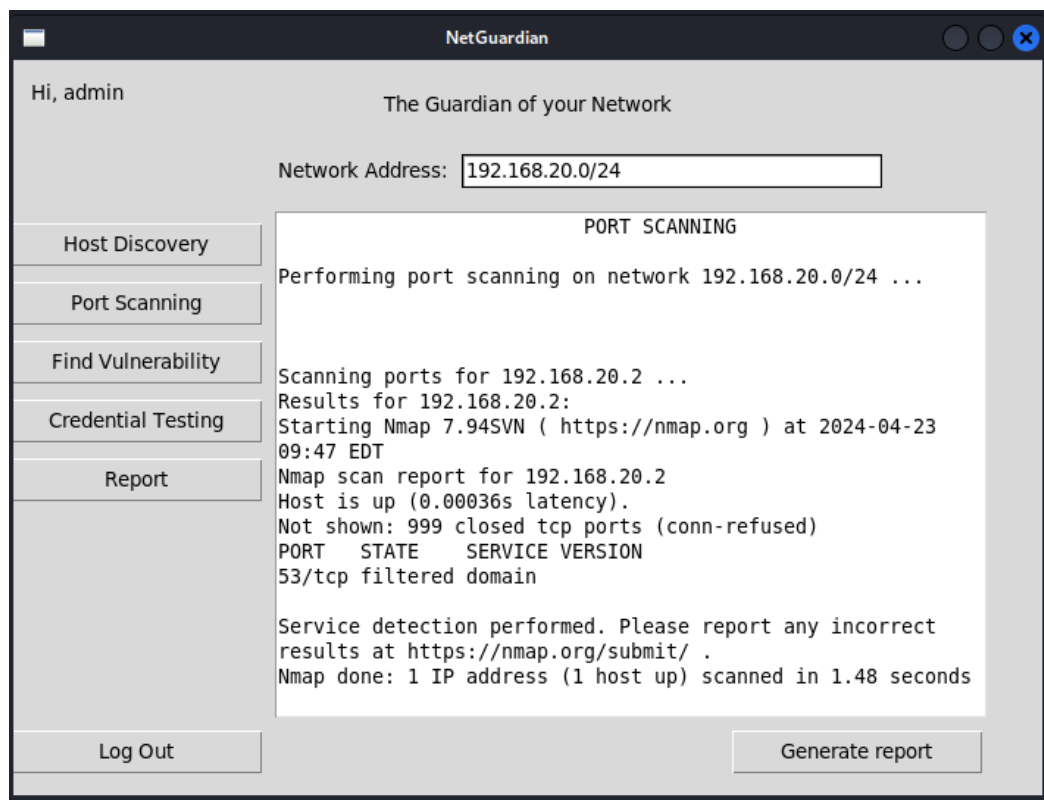


Figure 87: Results of Port Scanning.

4.3.6 Find Vulnerability

Objective	To test the “Find Vulnerability” button to discover the service and version of open ports of the active hosts on the network. Then request the National Vulnerability Database (NVD) using API for vulnerability on specific services and versions.
Action	Enter the network address and click the “Find Vulnerability” button.
Expected Results	The button should be able to discover specific vulnerabilities related to the service and version of open ports of the active hosts on the network and also, display the results on the output box.
Actual Results	The button was able to discover specific vulnerabilities related to the service and version of open ports of the active hosts on the network and also, display the results on the output box.
Conclusion	The test was successful.

Table 25: Find Vulnerability Feature Testing.

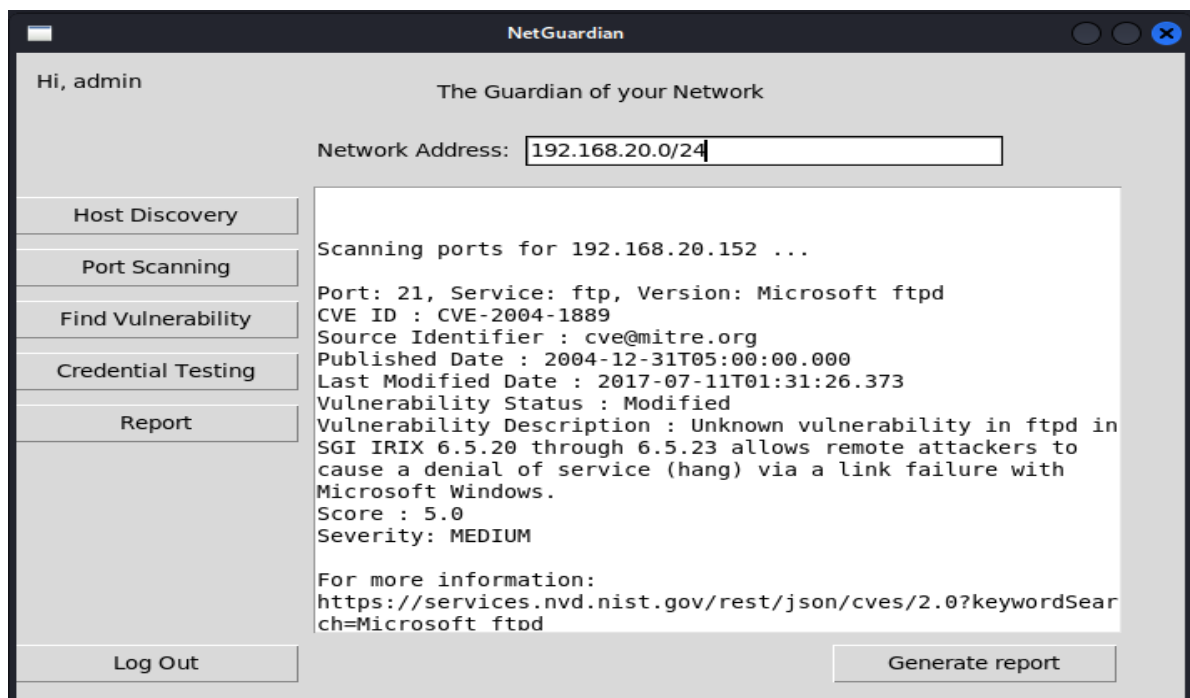


Figure 88: Results of Find Vulnerability.

4.3.7 Credential Testing

Objective	To test the “Credential Testing” button to discover default or weak passwords in the specific service on the active hosts by brute force using Hydra.
Action	Enter the network address and click the “Credential Testing” button. The credential Testing window will be open and select the IP address and the port number. Then, click the “Brute force” button and the result will be displayed in the output box.
Expected Results	The button should be able to discover default or weak passwords in the selected IP address.
Actual Results	The button should be able to discover default or weak passwords in the selected IP address.
Conclusion	The test was successful.

Table 26: Credential Testing Feature Testing.

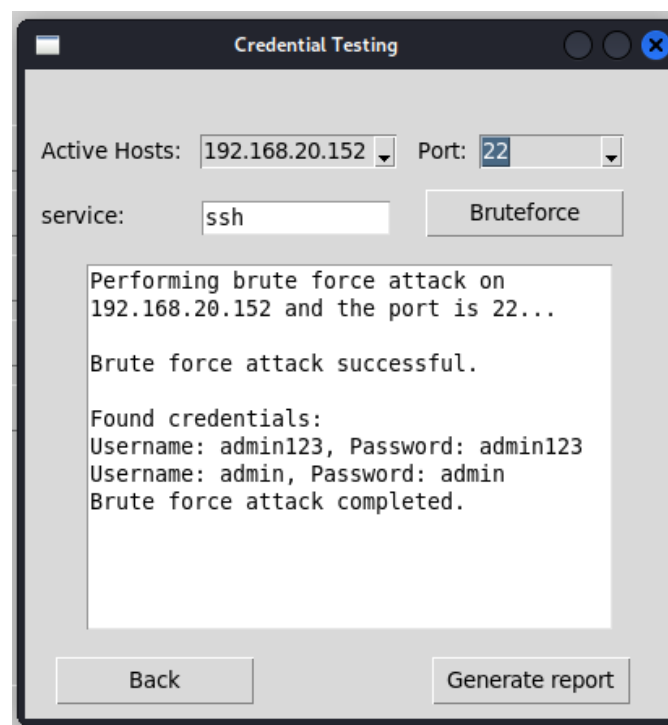


Figure 89: Output of Credential Testing.

4.3.8 Generate Report

Objective	To test the “Generate Report” button to generate the report from output in pdf format.
Action	After performing the task, click the “Generate Report” button and name the PDF file, and then save the file in the “report” folder.
Expected Results	The button should be able to generate a report from output and name it and save the file in the “reports” folder.
Actual Results	The button was able to generate a report from the output and name it and save the file in the “reports” folder.
Conclusion	The test was successful.

Table 27: Generate Report Feature Testing.

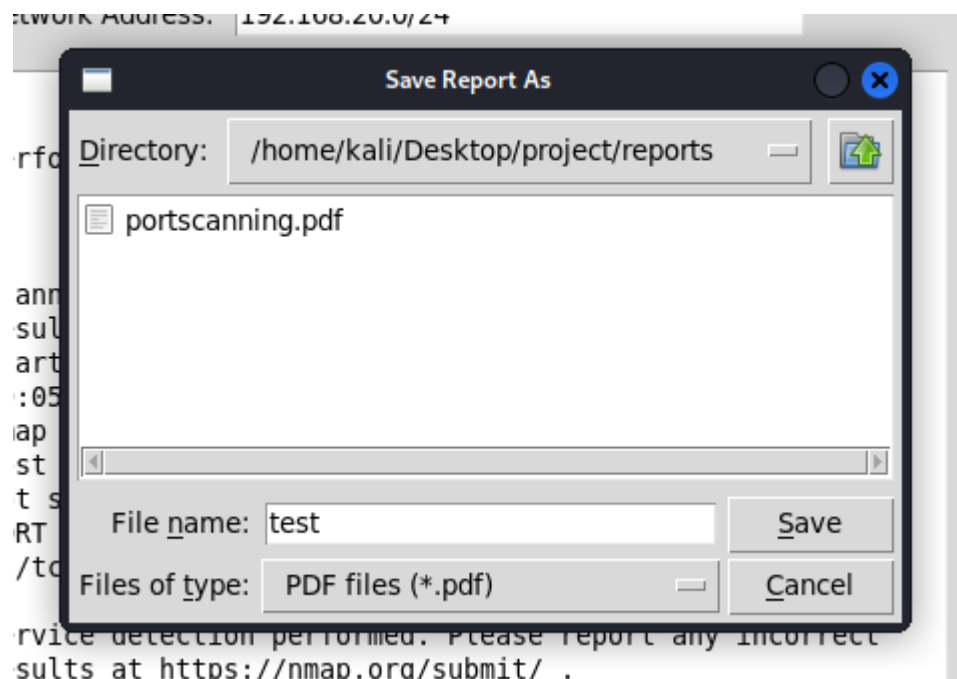


Figure 90: Naming File.

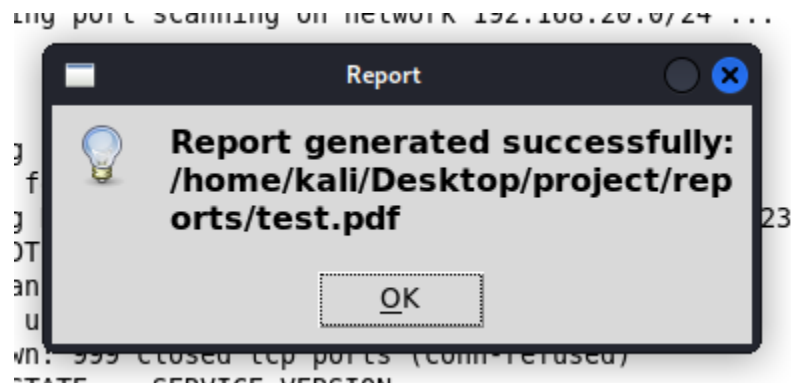


Figure 91: File Saved.

4.3.9 Report

Objective	To test the “Report” button to list all the pdf files in the report folder.
Action	Click the “Report” button and select the pdf file from the list and then click on the “Open” button.
Expected Results	The button should be able to open the selected PDF file.
Actual Results	The button was able to open the selected PDF file.
Conclusion	The test was successful.

Table 28: Report Button Testing.

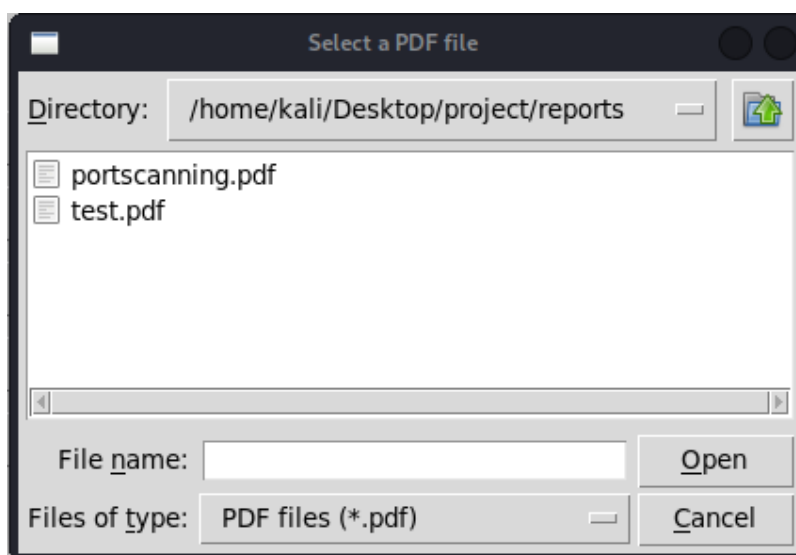


Figure 92: List the file.

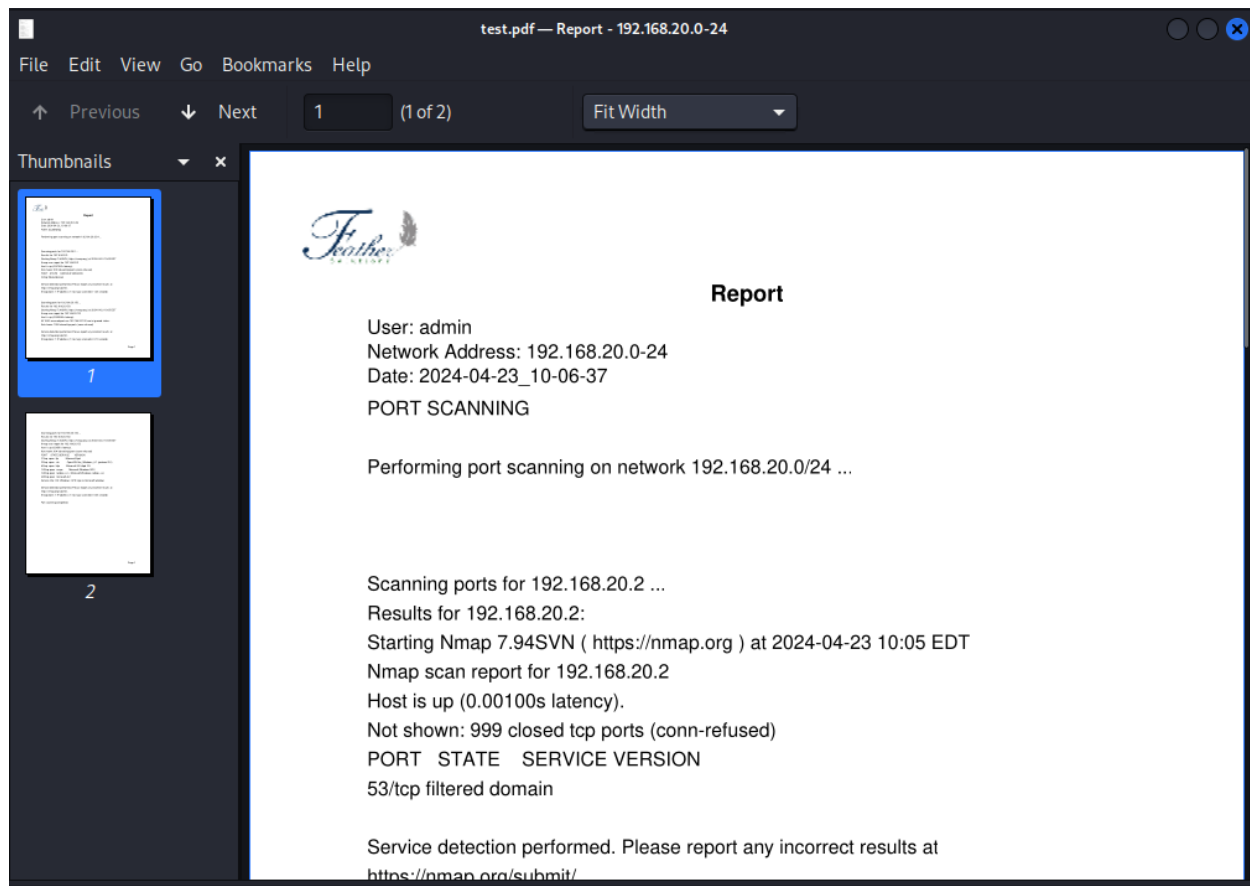


Figure 93: Report.

4.4 Critical Analysis

The results derived from both unit testing and system testing unequivocally demonstrate the successful execution of all test cases, affirming the flawless functionality of every feature within the application. The meticulous documentation of the testing procedures, including comprehensive test tables and annotated screenshots, enhances the transparency and reproducibility of the testing methodology.

Moreover, the development and testing journey of the 'NetGuardian' tool distinctly reflects alignment with its predetermined aims and objectives. Through the integration of various functionalities, the tool effectively addresses the challenges of network vulnerability assessment, empowering users to enhance network security proactively.

5 Conclusion

5.1 Legal, Social, and Ethical Issues

5.1.1 Legal Issues

- Privacy and Data Protection Laws:

NetGuardian collects and processes sensitive information, such as user credentials, and vulnerability data. Depending on the jurisdiction and the nature of the data collected, the tool must comply with relevant privacy and data protection laws, such as the General Data Protection Regulation (GDPR) in the European Union or the California Consumer Privacy Act (CCPA) in the United States.

- Intellectual Property Rights:

The development of NetGuardian involves the use of third-party libraries, frameworks, and API, which could be subject to intellectual property rights, such as copyrights or patents.

- Cybersecurity Regulations:

Depending on the industry and the type of data being protected, there may be specific cybersecurity regulations and standards that NetGuardian must adhere to.

- Liability for Security Breaches:

While NetGuardian aims to enhance network security, it is not immune to security vulnerabilities itself. If the tool is found to have security flaws that result in a data breach or other cybersecurity incident, the developers and operators of NetGuardian may be held liable for damages.

5.1.2 Social Issues

- Trust and Transparency:

NetGuardian collects and processes sensitive information about user credentials and vulnerabilities. Users may be concerned about how their data is

being used and whether it is adequately protected from unauthorized access or misuse.

- **Impact on Employment:**

The automation of certain cybersecurity tasks through tools like NetGuardian may have implications for employment within the cybersecurity industry. While automation can improve efficiency and accuracy in detecting and mitigating security threats, it may also reduce the demand for manual cybersecurity tasks traditionally performed by human analysts.

- **Ethical Considerations:**

NetGuardian's capabilities, such as vulnerability detection and credential testing, raise ethical questions about the appropriate use of cybersecurity tools and techniques. Users must consider the ethical implications of scanning and examining network systems, particularly when conducting assessments on networks belonging to third parties or unauthorized systems.

- **Cybersecurity Culture:**

NetGuardian's effectiveness relies not only on its technical capabilities but also on the cybersecurity culture within organizations. Promoting a culture of cybersecurity awareness, responsibility, and accountability is critical for ensuring the successful implementation and adoption of NetGuardian.

5.1.3 Ethical Issues

- **Privacy and Data Protection:**

NetGuardian collects and analyzes sensitive information about network configurations, user credentials, and vulnerabilities. Ethical considerations arise regarding how this data is obtained, stored, and used.

- **Responsible Disclosure:**

NetGuardian may identify vulnerabilities in network systems that could potentially be exploited by malicious actors. Ethical considerations arise

regarding the responsible disclosure of vulnerabilities to relevant parties, such as network administrators or software vendors.

- Dual-Use Concerns:

NetGuardian's capabilities, such as vulnerability detection and credential testing, may be used for both defensive and offensive purposes. Ethical considerations arise regarding the responsible use of cybersecurity tools and techniques to ensure that they are not used for malicious or unauthorized activities.

5.2 Advantages

NetGuardian presents a myriad of advantages that make it a compelling choice for organizations seeking to enhance their network security posture. They are:

- It is free and open-source, providing unparalleled accessibility and flexibility, allowing organizations to access, modify, and distribute the tool without any licensing restrictions.
- It is an affordable solution especially beneficial for small and medium-sized organizations with limited budgets.
- It is designed with simplicity and Streamlines the vulnerability assessment process, making it accessible and easy to navigate.
- Generates comprehensive reports of vulnerability assessments. Provides actionable insights into the security posture of the network, highlighting potential vulnerabilities.
- It incorporates user authentication to add an essential layer of security and ensures that only authorized personnel can access the tool, mitigating the risk of unauthorized access to sensitive network information.
- It also detects weak or default passwords on host devices, strengthening organizations' defenses against cyber threats.

5.3 Limitations

While NetGuardian offers numerous advantages, it also faces certain limitations that organizations should consider before implementing the tool. They are:

- The tool relies on third-party tools and the National Vulnerability Database (NVD) API for functionalities like port scanning and vulnerability detection.
- Users may experience delays or sluggish performance during network vulnerability assessments.
- The tool lacks a centralized dashboard for graphical representation of network security metrics and trends. Dashboards provide at-a-glance insights into network health, vulnerability status, and threat trends.
- The tool is only compatible with Linux-based systems, posing challenges for organizations with diverse IT environments.

5.4 Future Work

As the field of cybersecurity continues to evolve rapidly in response to emerging threats and technological advancements, the NetGuardian project presents numerous opportunities for future development and enhancement. In this section, we outline potential areas of future work to further strengthen and expand the capabilities of the NetGuardian network vulnerability assessment tool.

- Integration of Machine Learning Algorithms:

One promising avenue for future work involves the integration of machine learning algorithms into NetGuardian. Machine learning techniques can enhance the tool's ability to detect and classify network vulnerabilities by analyzing patterns in network traffic, system logs, and historical attack data. By leveraging machine learning models, NetGuardian can adapt and evolve its detection capabilities over time, improving its accuracy in identifying emerging threats and zero-day vulnerabilities.

- Enhanced Reporting:

Improving the reporting capabilities of NetGuardian represents another area for future development. Incorporating customizable report templates and automated report generation features can streamline the process of communicating assessment findings to stakeholders, facilitating informed decision-making and compliance reporting.

- Integration with Threat Intelligence Platforms:

Integrating NetGuardian with external threat intelligence platforms can enrich its vulnerability assessment capabilities by providing real-time insights into known threats, exploits, and attack vectors. By leveraging threat intelligence feeds from reputable sources, such as government agencies, cybersecurity vendors, and industry consortia, NetGuardian can enhance its ability to prioritize vulnerabilities based on their likelihood of exploitation and potential impact on the organization's security posture.

- Support for Cloud-based Environments:

With the increasing adoption of cloud computing and hybrid infrastructure models, future iterations of NetGuardian should include support for assessing vulnerabilities in cloud-based environments. This entails developing functionalities to scan virtualized networks, containerized applications, and cloud service configurations for security weaknesses and misconfigurations. Additionally, integration with cloud security posture management (CSPM) tools can provide organizations with comprehensive visibility and control over their cloud security posture.

- Enhanced Support for Compliance and Regulatory Frameworks:

Future iterations of NetGuardian should prioritize enhancing support for compliance and regulatory frameworks, such as the General Data Protection Regulation (GDPR), Payment Card Industry Data Security Standard (PCI DSS), and Health Insurance Portability and Accountability Act (HIPAA). This includes developing

predefined scan templates, audit checks, and reporting mechanisms tailored to specific regulatory requirements. Additionally, incorporating automated compliance assessments and continuous monitoring capabilities can help organizations maintain compliance posture and mitigate regulatory risks effectively.

- Scalability and Performance Optimization:

As organizations expand their digital footprints and infrastructure complexity grows, scalability and performance optimization become critical considerations for vulnerability assessment tools like NetGuardian (Fu et al., 2019). Future work should focus on optimizing the tool's performance to handle large-scale network environments, distributed architectures, and high-volume traffic without compromising accuracy or reliability. This may involve implementing parallel processing techniques, distributed scanning capabilities, and resource-efficient algorithms to ensure scalability and responsiveness.

- Support for Multiple Operating Systems:

To cater to diverse organizational environments, future versions of NetGuardian should aim to support multiple operating systems, including Windows, macOS, and Linux distributions. This cross-platform compatibility ensures that organizations can deploy NetGuardian seamlessly across their infrastructure without encountering compatibility issues.

- Responsive GUI and Dashboard for Graphical Visualization:

Enhancing the user experience, NetGuardian could incorporate a responsive Graphical User Interface (GUI) and interactive dashboard for graphical visualization of assessment findings. This would allow users to navigate through network vulnerabilities intuitively and gain actionable insights through visually appealing charts, graphs, and diagrams.

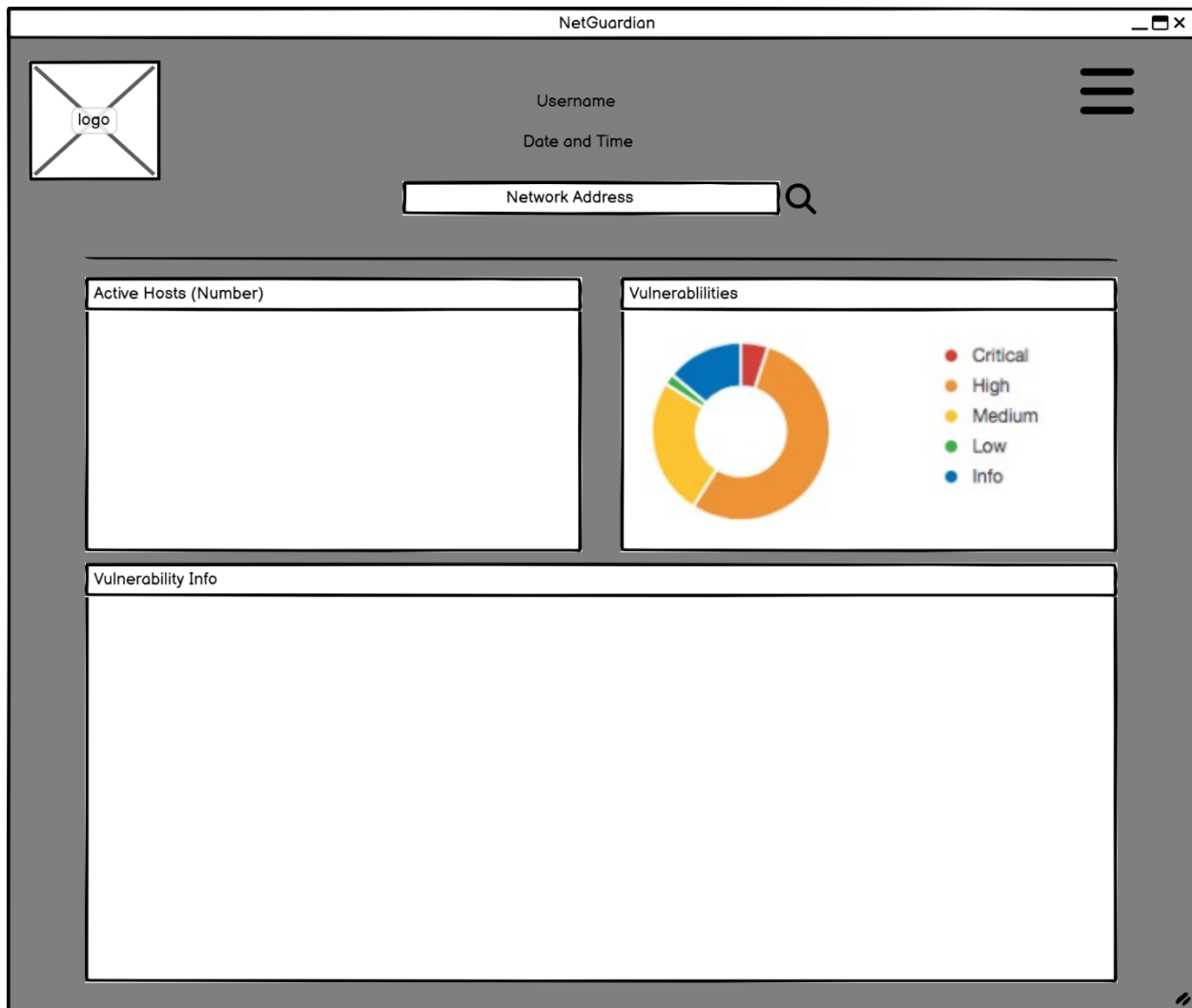


Figure 94: Wireframe of the System Dashboard

- Scheduled Automatic Scans:

Implementing the ability for users to schedule automatic scans on a weekly or customizable time basis can streamline vulnerability assessment processes and ensure continuous monitoring of network security (Nguyen et al., 2020). This feature allows organizations to proactively identify and address emerging threats without manual intervention, enhancing overall security posture.

- Implement User Behavior Analytics (UBA).

The implementation of user behavior analytics (UBA) within NetGuardian will enhance its capabilities for detecting and mitigating cybersecurity threats. By analyzing patterns of user behavior, NetGuardian will be able to identify anomalies and suspicious activities that may indicate unauthorized access or malicious intent. UBA algorithms will provide insights into user activity, enabling proactive threat detection and timely response to security incidents.

- Productization and Launch:

Finally, transitioning NetGuardian from a project to a product involves productization and launch activities. This includes refining the user interface, conducting thorough testing and quality assurance, developing documentation and user guides, implementing licensing and distribution strategies, and marketing the product to target audiences. By commercializing NetGuardian, organizations can maximize its reach and impact, empowering businesses of all sizes to enhance their network security defenses effectively.

In conclusion, the future work outlined above represents a roadmap for advancing the NetGuardian network vulnerability assessment tool to meet the evolving needs and challenges of modern cybersecurity landscapes. By embracing innovation, collaboration, and continuous improvement, NetGuardian aims to remain at the forefront of network security solutions, empowering organizations to safeguard their digital assets effectively against emerging threats and vulnerabilities.

6 References

Bakharev, N., 2024. *Unit testing: Definition, examples, and critical best practices*.

[Online]

Available at: <https://brightsec.com/blog/unit-testing/>

[Accessed 23 04 2024].

Bank of North Dakota, 2018. *81% of company data breaches due to poor passwords*.

[Online]

Available at: <https://bnd.nd.gov/81-of-company-data-breaches-due-to-poor-passwords/>

[Accessed 14 04 2024].

Black Hat Ethical Hacking, 2022. *Offensive security tool: Sparta*. [Online]

Available at: <https://www.blackhatethicalhacking.com/tools/sparta/>

[Accessed 14 04 2024].

Chidukwani, A., Zander, S. & Koutsakis, P., 2022. A survey on the cyber security of small-to-medium businesses: Challenges, research focus and recommendations. *IEEE Access*, Volume 10, pp. 85701-85719.

Chopskie, E., 2023. *Vulnerability assessment tools: Key features and 5 tools you should know*. [Online]

Available at: [https://brightsec.com/blog/vulnerability-assessment-tools-key-features-and-5-tools-you-should-](https://brightsec.com/blog/vulnerability-assessment-tools-key-features-and-5-tools-you-should-know/#:~:text=One%20of%20the%20most%20essential,activity%20but%20a%20continuous%20process.)

[know/#:~:text=One%20of%20the%20most%20essential,activity%20but%20a%20continuous%20process.](https://brightsec.com/blog/vulnerability-assessment-tools-key-features-and-5-tools-you-should-know/#:~:text=One%20of%20the%20most%20essential,activity%20but%20a%20continuous%20process.)

[Accessed 18 04 2024].

Curtis, J. & Oxburgh, G., 2022. Understanding cybercrime in 'real world' policing and law enforcement. *The Police Journal: Theory, Practice and Principles*, 96(4), pp. 573-592.

Figma, 2024. *What is wireframing? The Complete Guide [Free Checklist]*. [Online]

Available at: <https://www.figma.com/resource-library/what-is-wireframing/>

[Accessed 17 04 2024].

Ganesh, B., 2023. *Sparta - GUI toolkit to perform network penetration testing*. [Online] Available at: <https://gbhackers.com/sparta-network-penetration-testing-gui-toolkit/> [Accessed 23 April 2024].

Guida, G., Lamperti, G. & Hazewinkel, M., 2012. *Software prototyping in data and Knowledge Engineering*. Dordrecht: Springer Netherlands.

Gupta, D., Sharma, S. & Sarkar, D., 2012. Agile Processes and Methodologies: A Conceptual Study. *International Journal on Computer Science and Engineering* 4(5).

Issuu, 2023. *Agile development: A comprehensive guide for the modern era*. [Online] Available at: https://issuu.com/mobileliveca/docs/agile_development_a_comprehensive_guide_for_the_modern_era [Accessed 14 04 2024].

Kaspersky, 2022. *Kaspersky Threat Intelligence*. [Online] Available at: <https://www.kaspersky.com/enterprise-security/threat-intelligence> [Accessed 23 04 2024].

Knieriem, B. et al., 2018. An overview of the usage of default passwords. *Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering*, pp. 195-203.

Lapada, A., 2019. AIRTOR: A Portable Air Quality Monitor. *Indian Journal of Science and Technology*, 12(17), pp. 1-11.

Lucidchart, 2024. *What is a data flow diagram*. [Online] Available at: <https://www.lucidchart.com/pages/data-flow-diagram> [Accessed 17 04 2024].

Malla, A. & Timala, R. B., 2022. *A study on cyber crime cases in Nepal: Challenges and recommendations 2022*. Kathmandu: National Judicial Academy, Nepal.

Malla, . A. & Timala, R. B., 2022. *A Study on Cyber Crime Cases in Nepal: Challenges and Recommendations 2022*. [Online] Available at: <https://njanepal.org.np/public/reports/22071352423-Final A Study on Cyber>

Crime Cases in Nepal, 2022.pdf

[Accessed 14 04 2024].

MindMapping, 2024. *What is a mind map?*. [Online]

Available at: <https://www.mindmapping.com/mind-map>

[Accessed 17 04 2024].

Nmap, 2022. *Chapter 15. nmap reference guide*. [Online]

Available at: <https://nmap.org/book/man.html>

[Accessed 23 April 2024].

openvas, 2024. *Greenbone openvas*. [Online]

Available at: <https://www.openvas.org/>

[Accessed 14 04 2024].

Pressman, R. S., 2010. *Software engineering a practitioner's approach*. 7th ed. New York: McGraw-Hill.

Rapid7, 2024. *What is user and entity behavior analytics (UEBA)?*. [Online]

Available at: <https://www.rapid7.com/fundamentals/user-behavior-analytics/>

[Accessed 23 04 2024].

Rivera, A., 2020. *Building Docker images for multiple operating system architectures*.

[Online]

Available at: <https://circleci.com/blog/building-docker-images-for-multiple-os-architectures/>

[Accessed 23 04 2024].

Senarath, U. S., 2021. *Waterfall Methodology, Prototyping and Agile Development. Research*.

Senarath, U. S., 2021. *Waterfall Methodology, Prototyping and Agile Development. ResearchGate*.

Sharma, R. & Zamfiroiu, A., 2023. *Cybersecurity threats and vulnerabilities in the metaverse. 2023 International Conference on Intelligent Metaverse Technologies & Applications (iMETA)*.

Tenable®, 2024. *NESSUS Vulnerability Scanner: Network security solution*. [Online]
Available at: <https://www.tenable.com/products/nessus>
[Accessed 14 04 2024].

Tenable, 2024. *Nessus 10.4 user guide*. [Online]
Available at: https://docs.tenable.com/nessus/10_4/Content/PDF/Nessus_10_4.pdf
[Accessed 24 April 2024].

The Economic Times, 2024. *What is system testing? definition of system testing, system testing meaning*. [Online]
Available at: <https://economictimes.indiatimes.com/definition/system-testing>
[Accessed 23 04 2024].

Wadhwani, . P., 2023. *Security and Vulnerability Management Market Size*. [Online]
Available at: <https://www.gminsights.com/industry-analysis/security-and-vulnerability-management-market>
[Accessed 23 April 2024].

7 Bibliography

Hydra: Kali linux tools (2024) *Kali Linux*. Available at: <https://www.kali.org/tools/hydra/> (Accessed: 24 April 2024).

Lyon, G. (2010) *Nmap network scanning official NMAP project guide to network discovery and security scanning*. Sunnyvale, CA: Insecure.Com LLC.

Nenov, L., Zlatanov, I. and Manolova, A. (2022) 'Network tool for hybrid analysis with external and internal scanning for Vulnerability Assessment of network nodes', *2022 25th International Symposium on Wireless Personal Multimedia Communications (WPMC)* [Preprint]. doi:10.1109/wpmc55625.2022.10014942.

Patel, H. (2023) *An insight on (SDLC) software development lifecycle process models* [Preprint]. doi:10.31124/advance.22354453.

Project management definition, basics and approaches explained (2023) *Kissflow*. Available at: <https://kissflow.com/project/project-management-basics/> (Accessed: 24 April 2024).

Rahalkar, S. (2018) *Network vulnerability assessment: Identify security loopholes in your network's infrastructure*. Birmingham, Mumbai: Packt.

'Guide to vulnerability analysis for computer networks and systems' (2018) *Computer Communications and Networks* [Preprint]. doi:10.1007/978-3-319-92624-7.

8 Appendix

8.1 Appendix A: Considered Methodology

8.1.1 Waterfall Methodology

The Waterfall Model was the first process model to be introduced, often referred to as a linear sequential life cycle model due to the software development process following a linear sequential flow (Senarath, 2021). In this approach, the entire process of software development is divided into distinct phases. Typically, the outcome of one phase serves as the input for the subsequent phase in a sequential manner. The Waterfall Model is particularly well-suited for projects where the objectives are clearly defined from the outset.

The reasons for not selecting the Waterfall methodology for this project are:

- The model follows a linear sequence of steps that keeps a team moving forward. However, this traditional approach offers almost no room for change if something unexpected occurs during the project.
- Testing is often delayed until the end of development. This could lead to the late discovery of issues or bugs, which can be costly and time-consuming to fix.
- The model does not incorporate mid-process feedback from users or clients, and changes are made based on results. This means that if a requirement changes or is misunderstood, the error might not be discovered until the project is near completion.
- All requirements must be known before development, which can greatly delay the project kick-off. This can be challenging in projects where requirements may evolve or are not fully known at the start. The low flexibility level of the Waterfall model makes it difficult to make changes during development, and in some cases, it may even be impossible.

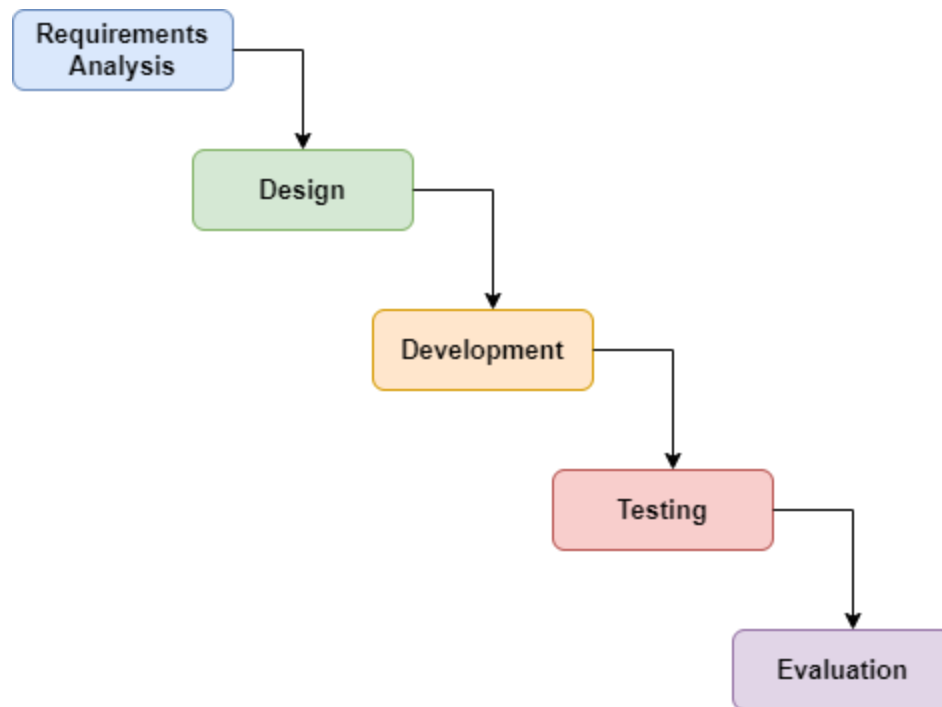


Figure 95: Waterfall Methodology Phases (Senarath, 2021).

8.1.2 Agile Methodology

Agile is a project management methodology that is iterative, introspective, and adaptive. In Agile practice, a project is divided into smaller sub-projects, commonly known as sprints. Upon the completion of each sprint, stakeholders and the team review the work done, make necessary adjustments for the upcoming sprint, and continue this process until the project is completed. The essence of Agile lies in the continuous, incremental delivery of value throughout the project's duration, rather than delivering it all at once at the end (Gupta, et al., 2012).

The agile methodology is not being selected for this project due to its inherent characteristics. The iterative nature of agile could potentially result in a project extending indefinitely, which is not suitable for this project that requires a defined timeline. Additionally, agile's sprint-based approach can make it difficult to estimate the total time, effort, cost, and resources at the start of a project. These factors combined make Agile a less desirable choice for this project.



Figure 96: Agile Methodology Phases (Issuu, 2023).

Back to the previous [page](#).

8.2 Appendix B: Pre-Survey

8.2.1 Pre-Survey Form

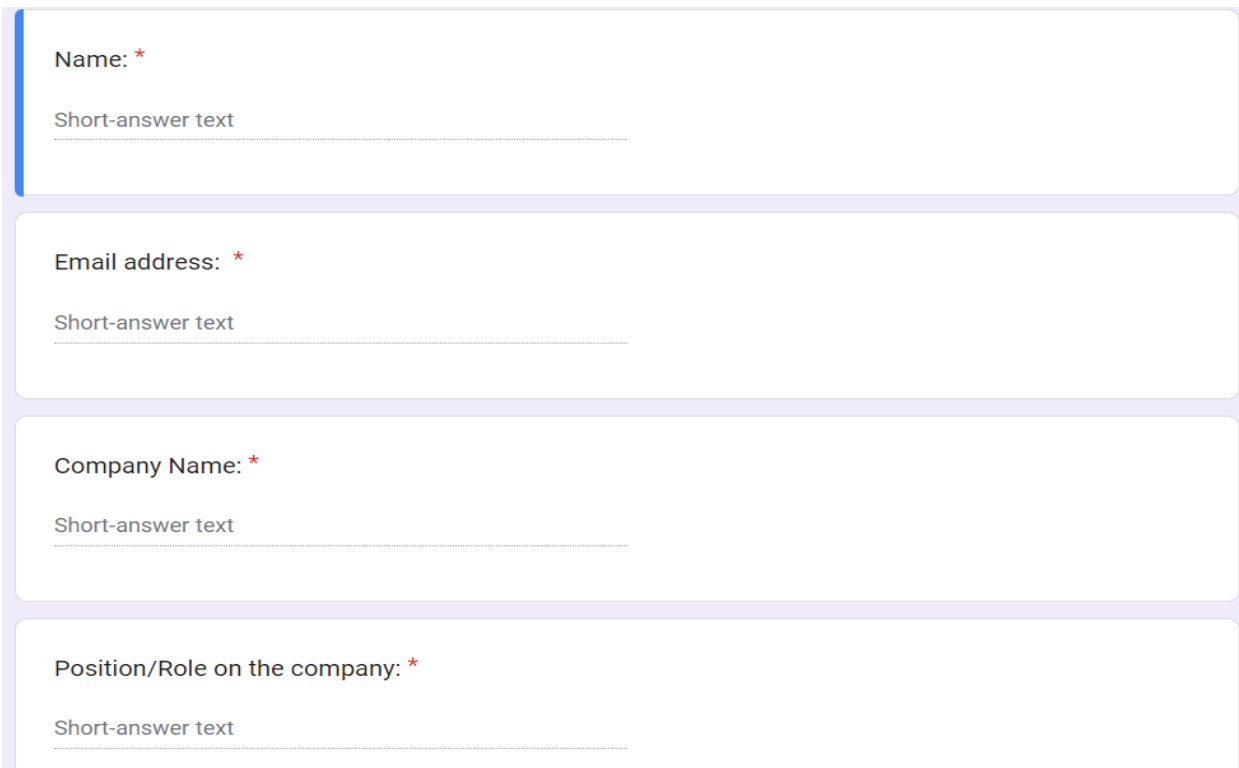
Survey Form: Network Vulnerability Assessment Tool

Thank you for participating in this survey! Your feedback is essential for improving the effectiveness and user experience of the "Network Vulnerability Assessment Tool" project. This survey aims to gather insights into your understanding of the project, your experiences with the network vulnerability assessment tool, feedback on the user interface, challenges, and overall project comprehension.

Please take a few minutes to share your thoughts and experiences by answering the following questions. Your responses will help us identify areas for improvement and ensure that the project aligns with user expectations.

Your participation is voluntary, and all responses will be kept confidential. If you have any questions or concerns, please feel free to reach out.

Figure 97: Pre-Survey Form Part 1.



This figure shows a vertical stack of four form sections. Each section has a label followed by a red asterisk, a 'Short-answer text' label, and a dotted line for input. The sections are: Name, Email address, Company Name, and Position/Role on the company.

Name: *

Short-answer text

Email address: *

Short-answer text

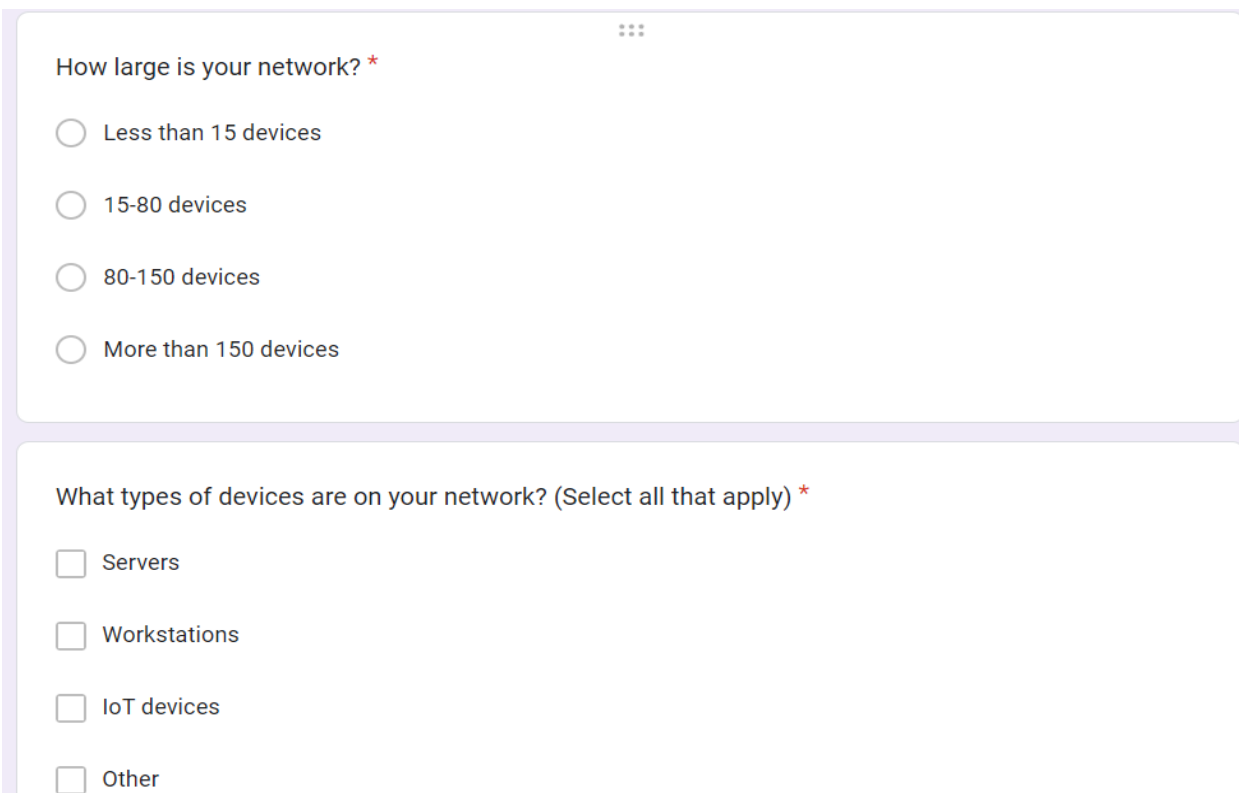
Company Name: *

Short-answer text

Position/Role on the company: *

Short-answer text

Figure 98: Pre-Survey Form Part 2.



This figure shows two form sections. The first section is titled 'How large is your network?' with a red asterisk and a three-dot menu icon. It contains four radio button options. The second section is titled 'What types of devices are on your network? (Select all that apply)' with a red asterisk. It contains four checkbox options.

How large is your network? *

Less than 15 devices

15-80 devices

80-150 devices

More than 150 devices

What types of devices are on your network? (Select all that apply) *

Servers

Workstations

IoT devices

Other

Figure 99: Pre-Survey Form Part 3.

Which operating systems are in use on your network? (Select all that apply) *

- ☐ Windows
- ☐ macOS
- ☐ Linux
- ☐ Other

How familiar are you with network vulnerability assessment tools? *

- ☐ Very Familiar
- ☐ Somewhat Familiar
- ☐ Neutral
- ☐ Not Very Familiar
- ☐ Not Familiar at All

Figure 100: Pre-Survey Form Part 4.

Do you currently use any network vulnerability assessment tools? *

- ☐ Yes
- ☐ No

If yes, which network vulnerability assessment tool(s) do you use? (Select all that apply) *

- ☐ Nessus
- ☐ OpenVAS
- ☐ Nexpose
- ☐ Other

Figure 101: Pre-Survey Form Part 5.

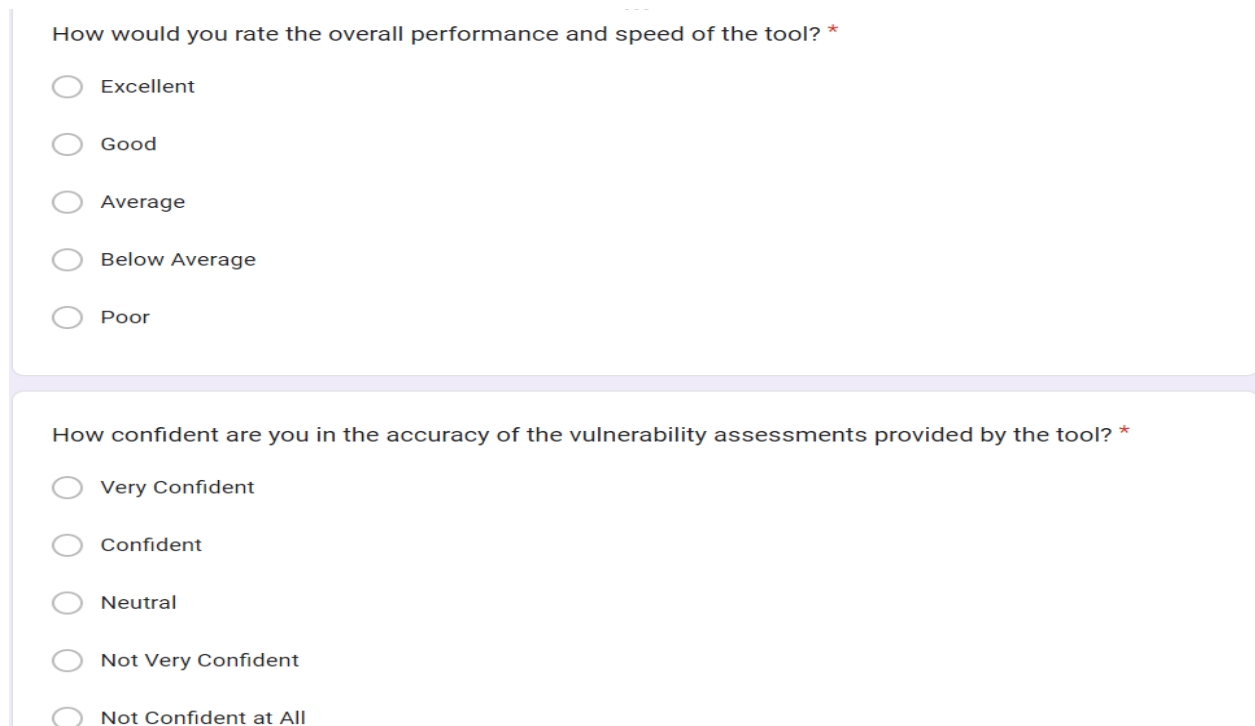
Which features do you find most important in a network vulnerability assessment tool? (Select ^{*} all that apply)

- ☐ Comprehensive vulnerability scanning
- ☐ Detailed reporting
- ☐ User-friendly
- ☐ Integration with other systems
- ☐ Other

How frequently do you perform network vulnerability assessments? ^{*}

- ☐ Daily
- ☐ Weekly
- ☐ Monthly
- ☐ Annually

Figure 102: Pre-Survey Form Part 6.



How would you rate the overall performance and speed of the tool? *

☐ Excellent

☐ Good

☐ Average

☐ Below Average

☐ Poor

How confident are you in the accuracy of the vulnerability assessments provided by the tool? *

☐ Very Confident

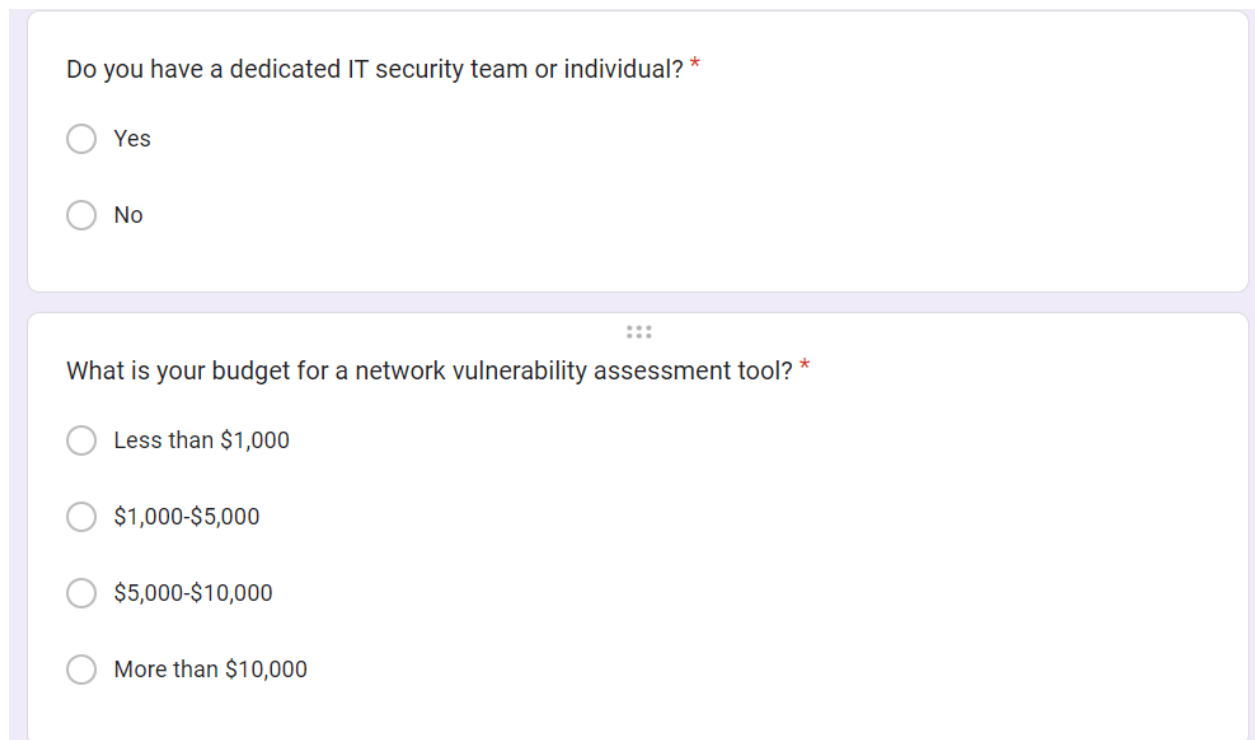
☐ Confident

☐ Neutral

☐ Not Very Confident

☐ Not Confident at All

Figure 103: Pre-Survey Form Part 7.



Do you have a dedicated IT security team or individual? *

☐ Yes

☐ No

What is your budget for a network vulnerability assessment tool? *

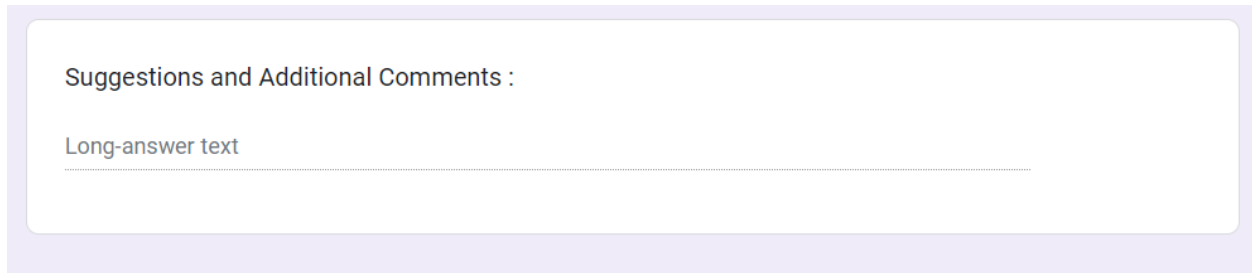
☐ Less than \$1,000

☐ \$1,000-\$5,000

☐ \$5,000-\$10,000

☐ More than \$10,000

Figure 104: Pre-Survey Form Part 8.

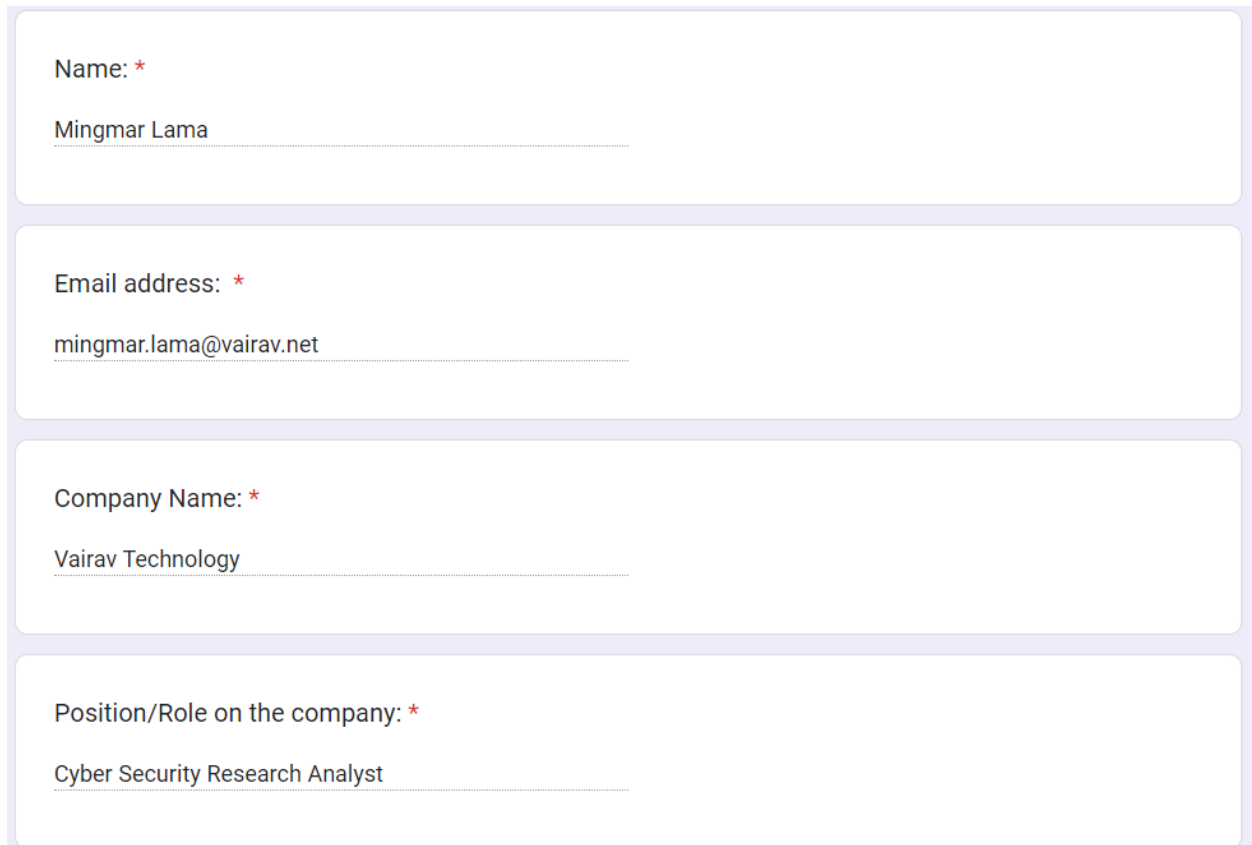


Suggestions and Additional Comments :

Long-answer text

Figure 105: Pre-Survey Form Part 9.

8.2.2 Sample of Filled Pre-survey Forms



Name: *

Mingmar Lama

Email address: *

mingmar.lama@vairav.net

Company Name: *

Vairav Technology

Position/Role on the company: *

Cyber Security Research Analyst

Figure 106: Filled Pre-Survey Forms Part 1.

How large is your network? *

☐ Less than 15 devices

☒ 15-80 devices

☐ 80-150 devices

☐ More than 150 devices

What types of devices are on your network? (Select all that apply) *

☒ Servers

☒ Workstations

☒ IoT devices

☐ Other

Figure 107: Filled Pre-Survey Forms Part 2.

Which operating systems are in use on your network? (Select all that apply) *

- ☒ Windows
- ☒ macOS
- ☒ Linux
- ☐ Other

How familiar are you with network vulnerability assessment tools? *

- ☐ Very Familiar
- ☐ Somewhat Familiar
- ☒ Neutral
- ☐ Not Very Familiar
- ☐ Not Familiar at All

Figure 108: Filled Pre-Survey Forms Part 3.

Do you currently use any network vulnerability assessment tools? *

☐ Yes

☒ No

If yes, which network vulnerability assessment tool(s) do you use? (Select all that apply) *

☐ Nessus

☐ OpenVAS

☐ Nexpose

☒ Other

Figure 109: Filled Pre-Survey Forms Part 4.

Which features do you find most important in a network vulnerability assessment tool? (Select all that apply) *

- ☒ Comprehensive vulnerability scanning
- ☒ Detailed reporting
- ☒ User-friendly
- ☒ Integration with other systems
- ☐ Other

How frequently do you perform network vulnerability assessments? *

- ☐ Daily
- ☐ Weekly
- ☐ Monthly
- ☐ Annually
- ☒ Other

Figure 110: Filled Pre-Survey Forms Part 5.

How would you rate the overall performance and speed of the tool? *

- ☐ Excellent
- ☐ Good
- ☒ Average
- ☐ Below Average
- ☐ Poor

How confident are you in the accuracy of the vulnerability assessments provided by the tool? *

- ☐ Very Confident
- ☐ Confident
- ☒ Neutral
- ☐ Not Very Confident
- ☐ Not Confident at All

Figure 111: Filled Pre-Survey Forms Part 6.

Do you have a dedicated IT security team or individual? *

☒ Yes

☐ No

What is your budget for a network vulnerability assessment tool? *

☒ Less than \$1,000

☐ \$1,000-\$5,000

☐ \$5,000-\$10,000

☐ More than \$10,000

Suggestions and Additional Comments :

Figure 112: Filled Pre-Survey Forms Part 7.

8.2.3 Pre-Survey Result

The survey was conducted among the 10 respondents, the distribution of network sizes varied significantly. Approximately 10% reported using less than 15 devices, 40% reported 15-80 devices, 20% reported 80-150 devices, and 30% reported more than 150 devices. Regarding device types, the majority of respondents reported the presence of servers (80%), workstations (60%), and IoT devices (40%) on their networks. No respondents selected "Other" for this category.

In terms of operating systems, all respondents reported the use of Windows, while 20% reported macOS and 70% reported Linux. No respondents selected "Other" for this category. Half of the respondents indicated being very familiar with network vulnerability

assessment tools, with 40% being somewhat familiar and 10% neutral. None reported being not very familiar or not familiar at all.

The majority of respondents (80%) reported currently using network vulnerability assessment tools, with the most popular tool being Nessus (80%), followed by OpenVAS (30%). No respondents reported using Nexpose, and 20% reported using other tools. When asked about the most important features in a network vulnerability assessment tool, respondents highlighted comprehensive vulnerability scanning (50%), detailed reporting (70%), user-friendliness (60%), and integration with other systems (40%). No respondents selected "Other." Regarding the frequency of vulnerability assessments, the majority of respondents reported performing assessments either weekly (30%) or monthly (40%), with smaller percentages reporting daily (10%) or annually (10%).

In terms of satisfaction with the overall performance and speed of the tool, respondents had varying opinions, with 40% rating it as excellent, 30% as good, and 30% as average. None rated it below average or poor. Confidence in the accuracy of vulnerability assessments provided by the tool was also diverse among respondents, with 30% being very confident, 30% confident, and 40% neutral. No respondents reported being not very confident or not confident at all. Most respondents (80%) reported having a dedicated IT security team or individual. Regarding the budget for a network vulnerability assessment tool, the majority of respondents (50%) reported a budget of \$1,000-\$5,000, while 40% reported less than \$1,000 and 10% reported more than \$10,000.

In the suggestions and additional comments section, three respondents provided feedback. One suggested integrating with other service provider APIs for more accurate and reliable results, while another simply expressed satisfaction with the project. The third respondent provided no additional comments.

How large is your network?

10 responses

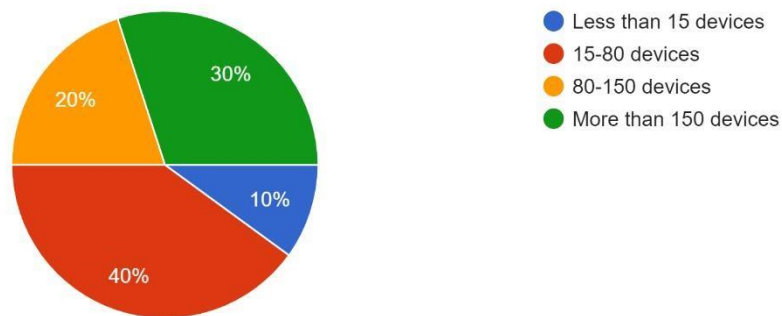


Figure 113: Pre-Survey Question 1 Result.

What types of devices are on your network? (Select all that apply)

10 responses

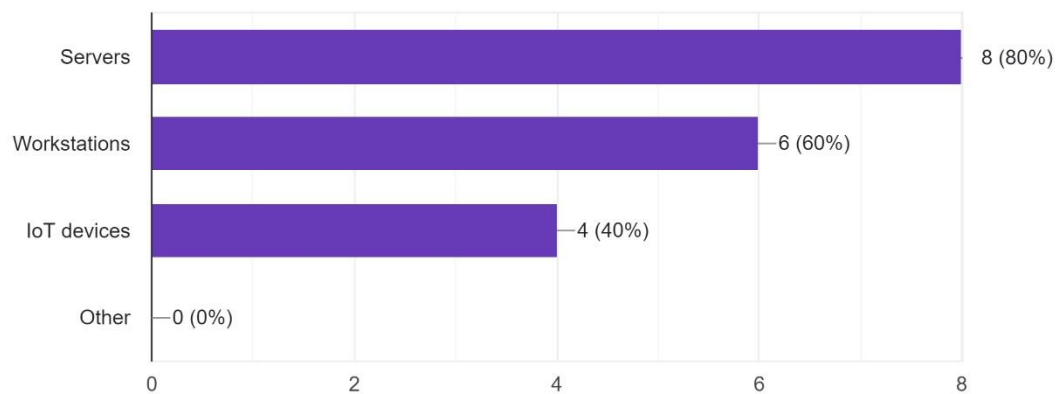


Figure 114: Pre-Survey Question 2 Result.

Which operating systems are in use on your network? (Select all that apply)

10 responses

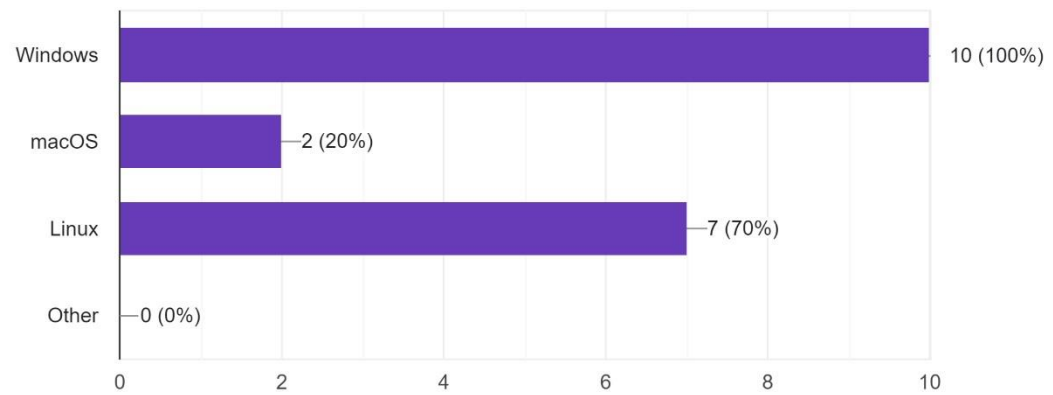


Figure 115: Pre-Survey Question 3 Result.

How familiar are you with network vulnerability assessment tools?

10 responses

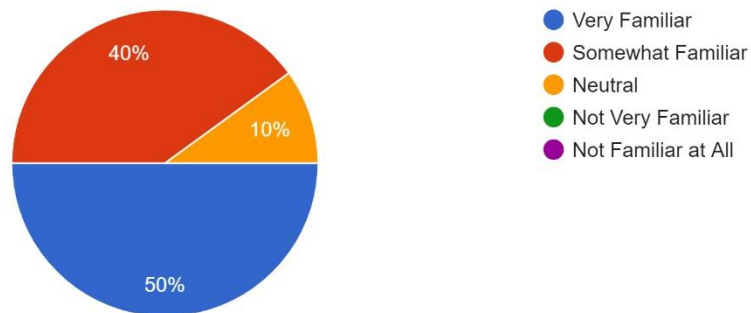


Figure 116: Pre-Survey Question 4 Result.

Do you currently use any network vulnerability assessment tools?

10 responses

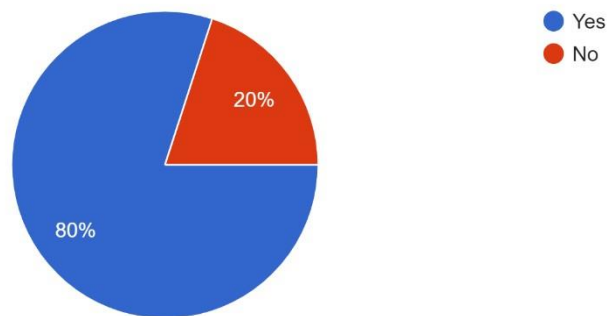


Figure 117: Pre-Survey Question 5 Result.

If yes, which network vulnerability assessment tool(s) do you use? (Select all that apply)

10 responses

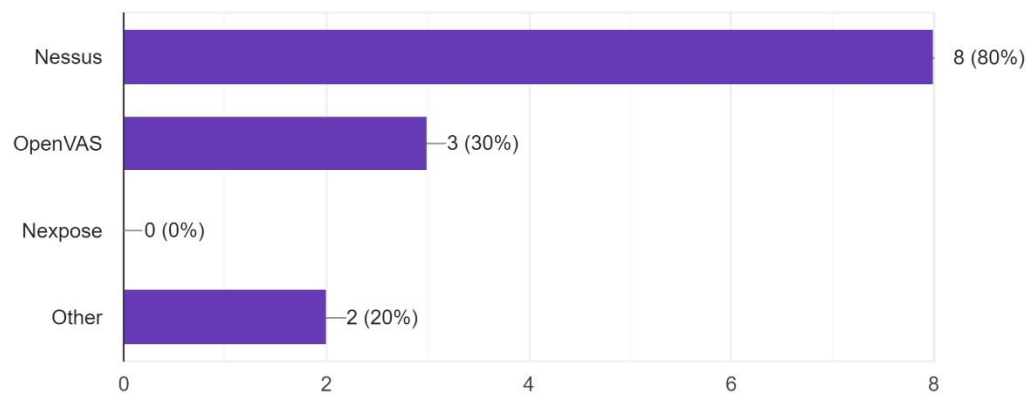


Figure 118: Pre-Survey Question 6 Result.

Which features do you find most important in a network vulnerability assessment tool? (Select all that apply)

10 responses

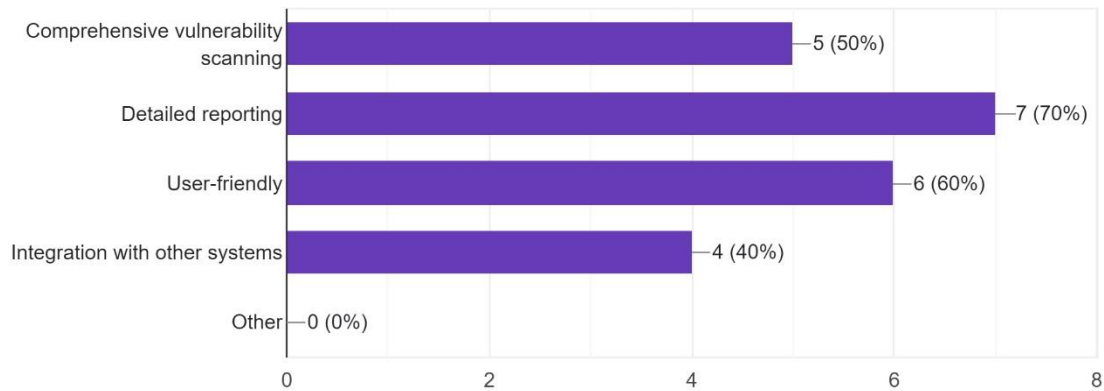


Figure 119: Pre-Survey Question 7 Result.

How frequently do you perform network vulnerability assessments?

10 responses

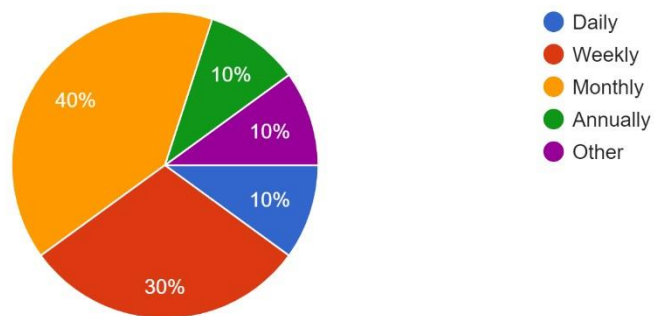


Figure 120: Pre-Survey Question 8 Result.

How would you rate the overall performance and speed of the tool?

10 responses

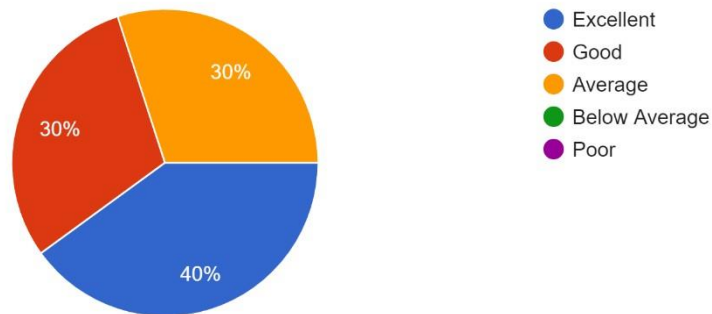


Figure 121: Pre-Survey Question 9 Result.

How confident are you in the accuracy of the vulnerability assessments provided by the tool?

10 responses

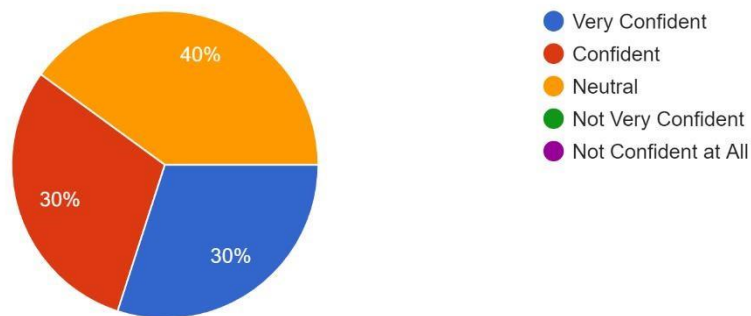


Figure 122: Pre-Survey Question 10 Result.

Do you have a dedicated IT security team or individual?

10 responses

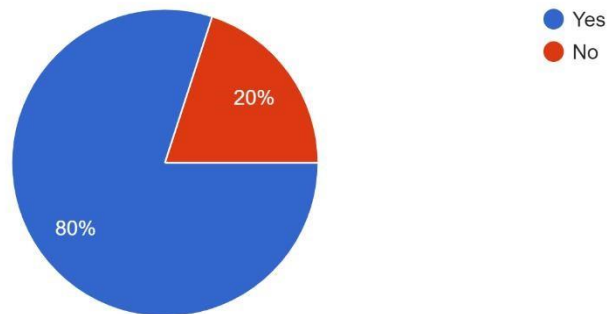


Figure 123: Pre-Survey Question 11 Result.

What is your budget for a network vulnerability assessment tool?

10 responses

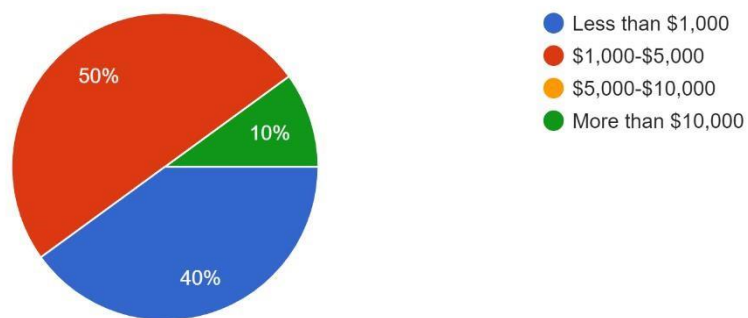


Figure 124: Pre-Survey Question 12 Result.

Back to the previous [page](#).

8.3 Appendix C: Post-Survey

8.3.1 Post-Survey Form

NetGuardian Project Post-Survey

The post-survey is designed to gather feedback from users who have experienced the NetGuardian network vulnerability assessment tool. This survey aims to evaluate various aspects of the project, including user-friendliness, effectiveness in enhancing network security, satisfaction with features and performance, and likelihood of recommendation to others. The responses obtained from this survey will help assess the overall success of the NetGuardian project and identify areas for improvement, ensuring that the tool continues to meet the needs of organizations in fortifying their network defenses against cyber threats.

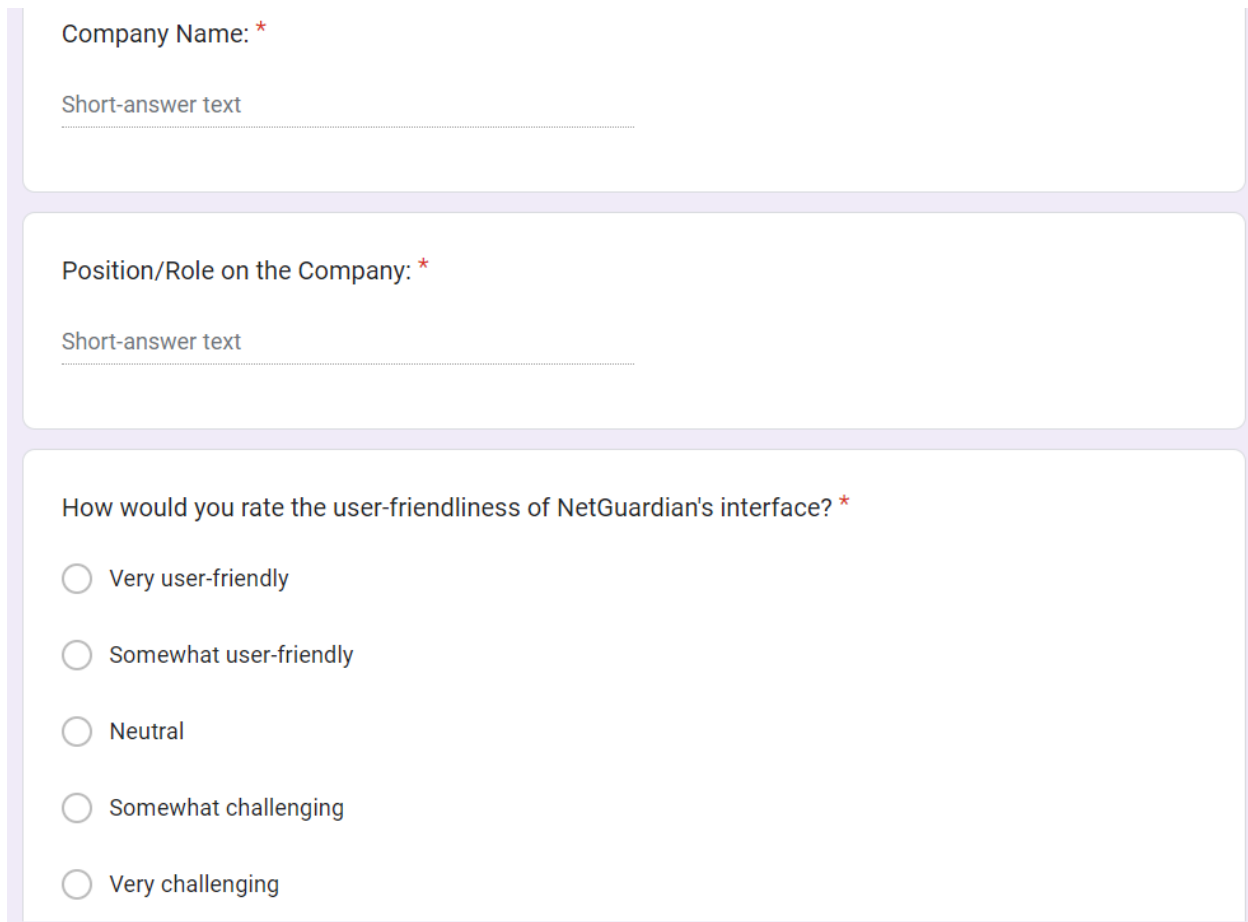
Name: *

Short-answer text

Email: *

Short-answer text

Figure 125: Post-Survey Form Part 1.



Company Name: *

Short-answer text

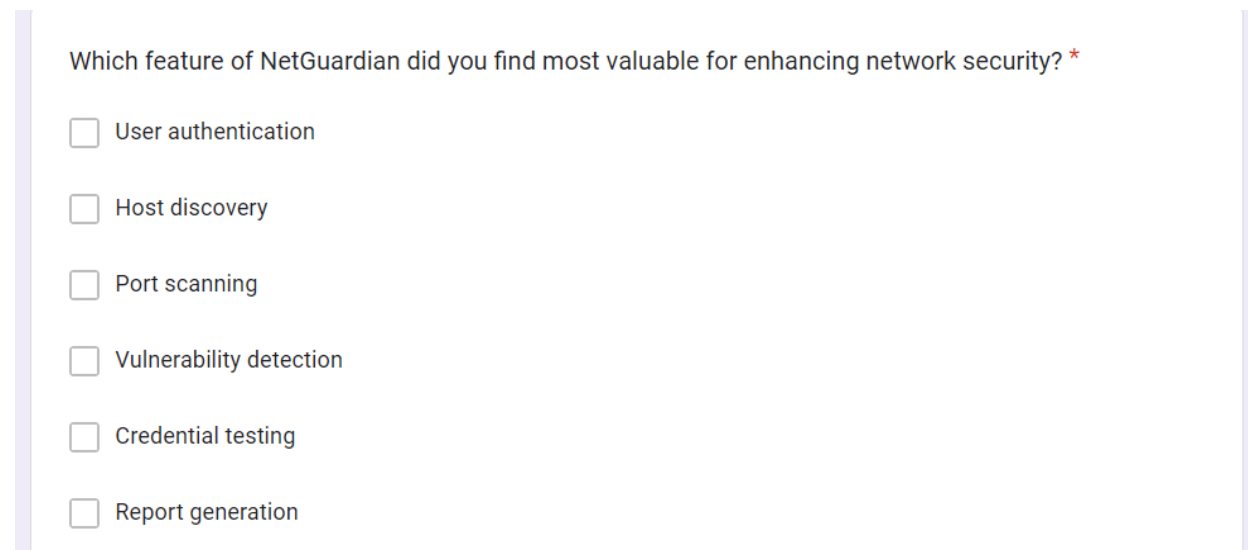
Position/Role on the Company: *

Short-answer text

How would you rate the user-friendliness of NetGuardian's interface? *

- ☐ Very user-friendly
- ☐ Somewhat user-friendly
- ☐ Neutral
- ☐ Somewhat challenging
- ☐ Very challenging

Figure 126: Post-Survey Form Part 2.



Which feature of NetGuardian did you find most valuable for enhancing network security? *

- ☐ User authentication
- ☐ Host discovery
- ☐ Port scanning
- ☐ Vulnerability detection
- ☐ Credential testing
- ☐ Report generation

Figure 127: Post-Survey Form Part 3.

On a scale of 1 to 5, how effective do you believe NetGuardian is in identifying potential network vulnerabilities? *

☐ 1 (Not effective at all)

☐ 2

☐ 3

☐ 4

☐ 5 (Extremely effective)

How likely are you to recommend NetGuardian to other organizations for improving their network security? *

☐ Very likely

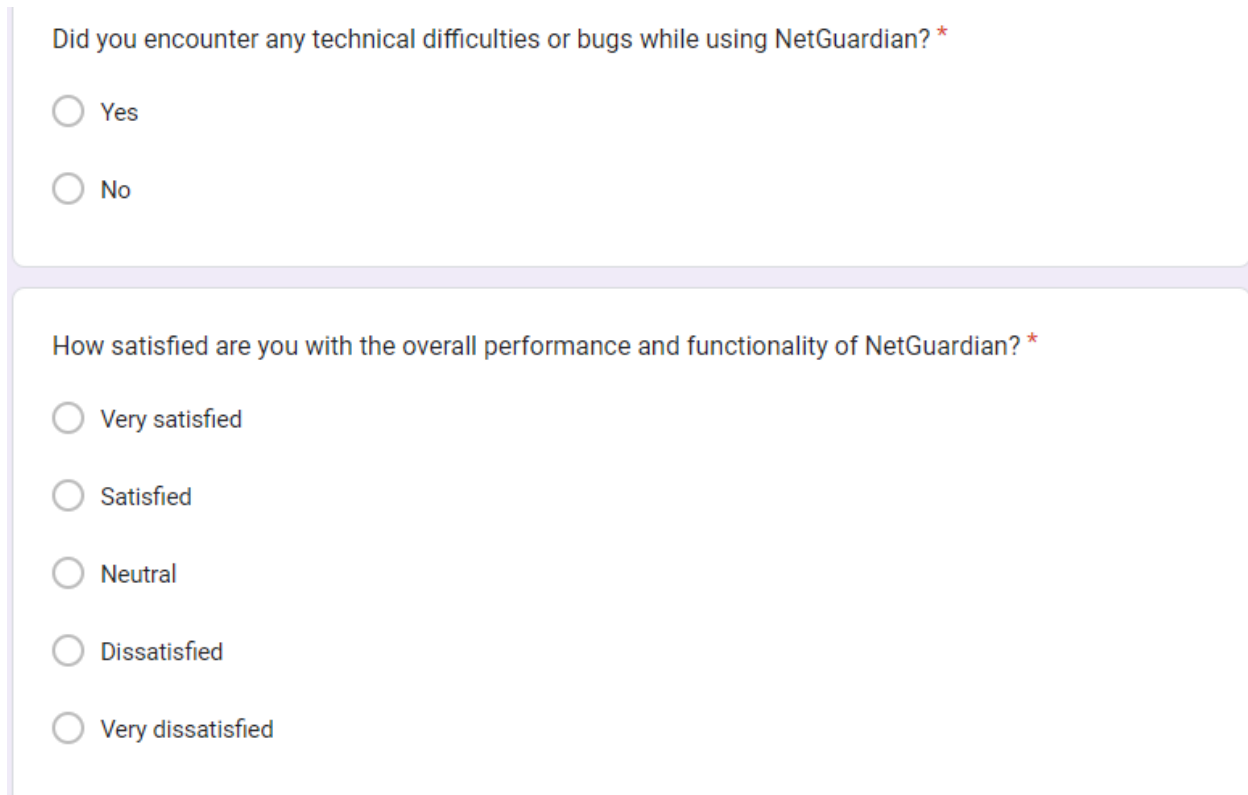
☐ Likely

☐ Neutral

☐ Unlikely

☐ Very unlikely

Figure 128: Post-Survey Form Part 4.



Did you encounter any technical difficulties or bugs while using NetGuardian? *

☐ Yes

☐ No

How satisfied are you with the overall performance and functionality of NetGuardian? *

☐ Very satisfied

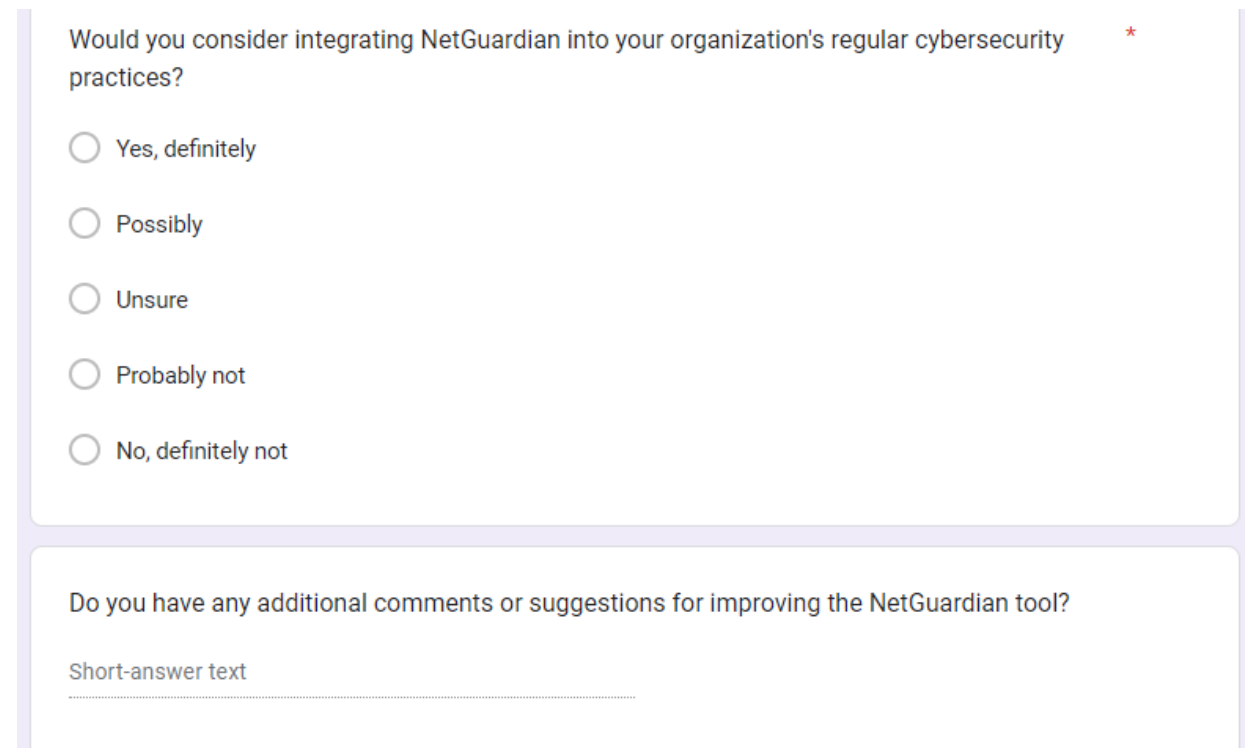
☐ Satisfied

☐ Neutral

☐ Dissatisfied

☐ Very dissatisfied

Figure 129: Post-Survey Form Part 5.



Would you consider integrating NetGuardian into your organization's regular cybersecurity practices? *

☐ Yes, definitely

☐ Possibly

☐ Unsure

☐ Probably not

☐ No, definitely not

Do you have any additional comments or suggestions for improving the NetGuardian tool?

Short-answer text

Figure 130: Post-Survey Form Part 6.

8.3.2 Sample of Filled Post-Survey Forms

Responses cannot be edited

NetGuardian Project Post-Survey

The post-survey is designed to gather feedback from users who have experienced the NetGuardian network vulnerability assessment tool. This survey aims to evaluate various aspects of the project, including user-friendliness, effectiveness in enhancing network security, satisfaction with features and performance, and likelihood of recommendation to others. The responses obtained from this survey will help assess the overall success of the NetGuardian project and identify areas for improvement, ensuring that the tool continues to meet the needs of organizations in fortifying their network defenses against cyber threats.

* Indicates required question

Name: *

Manoj Lama

Email: *

tembamanoj@gmail.com

Figure 131: Filled Post-Survey Forms Part 1.

How would you rate the user-friendliness of NetGuardian's interface? *

☒ Very user-friendly

☐ Somewhat user-friendly

☐ Neutral

☐ Somewhat challenging

☐ Very challenging

Which feature of NetGuardian did you find most valuable for enhancing network security? *

☐ User authentication

☐ Host discovery

☐ Port scanning

☒ Vulnerability detection

☐ Credential testing

☐ Report generation

Figure 132: Filled Post-Survey Forms Part 2.

On a scale of 1 to 5, how effective do you believe NetGuardian is in identifying potential network vulnerabilities? *

☐ 1 (Not effective at all)

☐ 2

☐ 3

☒ 4

☐ 5 (Extremely effective)

How likely are you to recommend NetGuardian to other organizations for improving their network security? *

☐ Very likely

☒ Likely

☐ Neutral

☐ Unlikely

☐ Very unlikely

Figure 133: Filled Post-Survey Forms Part 3.

Did you encounter any technical difficulties or bugs while using NetGuardian? *

☐ Yes

☒ No

How satisfied are you with the overall performance and functionality of NetGuardian? *

☐ Very satisfied

☒ Satisfied

☐ Neutral

☐ Dissatisfied

☐ Very dissatisfied

Figure 134: Filled Post-Survey Forms Part 4.

Would you consider integrating NetGuardian into your organization's regular cybersecurity practices? *

☒ Yes, definitely

☐ Possibly

☐ Unsure

☐ Probably not

☐ No, definitely not

Do you have any additional comments or suggestions for improving the NetGuardian tool?

.....

Figure 135: Filled Post-Survey Forms Part 5.

8.3.3 Post-Survey Result

Firstly, the user-friendliness of NetGuardian's interface received positive feedback, with a majority of respondents (60%) finding it "very user-friendly" and the remaining 40% rating it as "somewhat user-friendly." This suggests that the interface design effectively facilitates usability, which is crucial for ensuring smooth navigation and efficient utilization of the tool.

When asked about the most valuable feature for enhancing network security, respondents highlighted several key functionalities. Port scanning and vulnerability detection emerged as the most appreciated features, each garnering 60% of the responses. This indicates that users highly value NetGuardian's ability to identify potential vulnerabilities within their network infrastructure.

In terms of effectiveness, the majority of respondents (90%) rated NetGuardian positively for identifying potential network vulnerabilities, with 50% considering it "extremely effective" (rated 5 on a scale of 1 to 5). This high rating underscores the tool's efficacy in fulfilling its primary objective of identifying and mitigating security risks within the network.

Furthermore, the likelihood of recommending NetGuardian to other organizations for improving network security was notably high, with 80% of respondents expressing either a high likelihood (30% very likely, 50% likely) or neutrality (10%). This indicates a strong level of confidence in the tool's ability to deliver tangible security benefits.

Technical difficulties or bugs were reported by only 10% of respondents, suggesting that NetGuardian generally operates smoothly and reliably, with minimal issues encountered during usage. Overall satisfaction with NetGuardian's performance and functionality was high, with 90% of respondents either "very satisfied" (30%) or "satisfied" (60%). This positive sentiment reflects the tool's effectiveness in meeting user expectations and addressing their security needs.

Finally, the majority of respondents (90%) expressed a willingness to integrate NetGuardian into their organization's regular cybersecurity practices, with 50% indicating a definite intention to do so and 40% considering it a possibility. This demonstrates a

strong endorsement of NetGuardian as an integral component of cybersecurity measures within organizations.

In conclusion, the post-survey results affirm NetGuardian's effectiveness, user-friendliness, and value in enhancing network security, as well as the high satisfaction levels among users. These findings underscore NetGuardian's significance as a reliable tool for identifying and addressing network vulnerabilities, with strong potential for widespread adoption in organizational cybersecurity practices.

How would you rate the user-friendliness of NetGuardian's interface?

10 responses

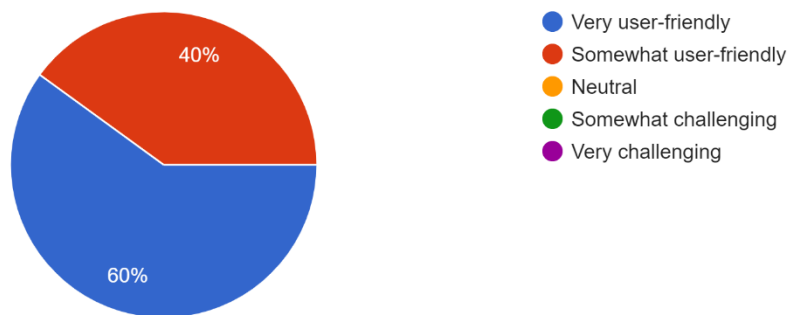


Figure 136: Post-Survey Question 1 Result.

Which feature of NetGuardian did you find most valuable for enhancing network security?

10 responses

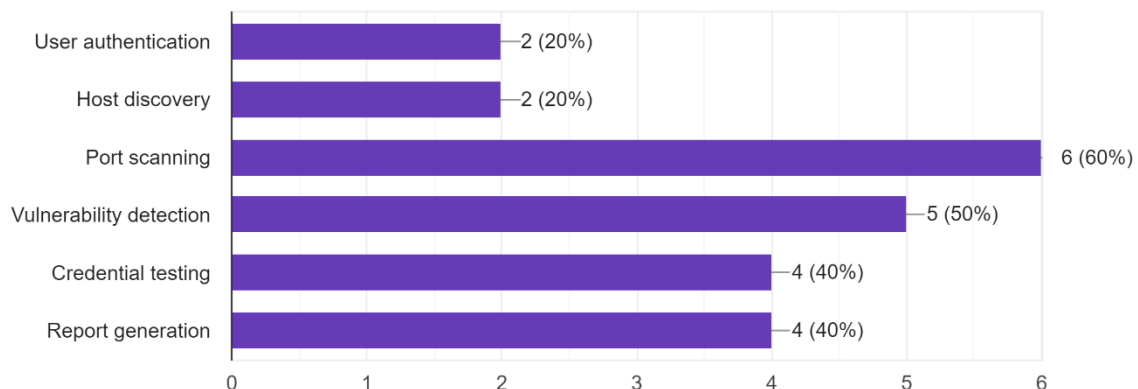


Figure 137: Post-Survey Question 2 Result.

On a scale of 1 to 5, how effective do you believe NetGuardian is in identifying potential network vulnerabilities?

10 responses

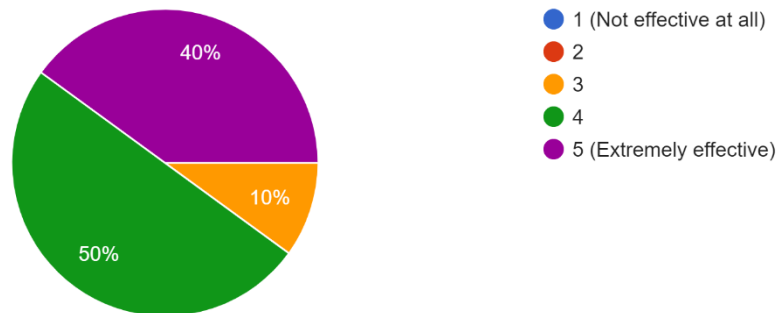


Figure 138: Post-Survey Question 3 Result.

How likely are you to recommend NetGuardian to other organizations for improving their network security?

10 responses

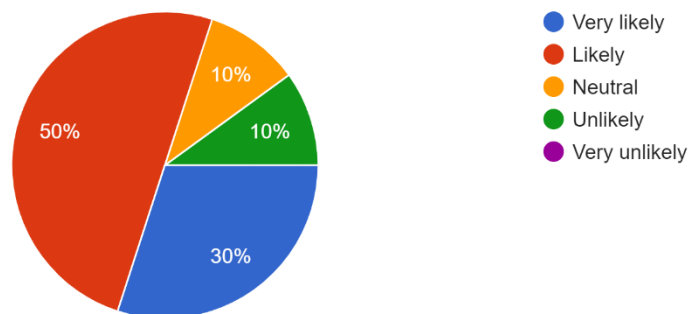


Figure 139: Post-Survey Question 4 Result.

Did you encounter any technical difficulties or bugs while using NetGuardian?

10 responses

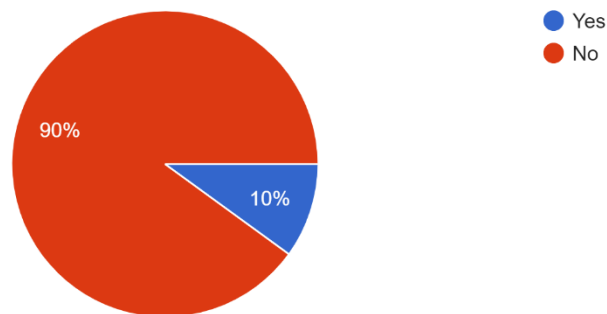


Figure 140: Post-Survey Question 5 Result.

How satisfied are you with the overall performance and functionality of NetGuardian?

10 responses

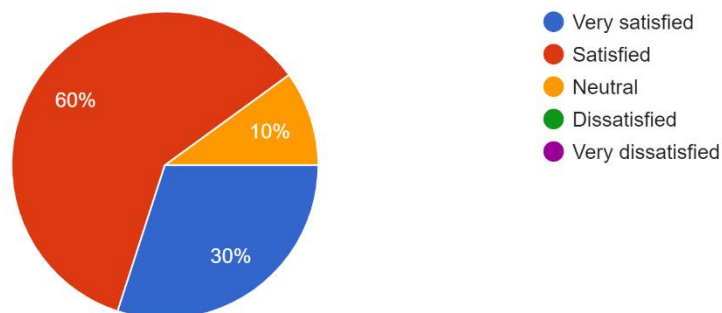


Figure 141: Post-Survey Question 6 Result.

Would you consider integrating NetGuardian into your organization's regular cybersecurity practices?

10 responses

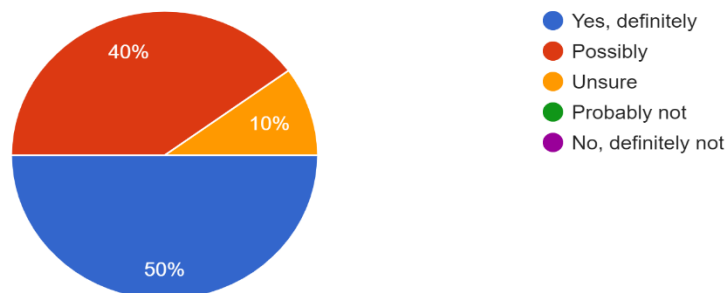


Figure 142: Post-Survey Question 7 Result.

Back to the previous [page](#).

8.4 Appendix D: Sample Codes

```

1
2 import tkinter as tk
3 from tkinter import messagebox
4 import sqlite3
5 import subprocess
6 from reportlab.lib.pagesizes import letter
7 from reportlab.pdfgen import canvas
8 from datetime import datetime
9 import math
10 import requests
11 import os
12 import re
13 import nmap
14 from tkinter import filedialog
15 import ipaddress
16 import socket
17 import concurrent.futures
18 import platform
19 from PIL import Image
20 from io import BytesIO
21 from tkinter import ttk
22
23 #function for login user
24 def login():
25     username = username_entry.get()
26     password = password_entry.get()
27
28     if not username or not password:
29         messagebox.showerror("Error", "Please enter both username and password.")
30         return
31
32     cursor.execute('SELECT * FROM users WHERE username = ? AND password = ?', (username, password))
33     user = cursor.fetchone()
34
35     if user:
36         messagebox.showinfo("Success", "Login successful!")
37         # Clear the username and password entries
38         username_entry.delete(0, tk.END)
39         password_entry.delete(0, tk.END)
40         open_main_window(username)
41     else:
42         messagebox.showerror("Error", "Invalid username or password.")
43
44 #for login
45 def toggle_view_password_login():
46     # Toggle between password visibility and hidden
47     current_state = password_entry["show"]
48     if current_state:
49         password_entry["show"] = ""
50     else:
51         password_entry["show"] = "*"
52
53 # for register
54 def toggle_view_password(entry_widget):
55     # Toggle between password visibility and hidden for the given entry widget
56     current_state = entry_widget["show"]
57     if current_state:
58         entry_widget["show"] = ""
59     else:
60         entry_widget["show"] = "*"

```

Figure 143: Sample Code Part 1.

```

61
62 #function for signup button
63 def open_register_window():
64     register_window = tk.Toplevel(root)
65     register_window.title("Register")
66     register_window.geometry("300x200")
67
68     tk.Label(register_window, text="Username:").pack()
69     new_username_entry = tk.Entry(register_window)
70     new_username_entry.pack()
71
72     tk.Label(register_window, text="Password:").pack()
73     new_password_entry = tk.Entry(register_window, show="*")
74     new_password_entry.pack()
75
76     view_password_checkbox = tk.Checkbutton(register_window, text="View Password", command=lambda: toggle_view_password(new_password_entry))
77     view_password_checkbox.pack()
78
79     def submit_registration():
80         new_username = new_username_entry.get()
81         new_password = new_password_entry.get()
82
83         if not new_username or not new_password:
84             messagebox.showerror("Error", "Please enter both username and password.")
85             return
86
87         # Password policy requirements
88         if not (re.search(r"[A-Z]", new_password) and # At least one uppercase letter
89                 re.search(r"[a-z]", new_password) and # At least one lowercase letter
90                 re.search(r"\d", new_password) and # At least one digit
91                 re.search(r"[!@#$%^&*]", new_password) and # At least one special character
92                 len(new_password) >= 8): # Minimum length of 8 characters
93             messagebox.showerror("Error", "Password must be at least 8 characters long and contain at least one uppercase letter, one lowercase letter, one digit, and one special character.")
94             return
95
96         try:
97             cursor.execute('INSERT INTO users VALUES (?, ?)', (new_username, new_password))
98             conn.commit()
99             messagebox.showinfo("Success", "Registration successful!")
100             register_window.destroy()
101         except sqlite3.IntegrityError:
102             messagebox.showerror("Error", "Username already exists. Please choose a different username.")
103
104     submit_button = tk.Button(register_window, text="Submit", command=submit_registration)
105     submit_button.pack(pady=10)
106
107
108 #validate network address function
109 def validate_network_address(network_address_entry):
110     network_address = network_address_entry.get().strip()
111     if not network_address:
112         messagebox.showerror("Error", "Network address cannot be empty.")
113         return False
114     elif not re.match(r"^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}/\d{1,2}$", network_address):
115         messagebox.showerror("Error", "Invalid network address format. Example: 192.168.1.0/24")
116         return False
117     return True
118

```

Figure 144: Source Code Part 2.

```

108 #validate network address function
109 def validate_network_address(network_address_entry):
110     network_address = network_address_entry.get().strip()
111     if not network_address:
112         messagebox.showerror("Error", "Network address cannot be empty.")
113         return False
114     elif not re.match(r"^\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}/\d{1,2}$", network_address):
115         messagebox.showerror("Error", "Invalid network address format. Example: 192.168.1.0/24")
116         return False
117     return True
118
119
120 # Function to enable the output box temporarily for writing
121 def enable_output_box(output_box):
122     output_box.configure(state='normal')
123
124 # Function to disable the output box after writing
125 def disable_output_box(output_box):
126     output_box.configure(state='disabled')
127

```

Figure 145: Source Code Part 3.

```

128 #Dashboard
129 def open_main_window(username):
130     main_window = tk.Toplevel(root)
131     main_window.title(f'NetGuardian')
132     main_window.geometry("700x500")
133
134     # Create and place labels
135     welcome_label = tk.Label(main_window, text="The Guardian of your Network")
136     login_user = tk.Label(main_window, text=f'MI, {username}')
137     welcome_label.pack(pady=20) # Add vertical gap (20 pixels) below the label
138     login_user.place(x=10, y=12)
139
140     # Left column buttons
141     button_width = 18 # Set the width of all buttons
142
143     # Network Address Label and Entry
144     network_address_label = tk.Label(main_window, text="Network Address:")
145     network_address_label.place(relx=0.34, rely=0.15, anchor='center') # Place at the center
146
147     network_address_entry = tk.Entry(main_window, width=35)
148     network_address_entry.place(relx=0.64, rely=0.15, anchor='center')
149
150     # Output Box
151     output_box = tk.Text(main_window, width=60, height=20, wrap=tk.WORD)
152     output_box.place(relx=0.6, rely=0.55, anchor='center')
153     output_box.configure(state='disabled') # Set the state to disabled
154
155     # buttons
156     host_discovery_button = tk.Button(main_window, text="Host Discovery", command=lambda: host_discovery_wrapper(network_address_entry, output_box), width=button_width)
157     host_discovery_button.place(relx=0.12, rely=0.25, anchor='center')
158
159     port_scanning_button = tk.Button(main_window, text="Port Scanning", command=lambda: port_scanning_wrapper(network_address_entry, output_box), width=button_width)
160     port_scanning_button.place(relx=0.12, rely=0.33, anchor='center')
161
162     find_vulnerability_button = tk.Button(main_window, text="Find Vulnerability", command=lambda: find_vulnerability_wrapper(network_address_entry, output_box), width=button_width)
163     find_vulnerability_button.place(relx=0.12, rely=0.41, anchor='center')
164
165     credential_testing_button = tk.Button(main_window, text="Credential Testing", command=lambda: credential_wrapper(network_address_entry, username), width=button_width)
166     credential_testing_button.place(relx=0.12, rely=0.49, anchor='center')
167
168     report_button = tk.Button(main_window, text="Report", command=report_history, width=button_width)
169     report_button.place(relx=0.12, rely=0.57, anchor='center')
170
171     # Log Out button
172     logout_button = tk.Button(main_window, text="Log Out", command=main_window.destroy, width=button_width)
173     logout_button.place(relx=0.12, rely=0.94, anchor='center')
174
175     # Generate Report button
176     generate_report_button = tk.Button(main_window, text="Generate report", command=lambda: generate_report(network_address_entry, output_box, username), width=18)
177     generate_report_button.place(relx=0.82, rely=0.94, anchor='center')
178
179     main_window.mainloop()
180

```

Figure 146: Source Code Part 4.

```

180
181 # Function to draw the company logo on the PDF canvas
182 def draw_company_logo(pdf_canvas, left_margin, top_margin):
183     try:
184         # Load the company logo image
185         logo_path = "../images/logo.png"
186         logo_image = Image.open(logo_path)
187
188         # Resize the logo if needed
189         max_logo_width = 80
190         max_logo_height = 40
191         logo_image.thumbnail((max_logo_width, max_logo_height))
192
193         # Calculate the position to center the logo in the left section of the header
194         logo_left = left_margin - 40
195         logo_top = top_margin
196
197         # Draw the logo on the PDF canvas
198         pdf_canvas.drawInlineImage(logo_image, logo_left, logo_top, width=logo_image.width, height=logo_image.height)
199     except Exception as e:
200         print(f'Error loading logo: {e}')
201
202
203 def host_discovery(network_address_entry, output_box):
204     # Get the network address from the entry widget
205     network_address = network_address_entry.get()
206     enable_output_box(output_box)
207     # Clear the output box before displaying new results
208     output_box.delete(1.0, tk.END)
209     # Perform host discovery using Nmap
210     output_box.insert(tk.END, f"HOST DISCOVERY \n\n")
211     output_box.insert(tk.END, f"Performing host discovery on network {network_address} ... \n\n")
212
213     try:
214         # Run the Nmap command
215         nmap_command = f'sudo nmap -sT {network_address} -O | awk "/Nmap scan report for/ {{print \'Active host is\'", $NF}} /MAC Address|OS details|Device type/ {{print}}"'
216         result = subprocess.run(nmap_command, shell=True, capture_output=True, text=True)
217
218         # Check if the command was successful
219         if result.returncode == 0:
220             # Display the Nmap output in the output box
221             output_box.insert(tk.END, result.stdout)
222         else:
223             # Display any errors in the output box
224             output_box.insert(tk.END, f'Error: {result.stderr}')
225     except Exception as e:
226         output_box.insert(tk.END, f'Error: {e}\n\n')
227
228     output_box.insert(tk.END, "\nHost discovery completed.")
229
230

```

Figure 147: Source Code Part 5.

```

231
232 def host_discovery_wrapper(network_address_entry, output_box):
233     if validate_network_address(network_address_entry):
234         host_discovery(network_address_entry, output_box)
235         #enable_output_box(output_box)
236         disable_output_box(output_box)
237
238
239 #port scanning function
240 def port_scanning(network_address_entry, output_box):
241     network_address = network_address_entry.get()
242     enable_output_box(output_box)
243     output_box.delete(1.0, tk.END)
244     output_box.insert(tk.END, f"                                PORT SCANNING \n\n")
245     output_box.insert(tk.END, f"Performing port scanning on network {network_address} ... \n\n")
246
247     try:
248         # Run the Nmap command for host discovery
249         nmap_command = f"nmap -sn {network_address}"
250         result = subprocess.run(nmap_command, shell=True, capture_output=True, text=True)
251
252         # Check if the command was successful
253         if result.returncode == 0:
254             # Extract the up hosts from the Nmap output using a regular expression
255             up_hosts = re.findall(r"\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b", result.stdout)
256
257             # Iterate over up hosts and perform port scanning, service, and service version detection
258             for host in up_hosts:
259                 output_box.insert(tk.END, f"\n\nScanning ports for {host} ... \n")
260                 try:
261                     # Run the Nmap command for port scanning, service, and service version detection
262                     port_scan_command = f"nmap -sV {host}"
263                     port_scan_result = subprocess.run(port_scan_command, shell=True, capture_output=True, text=True)
264
265                     # Check if the command was successful
266                     if port_scan_result.returncode == 0:
267                         # Display the Nmap output in the output box
268                         output_box.insert(tk.END, f"Results for {host}:\n")
269                         output_box.insert(tk.END, port_scan_result.stdout)
270                     else:
271                         # Display any errors in the output box
272                         output_box.insert(tk.END, f"Error: {port_scan_result.stderr}")
273                 except Exception as e:
274                     output_box.insert(tk.END, f"Error: {e}\n")
275             else:
276                 # Display any errors in the output box
277                 output_box.insert(tk.END, f"Error: {result.stderr}")
278         except Exception as e:
279             output_box.insert(tk.END, f"Error: {e}\n")
280
281     output_box.insert(tk.END, "\nPort scanning completed.")
282

```

Figure 148: Source Code Part 6.

```

284 def port_scanning_wrapper(network_address_entry, output_box):
285     if validate_network_address(network_address_entry):
286         port_scanning(network_address_entry, output_box)
287         disable_output_box(output_box)
288
289
290 def extract_service_and_version(output):
291     service_version_dict = {}
292     lines = output.split('\n')
293     for line in lines:
294         if 'open' in line: # assuming 'open' indicates a port is open
295             parts = line.split()
296             port = parts[0].replace("/tcp", "") # Remove "\tcp" from the port
297             service = parts[2]
298             version = ' '.join(parts[3:]) if len(parts) > 3 else "Unknown"
299             service_version_dict[port] = {'service': service, 'version': version}
300     return service_version_dict
301
302
303
304 def extract_vuln_info(data, output_box):
305     # Check if 'vulnerabilities' exists in the data
306     if 'vulnerabilities' in data:
307         cve_items = data['vulnerabilities']
308         if not cve_items:
309             output_box.insert(tk.END, f"\n")
310             output_box.insert(tk.END, f"No Vulnerability found!!\n\n")
311         else:
312             for item in cve_items:
313                 cve_id = item['cve']['id']
314                 source = item['cve']['sourceIdentifier']
315                 published = item['cve']['published']
316                 last_modified = item['cve']['lastModified']
317                 vulnStatus = item['cve']['vulnStatus']
318
319                 # Extract descriptions
320                 descriptions = [desc['value'] for desc in item['cve']['descriptions']]
321                 base_severity = [severity['baseSeverity'] for severity in item['cve']['metrics']['cvssMetricV2']]
322                 # Extract base scores
323                 base_scores = []
324                 for metric in item['cve']['metrics']['cvssMetricV2']:
325                     if 'cvssData' in metric:
326                         base_scores.append(metric['cvssData']['baseScore'])
327                     else:
328                         base_scores.append(None) # If 'cvssData' is not present, append None
329
330                 # Print each description and corresponding base scores
331                 for description, base_score, severity in zip(descriptions, base_scores, base_severity):
332                     output_box.insert(tk.END, f"CVE ID : {cve_id}\n")
333                     output_box.insert(tk.END, f"Source Identifier : {source}\n")
334                     output_box.insert(tk.END, f"Published Date : {published}\n")
335                     output_box.insert(tk.END, f>Last Modified Date : {last_modified}\n")
336                     output_box.insert(tk.END, f"Vulnerability Status : {vulnStatus}\n")
337                     output_box.insert(tk.END, f"Vulnerability Description : {description}\n")
338                     output_box.insert(tk.END, f"Score : {base_score}\n")
339                     output_box.insert(tk.END, f"Severity: {severity}\n\n")
340             else:
341                 output_box.insert(tk.END, f"No vulnerabilities information found")
342
343

```

Figure 149: Source Code Part 7.

```

345 def check_vulnerabilities(keyword, output_box):
346     base_url = "https://services.nvd.nist.gov/rest/json/cves/2.0"
347     params = {"keywordSearch": keyword}
348
349     try:
350         response = requests.get(base_url, params=params)
351         if response.status_code == 200:
352             data = response.json()
353             #output_box.insert(tk.END, f"{data}\n\n")
354             extract_vuln_info(data, output_box)
355             output_box.insert(tk.END, f"For more information: https://services.nvd.nist.gov/rest/json/cves/2.0?keywordSearch={keyword}\n\n\n")
356         else:
357             output_box.insert(tk.END, f"Error accessing NVD API: {response.status_code}\n")
358             return False
359     except requests.RequestException as e:
360         output_box.insert(tk.END, f"An error occurred during the request: {e}\n")
361         return False
362

```

Figure 150: Source Code Part 8.

```

363
364 def find_vulnerability(network_address_entry, output_box):
365     network_address = network_address_entry.get()
366     enable_output_box(output_box)
367     # Clear the output box before displaying new results
368     output_box.delete(1.0, tk.END)
369     output_box.insert(tk.END, "Vulnerability Scanning \n\n")
370     output_box.insert(tk.END, f"Performing scanning on network {network_address} ... \n\n")
371
372     try:
373         # Run the Nmap command for host discovery
374         nmap_command = f"nmap -sn {network_address}"
375         result = subprocess.run(nmap_command, shell=True, capture_output=True, text=True, check=False)
376
377         # Check if the command was successful
378         if result.returncode == 0:
379             # Extract the up hosts from the Nmap output using a regular expression
380             up_hosts = re.findall(r"\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b", result.stdout)
381
382             # Iterate over up hosts and perform port scanning, service, and service version detection
383             for host in up_hosts:
384                 output_box.insert(tk.END, f"\n\nScanning ports for {host} ... \n\n")
385                 try:
386                     # Run the Nmap command for port scanning, service, and service version detection
387                     scanner = nmap.PortScanner()
388                     scanner.scan(host, arguments='-p 1-1000 -sV') # Scan ports 1 to 1000
389                     result = "" # Initialize result for each host
390                     for host in scanner.all_hosts():
391                         for proto in scanner[host].all_protocols():
392                             for port in scanner[host][proto]:
393                                 port_info = scanner[host][proto][port]
394                                 result += f"{port}/{proto} {port_info['state']} {port_info['name']} {port_info['product']} {port_info['version']}\n\n"
395
396                     service_version_info = extract_service_and_version(result)
397                     for port, info in service_version_info.items():
398                         output_box.insert(tk.END, f"Port: {port}, Service: {info['service']}, Version: {info['version']}\n\n")
399                         # Check for vulnerabilities
400                         if info['version'] == 'Unknown':
401                             check_vulnerabilities(info['service'], output_box)
402                         else:
403                             check_vulnerabilities(info['version'], output_box)
404
405                 except Exception as e:
406                     output_box.insert(tk.END, f"Error: {e}\n\n")
407             else:
408                 # Display any errors in the output box
409                 output_box.insert(tk.END, f"Error: {result.stderr}")
410         except Exception as e:
411             output_box.insert(tk.END, f"Error: {e}\n\n")
412         output_box.insert(tk.END, "\n\nScanning completed.")
413
414

```

Figure 151: Source Code Part 9.

```

415 def find_vulnerability_wrapper(network_address_entry, output_box):
416     if validate_network_address(network_address_entry):
417         find_vulnerability(network_address_entry, output_box)
418         disable_output_box(output_box)
419
420
421 def scan_network(network_address_entry, username):
422     nm = nmap.PortScanner()
423     network_address = network_address_entry.get()
424     nm.scan(hosts=network_address, arguments='-sn')
425     active_hosts = nm.all_hosts()
426
427     #ip_dropdown['values'] = active_hosts
428     open_credential_window(network_address_entry, active_hosts, username)
429
430 def scan_ports(ip_dropdown, port_dropdown, service_text):
431     selected_ip = ip_dropdown.get()
432     nm = nmap.PortScanner()
433     nm.scan(hosts=selected_ip, arguments='-p 1-1000')
434
435     open_ports = []
436     for host in nm.all_hosts():
437         for proto in nm[host].all_protocols():
438             ports = nm[host][proto].keys()
439             open_ports.extend(ports)
440
441     port_dropdown['values'] = sorted(open_ports)
442
443     service_text.delete(1.0, tk.END) # Clear previous text
444
445 def show_services(ip_dropdown, port_dropdown, service_text):
446     selected_ip = ip_dropdown.get()
447     selected_port = port_dropdown.get()
448
449     nm = nmap.PortScanner()
450     nm.scan(hosts=selected_ip, arguments='-sV -p ' + selected_port)
451
452     service_info = nm[selected_ip]['tcp'][int(selected_port)]
453     service_name = service_info['name']
454     service_text.delete(1.0, tk.END) # Clear previous text
455     service_text.insert(tk.END, service_name)

```

Figure 152: Source Code Part 10.

```

457
458 def brute_force_attack(ip_dropdown, port_dropdown, output_box, service_text):
459     selected_ip = ip_dropdown.get()
460     selected_port = port_dropdown.get()
461     enable_output_box(output_box)
462     # Clear the output box before displaying new results
463     output_box.delete(1.0, tk.END)
464     # Retrieve the text from the service_text widget
465     selected_service = service_text.get("1.0", "end-1c") # Get all text excluding the trailing newline character
466
467     usernames_list = "../credential/usernames.txt" # Replace with the actual username
468     password_list = "../credential/passwords.txt" # Replace with the path to your password list
469
470     # Execute hydra command for SSH brute force attack
471     command = f"hydra -L {usernames_list} -P {password_list} {selected_ip} -s {selected_port} {selected_service}"
472     output_box.insert(tk.END, f"Performing brute force attack on {selected_ip} and the port is {selected_port} ... \n\n")
473     try:
474         result = subprocess.run(command, shell=True, capture_output=True, text=True)
475         if result.returncode == 0:
476             output_box.insert(tk.END, "Brute force attack successful.\n\n")
477
478             # Extract and display usernames and passwords
479             matches = re.findall(r"login:\s+(\S+)\s+password:\s+(\S+)", result.stdout)
480             if matches:
481                 output_box.insert(tk.END, "Found credentials:\n")
482                 for match in matches:
483                     brute_username, password = match
484                     output_box.insert(tk.END, f"Username: {brute_username}, Password: {password}\n")
485             else:
486                 output_box.insert(tk.END, "No credentials found.\n")
487         else:
488             output_box.insert(tk.END, f"Error: {result.stderr}\n")
489     except Exception as e:
490         output_box.insert(tk.END, f"Error: {e}\n")
491
492     output_box.insert(tk.END, "Brute force attack completed.\n")
493     disable_output_box(output_box)
494

```

Figure 153: Source Code Part 11.

```

494
495 def open_credential_window(network_address_entry, active_hosts, username):
496     credential_window = tk.Toplevel(root)
497     credential_window.title("Credential Testing")
498     credential_window.geometry("480x480")
499
500     #set for ip dropdown and label
501     ip_dropdown = ttk.Combobox(credential_window, width = 13, state="readonly")
502     #ip_dropdown.set(active_hosts[0]) # Set default active host
503     ip_dropdown.place(relx=0.27, rely=0.1)
504     ip_label = tk.Label(credential_window, text="Active Hosts:")
505     ip_label.place(relx=0.02, rely=0.10)
506     ip_dropdown.bind("<<ComboboxSelected>>", lambda event: scan_ports(ip_dropdown, port_dropdown, service_text))
507     ip_dropdown['values'] = active_hosts
508
509
510     #setting for port dropdown and label
511     port_dropdown = ttk.Combobox(credential_window, width = 9, state="readonly") # Set the state to readonly
512     port_dropdown.set("") # Initially empty until host selected
513     port_dropdown.place(relx=0.7, rely=0.1)
514     port_label = tk.Label(credential_window, text="Port:")
515     port_label.place(relx=0.6, rely=0.1)
516     port_dropdown.bind("<<ComboboxSelected>>", lambda event: show_services(ip_dropdown, port_dropdown, service_text))
517
518
519     #setting for service label
520     service_label = tk.Label(credential_window, text="service:")
521     service_label.place(relx=0.02, rely=0.2)
522     service_text = tk.Text(credential_window, height=1, width=14, state="normal")
523     service_text.place(relx=0.27, rely=0.2)
524     #service_text.configure(state="disable")
525
526     # Output Box
527     output_box = tk.Text(credential_window, width=40, height=13, wrap=tk.WORD)
528     output_box.place(relx=0.5, rely=0.58, anchor='center')
529     output_box.configure(state='disabled') # Set the state to disabled
530
531     # Bruteforce button
532     brute_force = tk.Button(credential_window, text="Bruteforce", command=lambda: brute_force_attack(ip_dropdown, port_dropdown, output_box, service_text), width=12)
533     brute_force.place(relx=0.77, rely=0.22, anchor='center')
534
535     # Generate report button
536     credential_report_button = tk.Button(credential_window, text="Generate report", command=lambda: generate_report(network_address_entry, output_box, username), width=12)
537     credential_report_button.place(relx=0.78, rely=0.94, anchor='center')
538
539     # back button
540     back_button = tk.Button(credential_window, text="Back", command=credential_window.destroy, width=12)
541     back_button.place(relx=0.2, rely=0.94, anchor='center')
542

```

Figure 154: Source Code Part 12.

```

542
543
544 def credential_wrapper(network_address_entry, username):
545     if validate_network_address(network_address_entry):
546         scan_network(network_address_entry, username)
547         #disable_output_box(output_box)
548
549
550 def generate_report(network_address_entry, output_box, username):
551     # Get the current date
552     current_date = datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
553
554     # Get the network address and output text
555     network_address = network_address_entry.get().replace('/', '-')
556     report_text = output_box.get("1.0", tk.END)
557
558     # Check if the output box is empty
559     if not report_text.strip():
560         messagebox.showerror("Error", "Output box is empty. Please perform actions before generating a report.")
561         return
562     report_folder = '../reports'
563
564
565     try:
566         # List all files in the reports folder
567         pdf_files = [f for f in os.listdir(report_folder) if f.endswith(".pdf")]
568         # Prompt the user for the file name and location
569         file_path = filedialog.asksaveasfilename(
570             initialdir=report_folder,
571             defaultextension=".pdf",
572             filetypes=[("PDF files", "*.pdf")],
573             title="Save Report As"
574         )
575
576         if file_path:
577             # Create the PDF document
578             pdf_canvas = canvas.Canvas(file_path, pagesize=letter)
579
580             # Set margins
581             left_margin = 72
582             right_margin = letter[0] - 72
583             top_margin = letter[1] - 72
584             bottom_margin = 72
585
586             # Add metadata to the PDF
587             pdf_canvas.setTitle(f"Report - {network_address}")
588
589             # Add header only on the first page
590             pdf_canvas.setFont("Helvetica-Bold", 14)
591
592             # Draw company logo in the left section of the header
593             draw_company_logo(pdf_canvas, left_margin, top_margin)
594
595             # Continue with the rest of the header
596             pdf_canvas.drawCentredString((left_margin + right_margin) / 2, top_margin - 20, "Report")
597             top_margin -= 40
598
599             pdf_canvas.setFont("Helvetica", 12)
600

```

Figure 155: Source Code Part 13.

```

601 # Add username, network address, and date only on the first page
602 pdf_canvas.drawString(left_margin, top_margin, f"User: {username}")
603 top_margin -= 15
604 pdf_canvas.drawString(left_margin, top_margin, f"Network Address: {network_address}")
605 top_margin -= 15
606 pdf_canvas.drawString(left_margin, top_margin, f"Date: {current_date}")
607 top_margin -= 20
608
609 # Add page number in the right section of the footer on the first page
610 pdf_canvas.drawRightString(right_margin, bottom_margin - 20, "Page 1")
611
612 # Split the report text into lines and add them to the PDF
613 lines = report_text.split("\n")
614
615 line_spacing = 1.5 # Adjust this value for the desired line spacing
616
617 page_number = 1
618 for line in lines:
619     # Check if there's enough space for the current line
620     if top_margin - bottom_margin <= 0:
621         pdf_canvas.showPage() # Start a new page
622         page_number += 1
623         top_margin = letter[1] - 25 # Reset the position for the new page
624
625     # Add metadata to the PDF for subsequent pages
626     pdf_canvas.setTitle(f"Report - {network_address}")
627
628     # Add header for the new page
629     pdf_canvas.setFont("Helvetica-Bold", 14)
630     top_margin -= 40
631
632     pdf_canvas.setFont("Helvetica", 12)
633
634     # Add page number in the right section of the footer for subsequent pages
635     pdf_canvas.drawRightString(right_margin, bottom_margin - 20, f"Page {page_number}")
636
637     # Calculate the width of the line
638     line_width = pdf_canvas.stringWidth(line.strip(), "Helvetica", 12)
639     # Check if the line exceeds the right margin
640     if left_margin + line_width > right_margin:
641         # Split the line into words
642         words = line.split()
643         wrapped_line = ""
644         for word in words:
645             # Check if adding the word exceeds the right margin
646             if pdf_canvas.stringWidth(wrapped_line + word, "Helvetica", 12) <= right_margin - left_margin:
647                 wrapped_line += word + " "
648             else:
649                 # Draw the wrapped line
650                 pdf_canvas.drawString(left_margin, top_margin, wrapped_line.strip())
651                 top_margin -= 12 * line_spacing # Adjusting line spacing
652                 wrapped_line = word + " "
653         # Draw the remaining wrapped line
654         pdf_canvas.drawString(left_margin, top_margin, wrapped_line.strip())
655         top_margin -= 12 * line_spacing # Adjusting line spacing
656     else:
657         pdf_canvas.drawString(left_margin, top_margin, line.strip())
658         top_margin -= 12 * line_spacing # Adjusting line spacing
659
660 pdf_canvas.save()
661
662 messagebox.showInfo("Report", f"Report generated successfully: {file_path}")
663
664 except Exception as e:
665     messagebox.showError("Error", f"Failed to generate report: {e}")
666

```

Figure 156: Source Code Part 14.

```

670 def report_history():
671     report_folder = r"../reports"
672     try:
673         # List all files in the reports folder
674         pdf_files = [f for f in os.listdir(report_folder) if f.endswith(".pdf")]
675
676         if pdf_files:
677             file_path = filedialog.askopenfilename(
678                 initialdir=report_folder,
679                 title="Select a PDF file",
680                 filetypes=[("PDF files", "*.pdf")],
681                 defaulttextextension=".pdf"
682             )
683
684             if file_path:
685                 # Determine the platform (Windows or Unix-like)
686                 if platform.system().lower() == 'windows':
687                     # On Windows, use the "start" command
688                     subprocess.run(["start", "", file_path], shell=True)
689                 else:
690                     # On Unix-like systems, use the "xdg-open" command
691                     subprocess.run(["xdg-open", file_path])
692             else:
693                 messagebox.showInfo("No File Selected", "No PDF file selected.")
694         else:
695             messagebox.showInfo("No PDF Files", f"No PDF files found in {report_folder}")
696
697     except Exception as e:
698         messagebox.showError("Error", f"Failed to list report files: {e}")
699

```

Figure 157: Source Code Part 15.

```

700
701 root = tk.Tk()
702 root.title("Login System")
703 root.geometry("400x300")
704 #root.iconbitmap(r"../images/icon.ico")
705
706
707 conn = sqlite3.connect('../database/user_credentials.db')
708 cursor = conn.cursor()
709
710 cursor.execute('''
711     CREATE TABLE IF NOT EXISTS users (
712         username TEXT PRIMARY KEY,
713         password TEXT NOT NULL
714     )
715 ''')
716 conn.commit()
717
718 tk.Label(root, text="Username:").pack()
719 username_entry = tk.Entry(root)
720 username_entry.pack()
721
722 tk.Label(root, text="Password:").pack()
723 password_entry = tk.Entry(root, show="*")
724 password_entry.pack()
725
726 view_password_checkbox = tk.Checkbutton(root, text="View Password", command=toggle_view_password_login)
727 view_password_checkbox.pack()
728
729 # Left column buttons
730 button_width = 6 # Set the width of all buttons
731
732 login_button = tk.Button(root, text="Log In", command=login, font=("Arial", 12), activebackground="#45a049", width=button_width)
733 login_button.place(relx=0.5, rely=0.45, anchor='center')
734
735 register_button = tk.Button(root, text="Sign Up", command=open_register_window, font=("Arial", 12), activebackground="#45a049", width=button_width)
736 register_button.place(relx=0.5, rely=0.60, anchor='center')
737
738 root.mainloop()
739
740 conn.close()
741

```

Figure 158: Source Code Part 16.

8.5 Appendix E: Design

8.5.1 Sequence Diagram

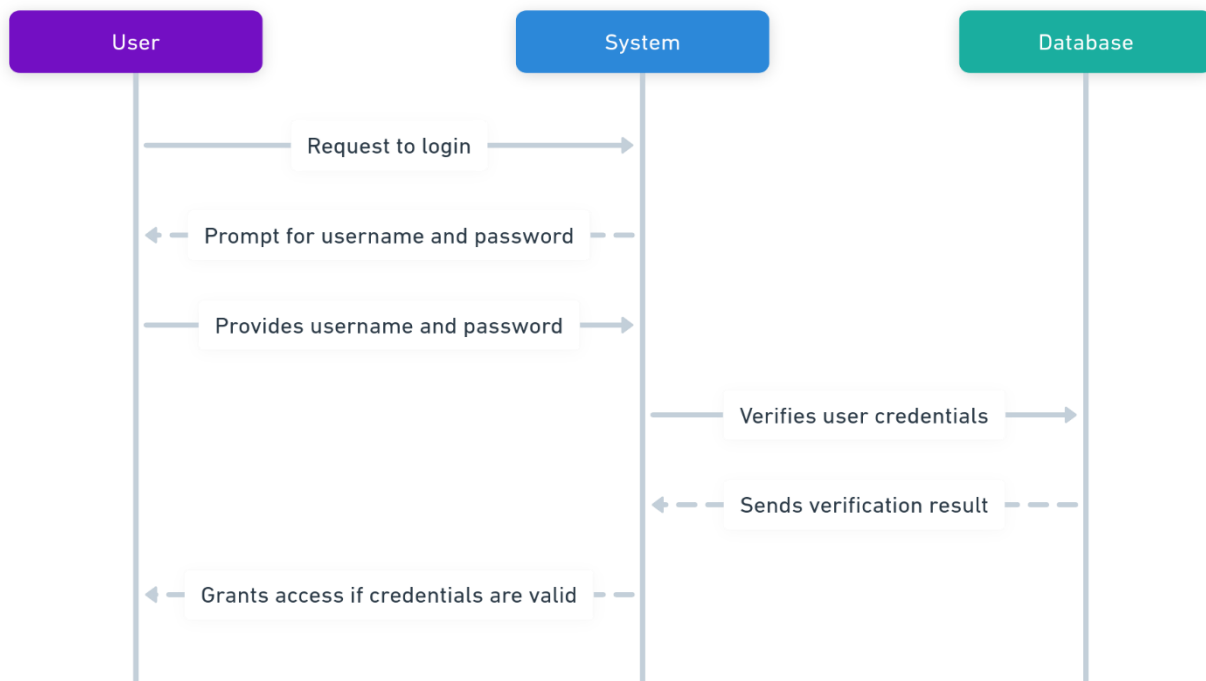


Figure 159: Sequence Diagram of Login Button.

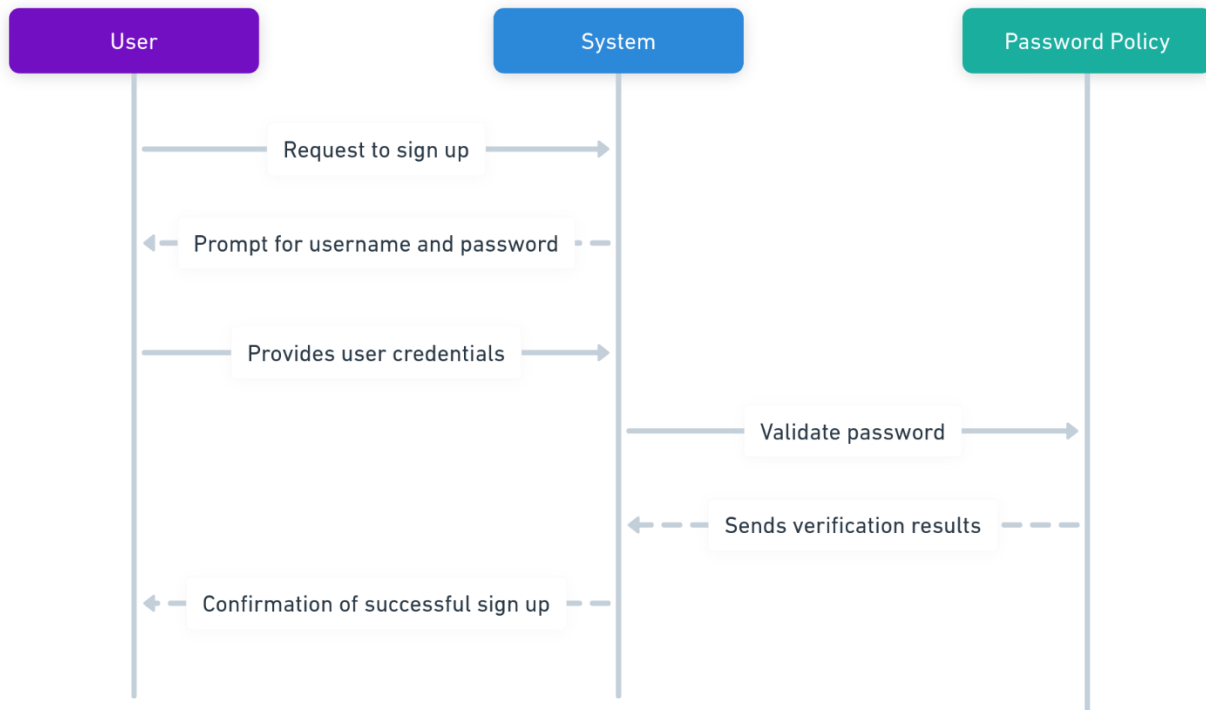


Figure 160: Sequence Diagram of Sign Up Button.

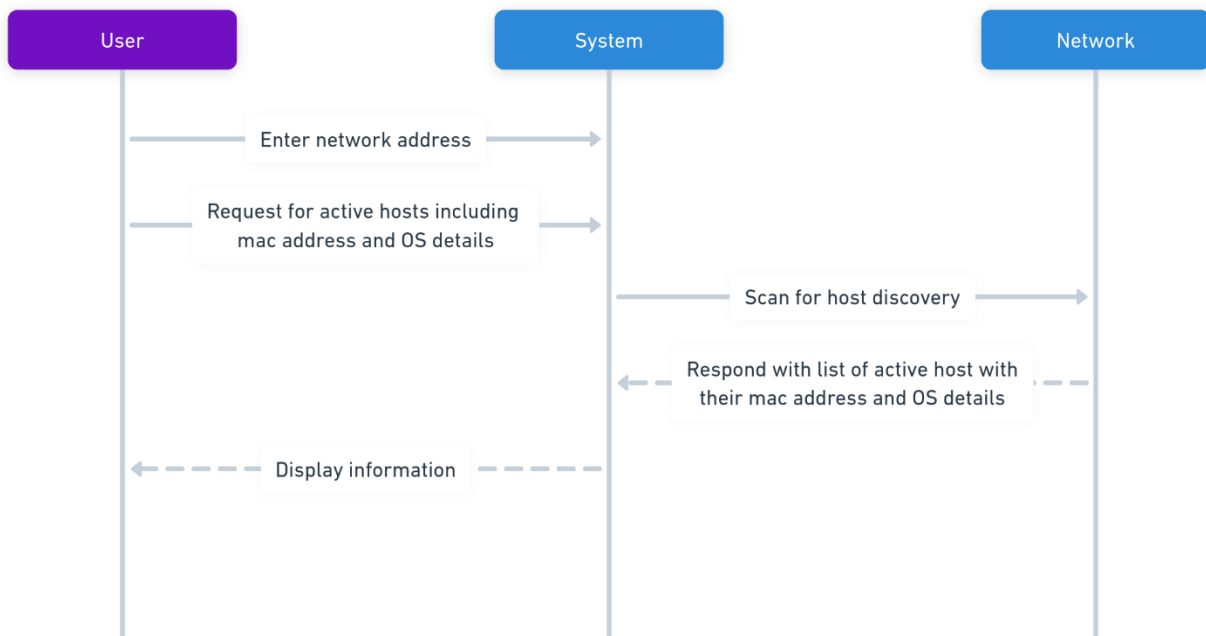


Figure 161: Sequence Diagram of Host Discovery.

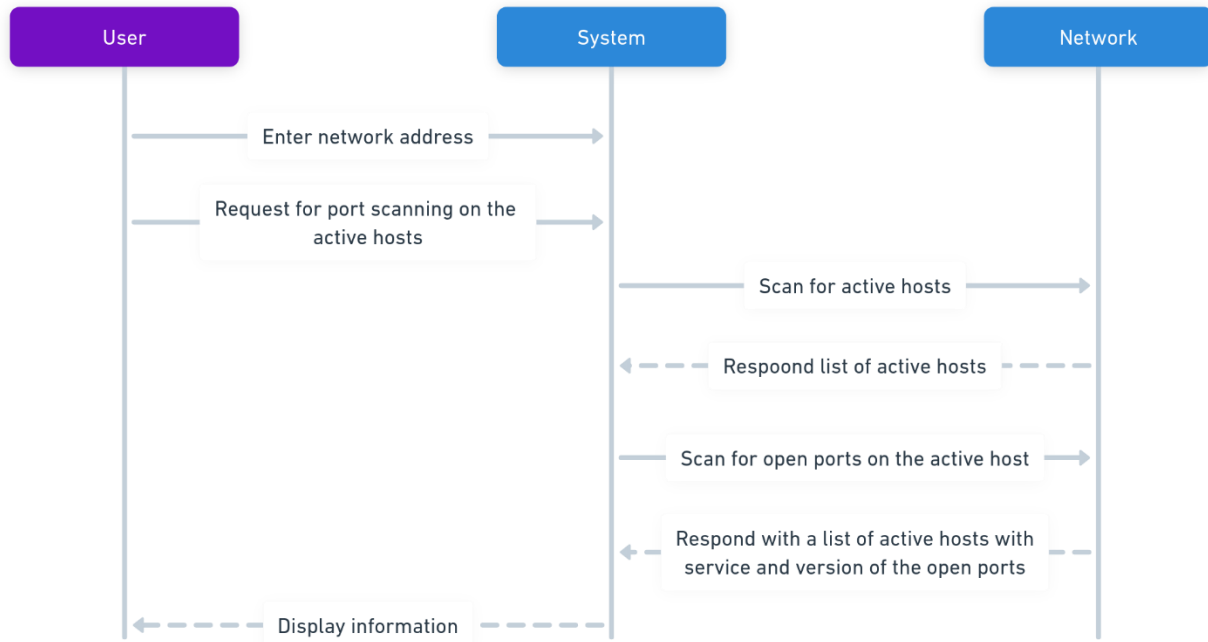


Figure 162: Sequence Diagram of Port Scanning Button.

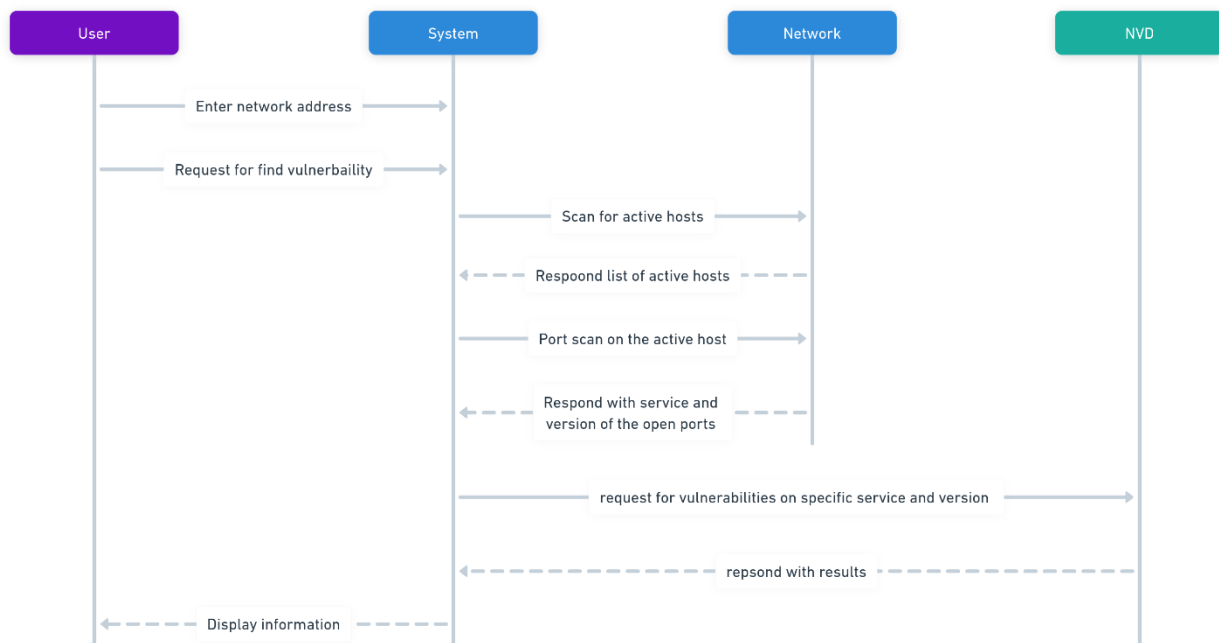


Figure 163: Sequence Diagram of Find Vulnerability Button.

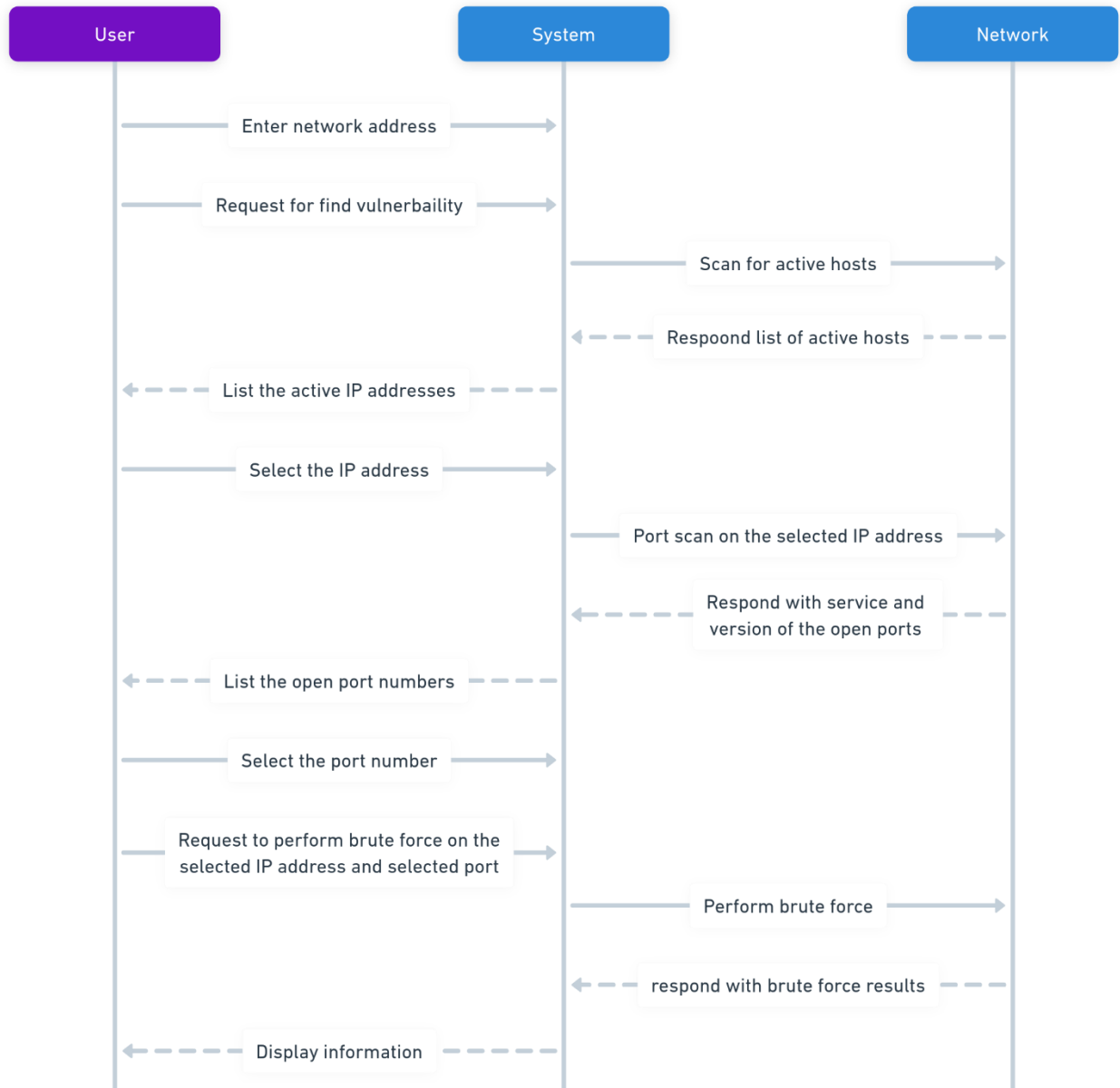


Figure 164: Sequence Diagram of Credential Testing Button.

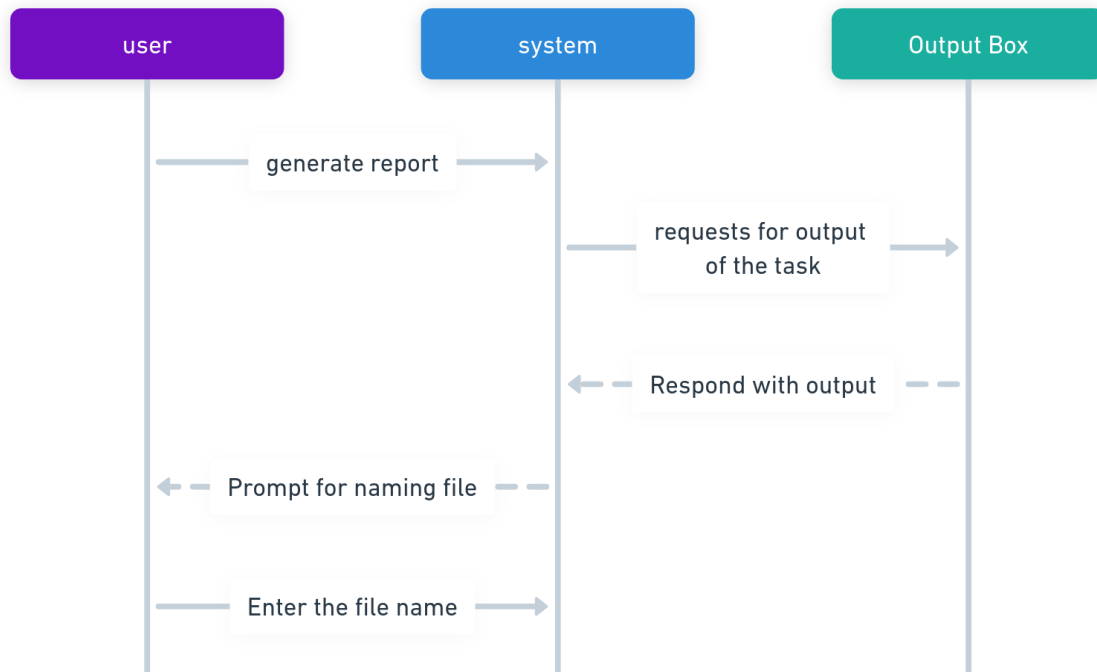


Figure 165: Sequence Diagram of Generate Report Button.

8.5.2 Work Breakdown Structure (WBS)

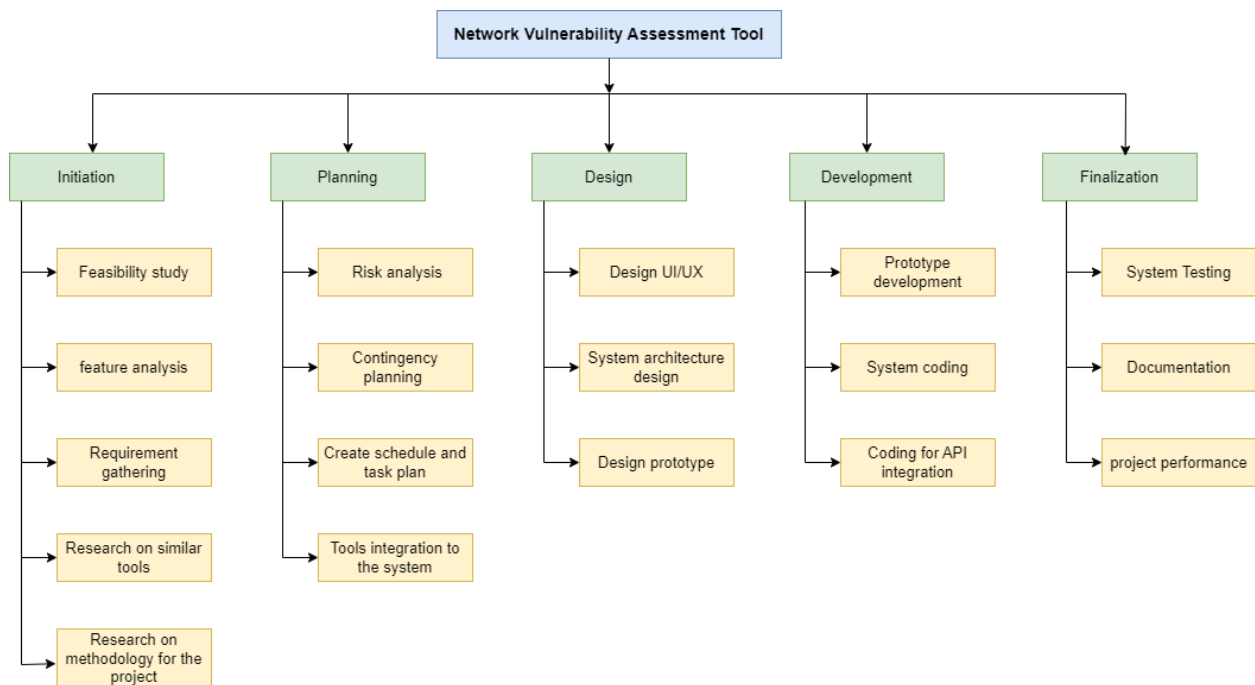


Figure 166: Work Breakdown Structure of the Project.

8.5.3 Gantt Chart

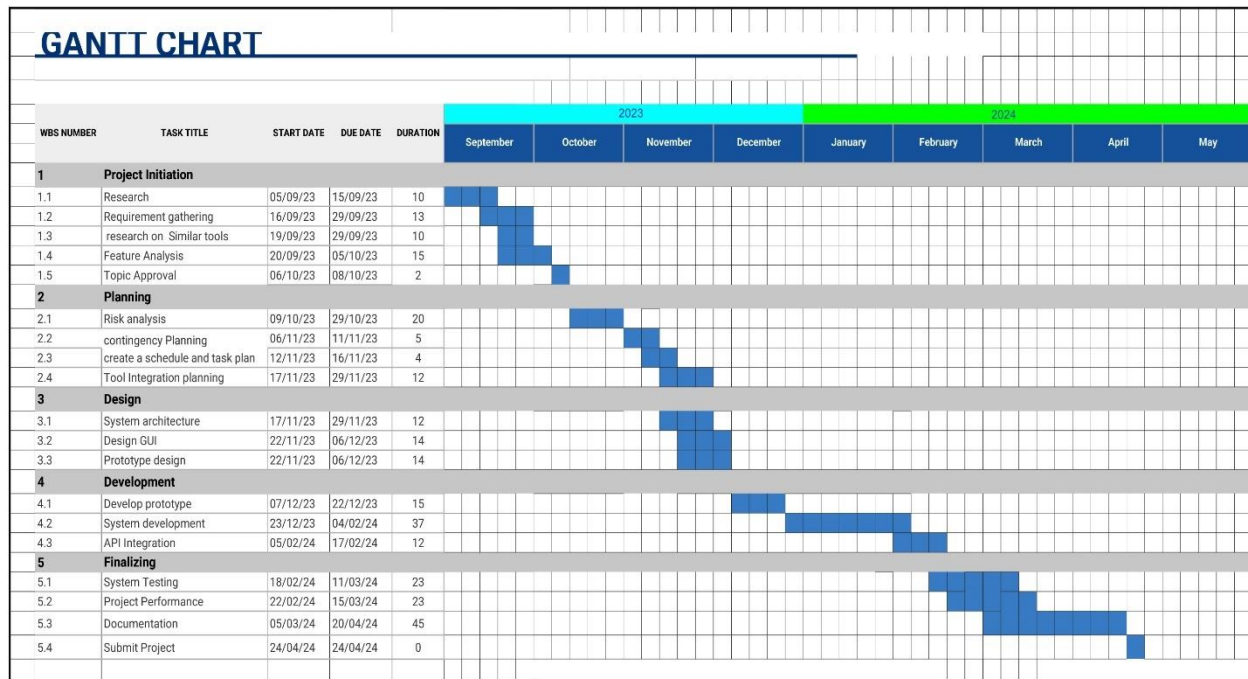


Figure 167: Gantt Chart for the Project.

Back to the previous [page](#).

8.6 Appendix F: Screenshots of the System

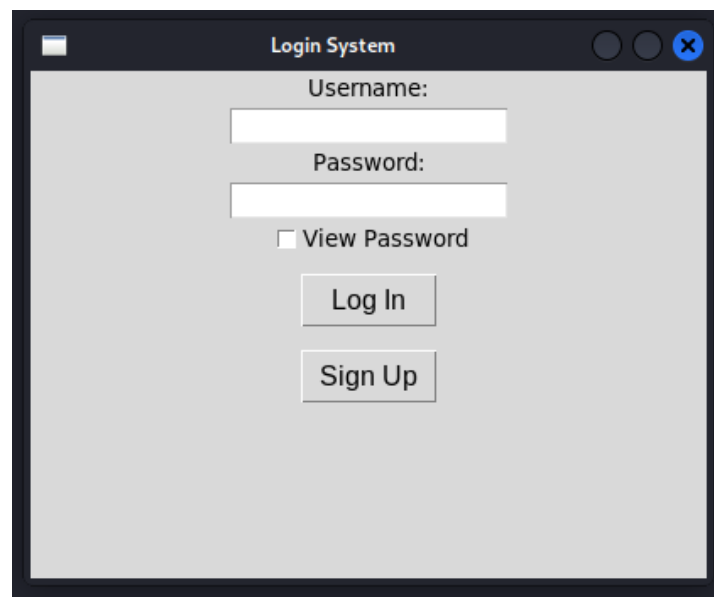
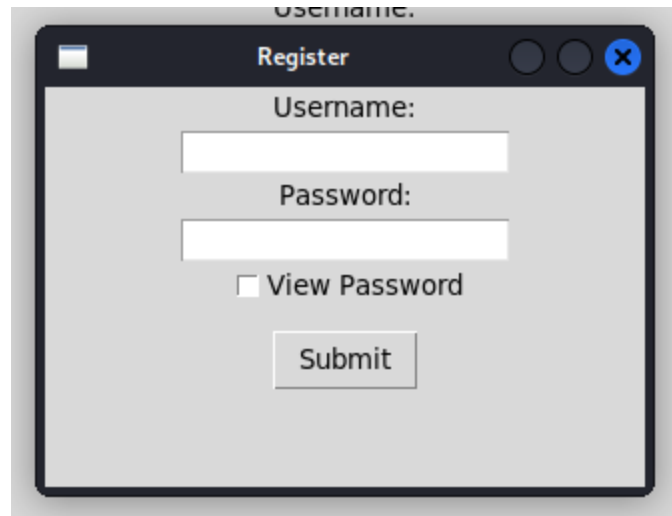
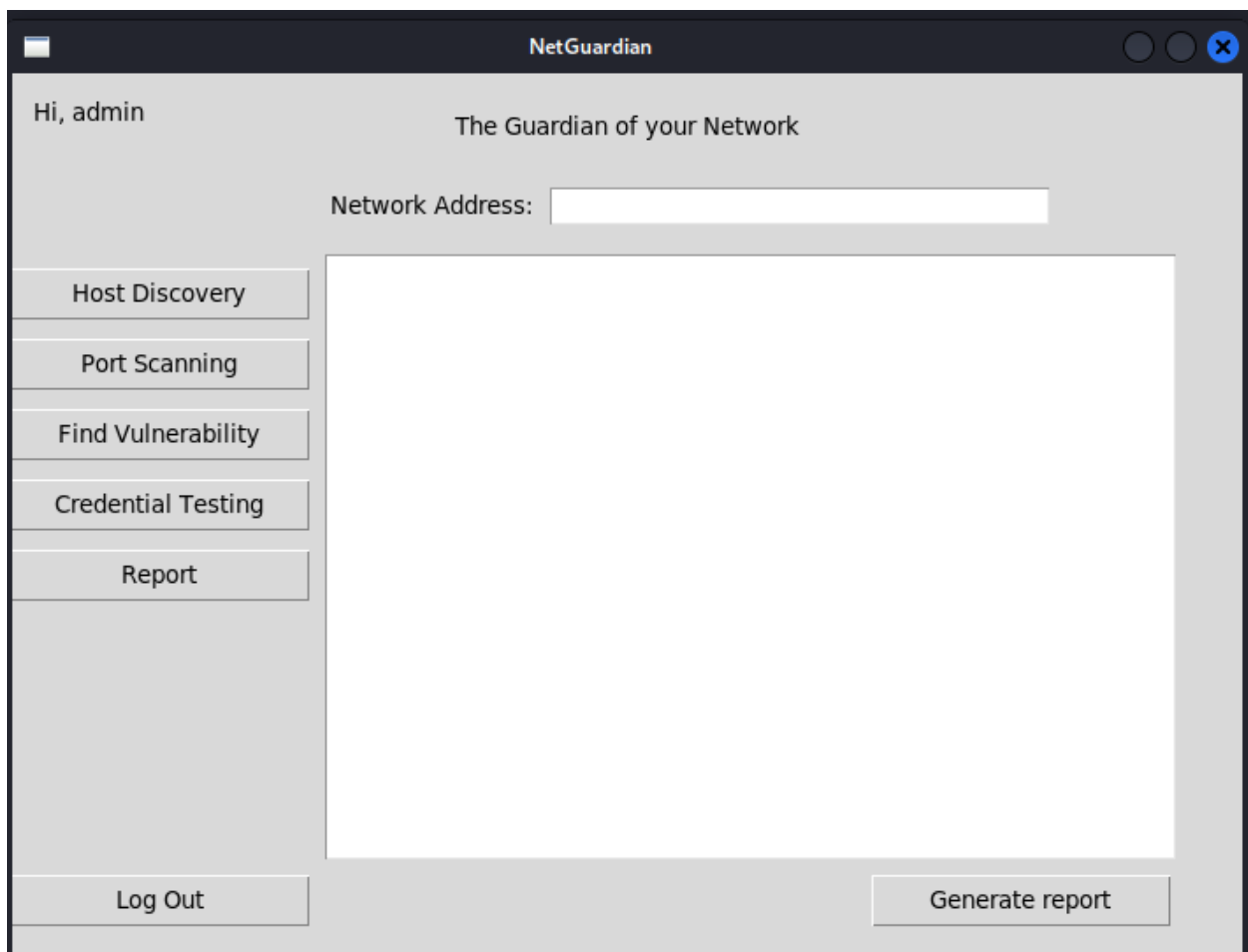


Figure 168: Login Window.



A screenshot of a 'Register' window. The window has a title bar with the text 'Register' and standard window controls. Inside, there is a 'Username:' label followed by a text input field. Below that is a 'Password:' label followed by another text input field. Under the password field is a checkbox labeled 'View Password'. At the bottom center is a 'Submit' button.

Figure 169: Sign Up Window.



A screenshot of the 'NetGuardian' user dashboard. The window title is 'NetGuardian'. The main area displays 'Hi, admin' and 'The Guardian of your Network'. Below this is a 'Network Address:' label with a text input field. On the left side, there is a vertical menu with buttons: 'Host Discovery', 'Port Scanning', 'Find Vulnerability', 'Credential Testing', 'Report', and 'Log Out'. The main content area is a large empty box. At the bottom right, there is a 'Generate report' button.

Figure 170: User Dashboard.

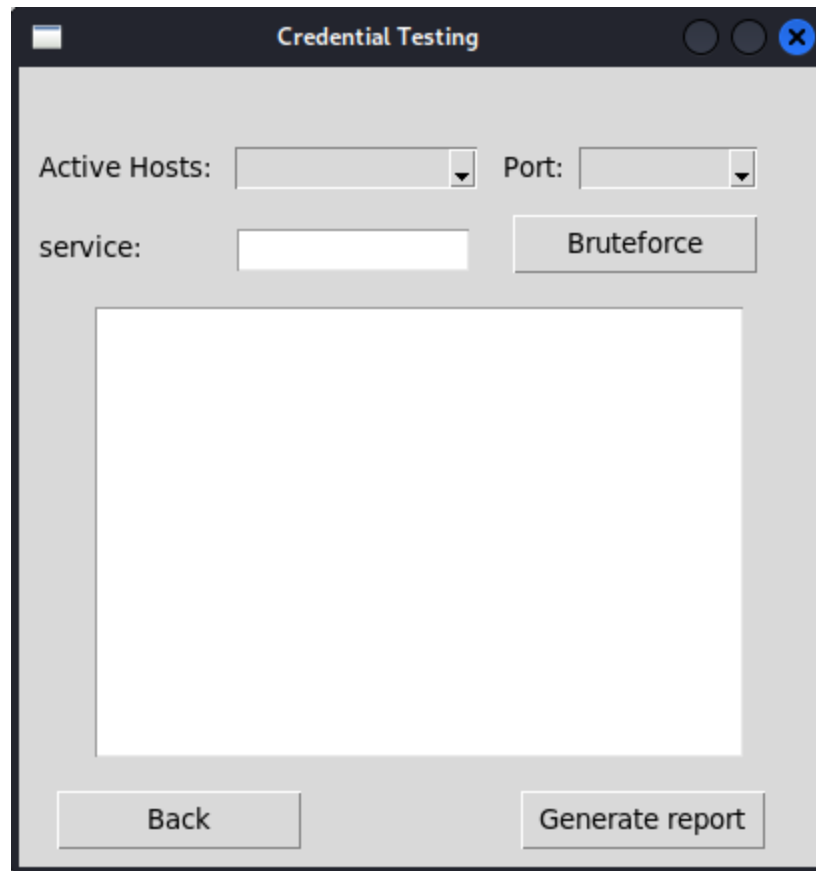


Figure 171: Credential Testing Window.

8.7 Appendix G: User Feedback

8.7.1 User Feedback Form

User Feedback Form

B *I* U  

Thank you for using NetGuardian! Your feedback is invaluable to us as we strive to enhance our network vulnerability assessment tool. Please take a moment to provide us with your insights.

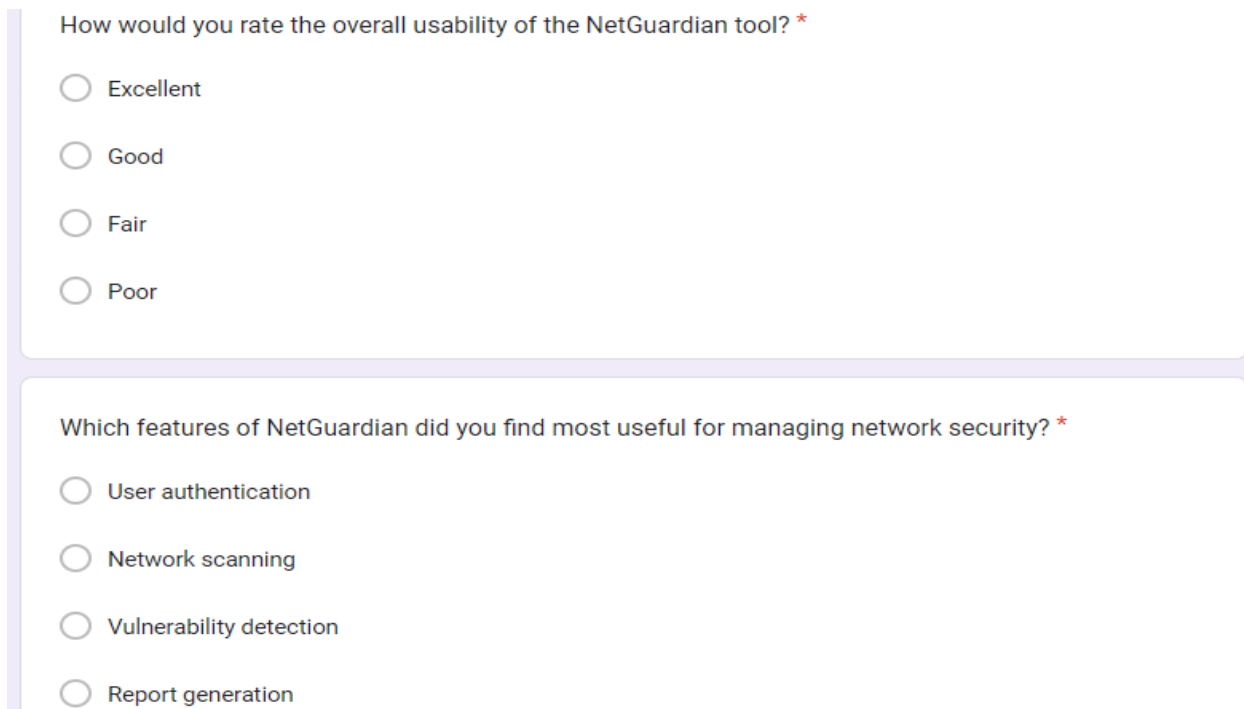
Name: *

Short-answer text

Email: *

Short-answer text

Figure 172: User Feedback Form Part 1.



How would you rate the overall usability of the NetGuardian tool? *

☐ Excellent

☐ Good

☐ Fair

☐ Poor

Which features of NetGuardian did you find most useful for managing network security? *

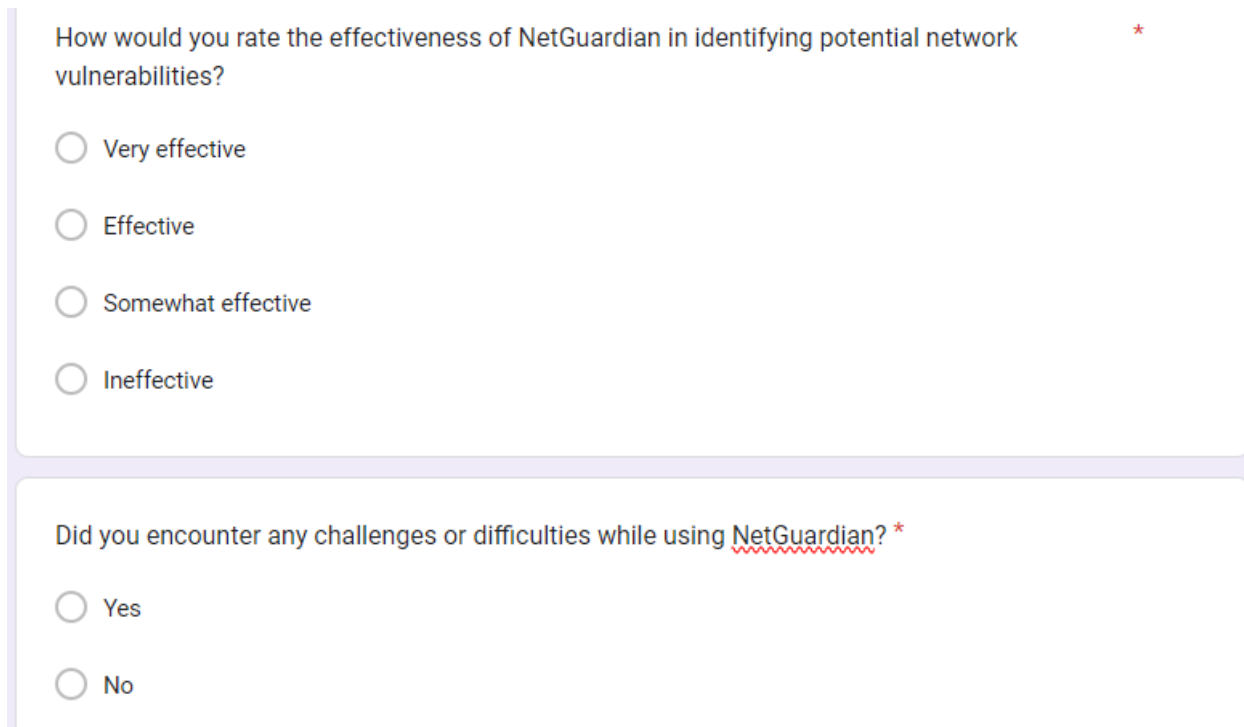
☐ User authentication

☐ Network scanning

☐ Vulnerability detection

☐ Report generation

Figure 173: User Feedback Form Part 2.



How would you rate the effectiveness of NetGuardian in identifying potential network vulnerabilities? *

☐ Very effective

☐ Effective

☐ Somewhat effective

☐ Ineffective

Did you encounter any challenges or difficulties while using NetGuardian? *

☐ Yes

☐ No

Figure 174: User Feedback Form Part 3.

How satisfied are you with the level of support and assistance provided during your use of NetGuardian

☐ Very satisfied

☐ Satisfied

☐ Neutral

☐ Dissatisfied

★

Would you recommend NetGuardian to other small businesses for managing network security?

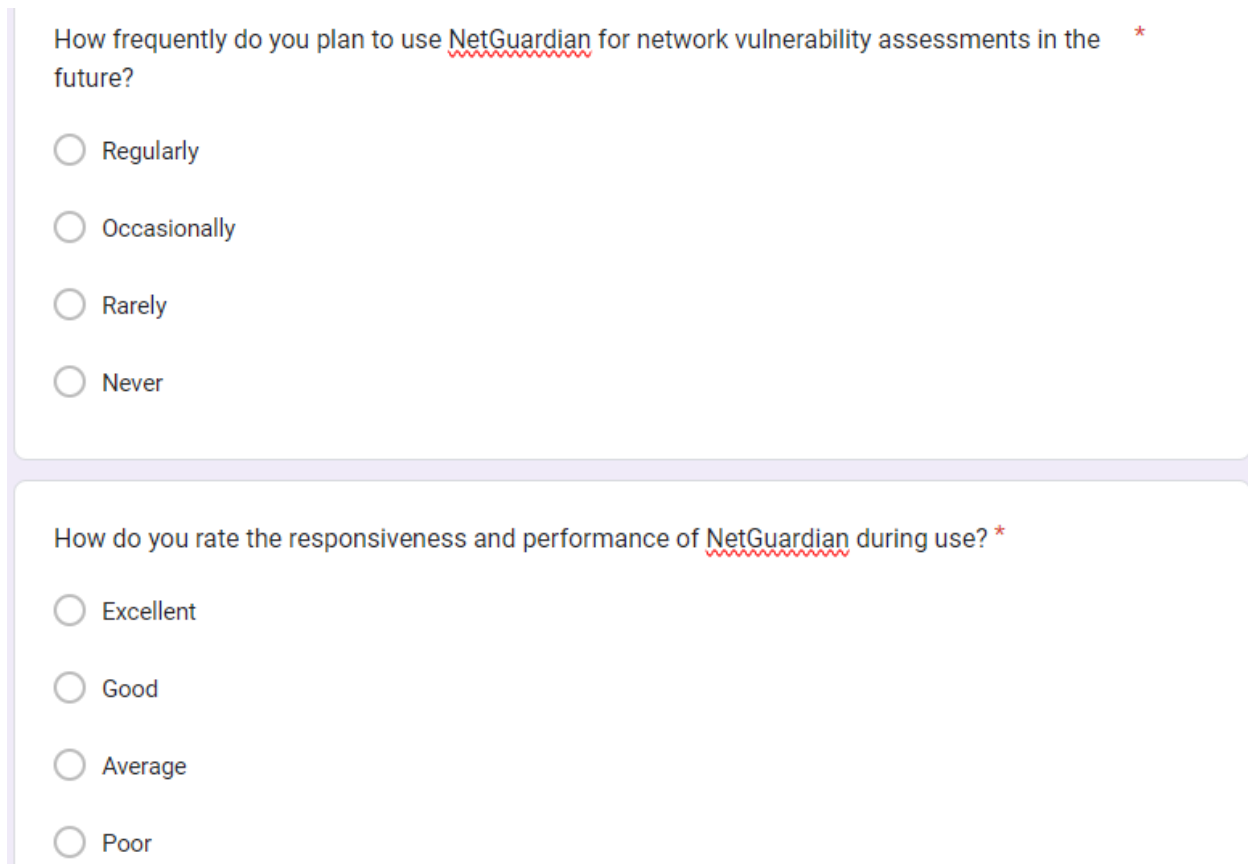
☐ Definitely

☐ Probably

☐ Not sure

☐ Probably not

Figure 175: User Feedback Form Part 4.



How frequently do you plan to use NetGuardian for network vulnerability assessments in the future? *

☐ Regularly

☐ Occasionally

☐ Rarely

☐ Never

How do you rate the responsiveness and performance of NetGuardian during use? *

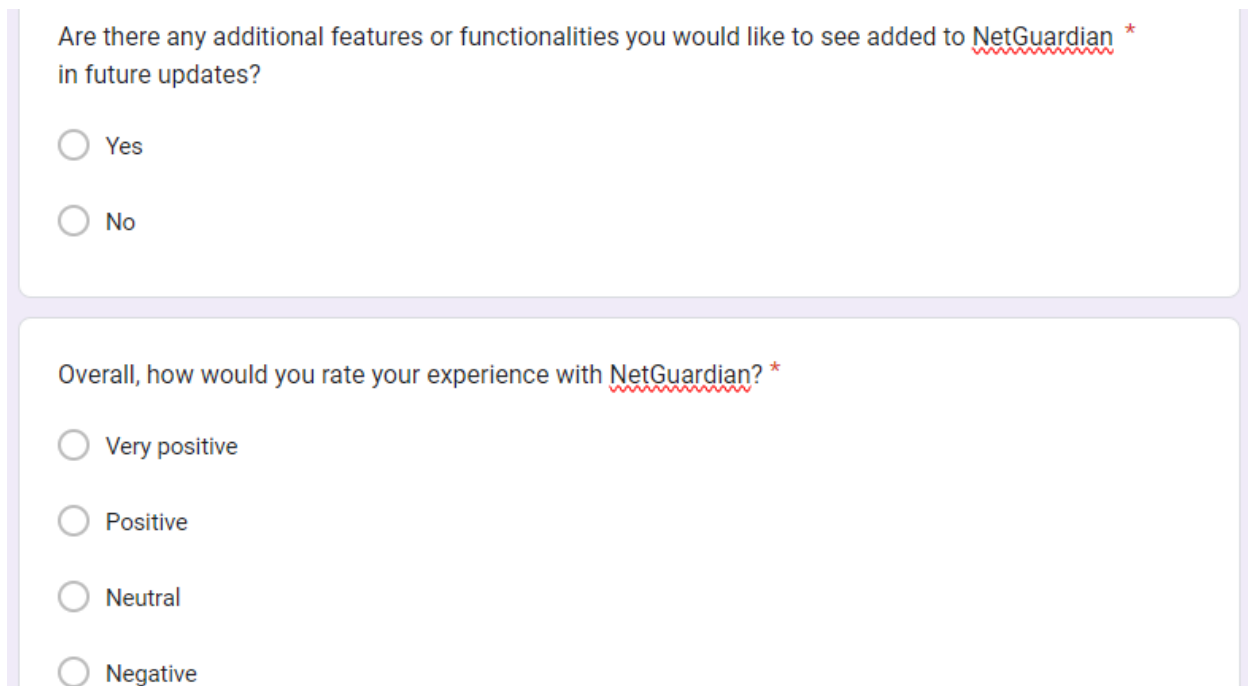
☐ Excellent

☐ Good

☐ Average

☐ Poor

Figure 176: User Feedback Form Part 5.



Are there any additional features or functionalities you would like to see added to NetGuardian in future updates? *

☐ Yes

☐ No

Overall, how would you rate your experience with NetGuardian? *

☐ Very positive

☐ Positive

☐ Neutral

☐ Negative

Figure 177: User Feedback Form Part 6.

8.7.2 Sample of the Filled Feedback Forms

User Feedback Form

Thank you for using NetGuardian! Your feedback is invaluable to us as we strive to enhance our network vulnerability assessment tool. Please take a moment to provide us with your insights.

* Indicates required question

Name: *

Dip Parajuli

Email: *

parajulidip98@gmail.com

How would you rate the overall usability of the NetGuardian tool? *

☐ Excellent

☒ Good

☐ Fair

☐ Poor

Figure 178: Filled Feedback Forms Part 1.

Which features of NetGuardian did you find most useful for managing network security? *

- ☐ User authentication
- ☐ Network scanning
- ☒ Vulnerability detection
- ☐ Report generation

How would you rate the effectiveness of NetGuardian in identifying potential network vulnerabilities? *

- ☐ Very effective
- ☒ Effective
- ☐ Somewhat effective
- ☐ Ineffective

Did you encounter any challenges or difficulties while using NetGuardian? *

- ☒ Yes
- ☐ No

Figure 179: Filled Feedback Forms Part 2.

How satisfied are you with the level of support and assistance provided during your use of NetGuardian *

- ☐ Very satisfied
- ☐ Satisfied
- ☒ Neutral
- ☐ Dissatisfied
-

Would you recommend NetGuardian to other small businesses for managing network security? *

- ☒ Definitely
- ☐ Probably
- ☐ Not sure
- ☐ Probably not

Figure 180: Filled Feedback Forms Part 3.

How frequently do you plan to use NetGuardian for network vulnerability assessments in the future? *

- ☒ Regularly
- ☐ Occasionally
- ☐ Rarely
- ☐ Never

How do you rate the responsiveness and performance of NetGuardian during use? *

- ☐ Excellent
- ☒ Good
- ☐ Average
- ☐ Poor

Figure 181: Filled Feedback Forms Part 4.

Are there any additional features or functionalities you would like to see added to NetGuardian in future updates? *

☒ Yes

☐ No

Overall, how would you rate your experience with NetGuardian? *

☐ Very positive

☒ Positive

☐ Neutral

☐ Negative

Figure 182: Filled Feedback Forms Part 5.

8.8 Appendix H: Future Work

8.8.1 Readings for the Future Work

What is User and Entity Behavior Analytics? (UEBA)

User and entity behavior analytics (UEBA), also known as user behavior analytics (UBA), is the process of gathering insight into the network events that users generate every day. Once collected and analyzed, it can be used to [detect the use of compromised credentials](#) [\[1\]](#), lateral movement, and other malicious behavior.

The Gartner Market Guide added 'Entity' to User Behavior Analytics due to increasing threats from external forces, rather than just individual users. These external forces include, but are not limited to, routers, servers, applications, and other network devices that could possibly be compromising.

In summary, these other types of behavior analytics deviate from traditional consumer behavioral analytics to focus on the behavior of systems and the user accounts on them.

How UEBA Enables Faster Threat Detection

Today's networks gather endless amounts of information, especially with users moving seamlessly between IPs, assets, cloud services, and mobile devices. UBA focuses on user activity as opposed to static threat indicators, meaning it can detect attacks that haven't been mapped to threat intelligence and alert on malicious behavior earlier in an attack.

As networks have become more complex, it's become easier than ever to successfully infiltrate a corporate network and masquerade as an internal employee, circumventing external defenses. If an attacker is able to penetrate a network and remain there undetected, they can repeatedly steal sensitive data and cause monetary damage.

[User Behavior Analytics](#) exposes stealthy, attacker activities by uncovering patterns in user behavior to identify what's "normal" behavior, and what may be evidence of intruder compromise, insider threats, or risky behavior on a network.

Figure 183: User and Entity Behavior Analytics (*Rapid7, 2024*).

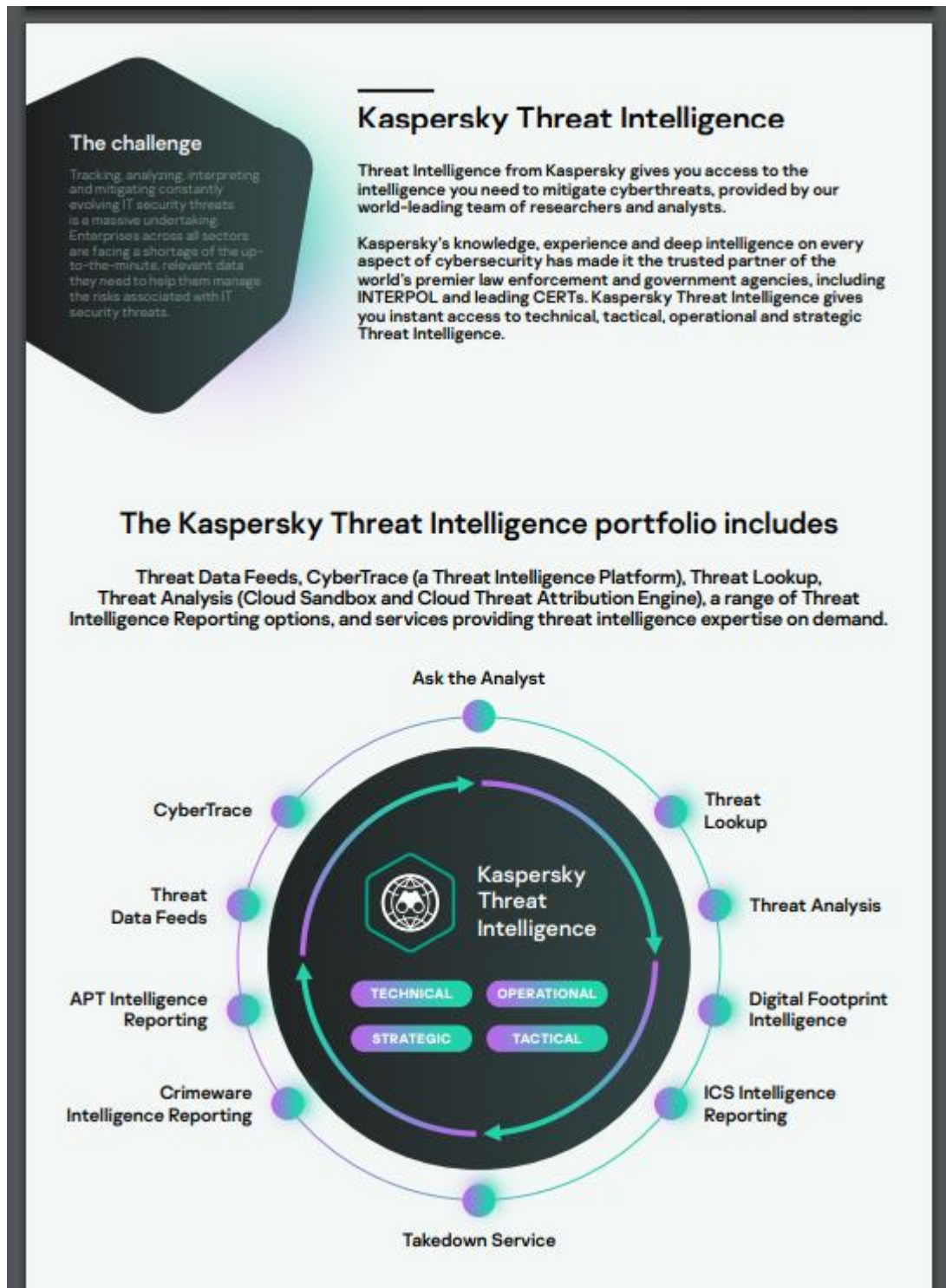


Figure 184: Threat Intelligence (Kaspersky, 2022).

There are often circumstances where software is compiled and packaged into artifacts that must function on multiple [operating systems \(OS\)](#) and [processor architectures](#). It is almost impossible to execute an application on a different OS/architecture platform than the one it was designed for. That's why it's a common practice to build releases for many different platforms. This can be difficult to accomplish when the platform you are using to build artifacts is different from the platforms you want to target for deployment. For instance, developing an application on Windows and deploying it to Linux and macOS machines involves provisioning and configuring build machines for each of the operating systems and architecture platforms you're targeting. Multi-OS builds within pipelines can be achieved using a variety of techniques, but due to the stringent characteristics of processor architectures, artifacts must be compiled and produced on the same hardware that they are targeting.

[Docker](#) is a modern way to package applications into immutable and deployable artifacts in the form of [Docker images](#) and [containers](#). As with traditional artifact packaging, Docker images also experience the same processor architecture build constraints. Docker images must also be built on the hardware architectures they're intended to run on. In this post I'll discuss how to build Docker images within [CI pipelines](#) that target multiple processor architectures such as linux/amd64, linux/arm64, linux/riscv64, etc.

Figure 185: Docker (Rivera, 2020).

8.9 Feature Changing Approval

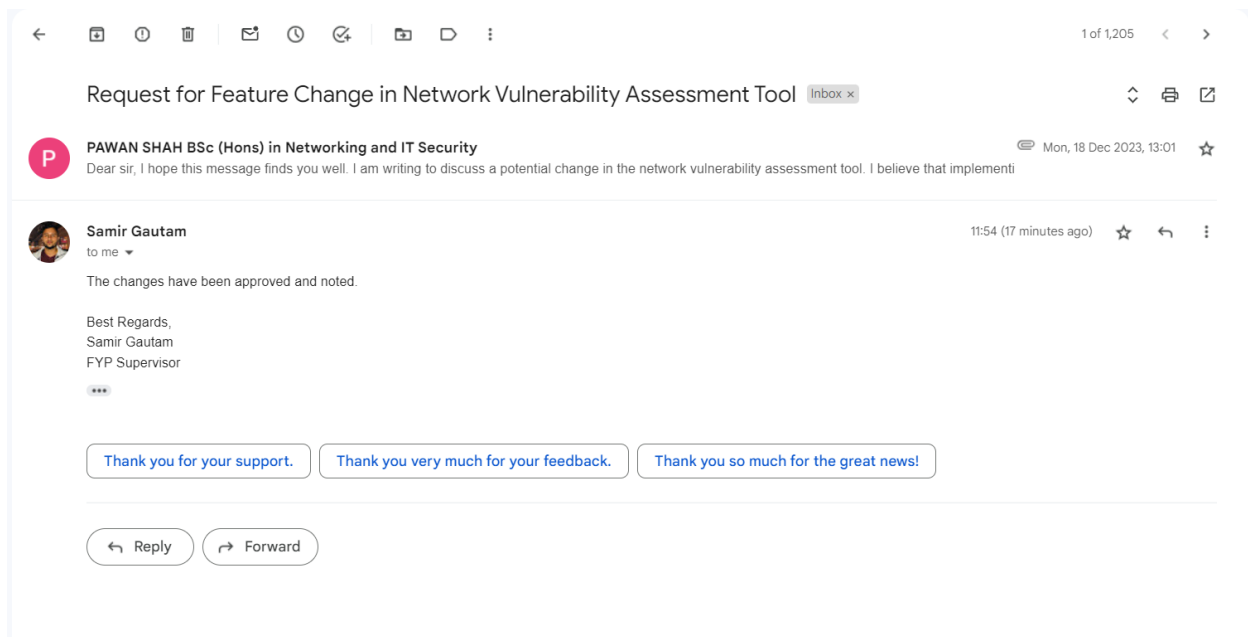


Figure 186: Feature Changing Approval.

8.10 Client Approval Letter



Figure 187: Client Approval Letter.